

分布式消息队列RocketMQ

主讲: Reythor雷(老雷)

第1章 RocketMQ概述

一、MQ概述

1、MQ简介

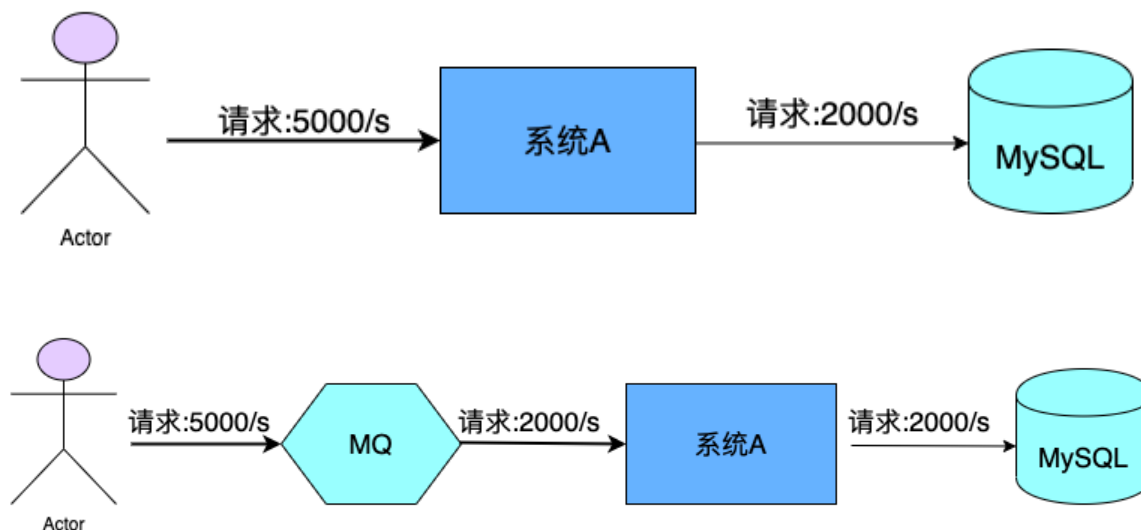
MQ, Message Queue, 是一种提供[消息队列服务](#)的中间件, 也称为消息中间件, 是一套提供了消息生产、存储、消费全过程API的软件系统。消息即数据。一般消息的体量不会很大。

2、MQ用途

从网上可以查看到很多的关于MQ用途的叙述, 但总结起来其实就以下三点。

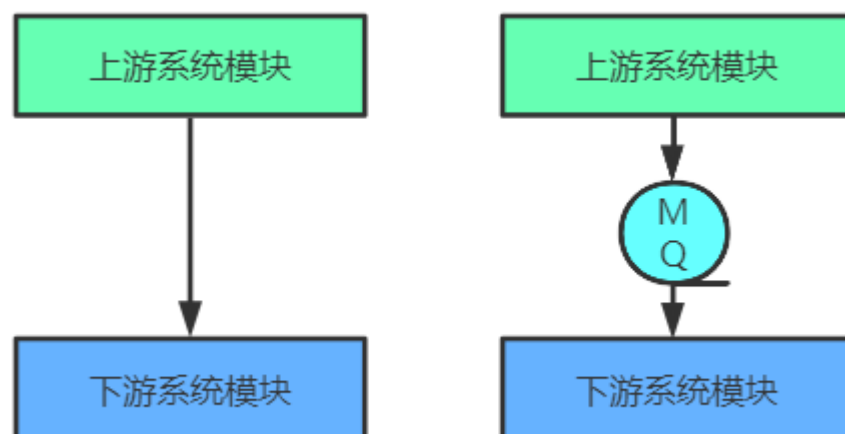
限流削峰

MQ可以将系统的[超量](#)请求暂存其中, 以便系统后期可以慢慢进行处理, 从而避免了请求的丢失或系统被压垮。



异步解耦

上游系统对下游系统的调用若为同步调用，则会大大降低系统的吞吐量与并发度，且系统耦合度太高。而异步调用则会解决这些问题。所以两层之间若要实现由同步到异步的转化，一般性做法就是，在这两层间添加一个MQ层。



数据收集

分布式系统会产生海量级数据流，如：业务日志、监控数据、用户行为等。针对这些数据流进行实时或批量采集汇总，然后对这些数据流进行大数据分析，这是当前互联网平台的必备技术。通过MQ完成此类数据收集是最好的选择。

3、常见MQ产品

ActiveMQ

ActiveMQ是使用Java语言开发一款MQ产品。早期很多公司与项目中都在使用。但现在的社区活跃度已经很低。现在的项目中已经很少使用了。

RabbitMQ

RabbitMQ是使用ErLang语言开发的一款MQ产品。其吞吐量较Kafka与RocketMQ要低，且由于其不是Java语言开发，所以公司内部对其实现定制化开发难度较大。

Kafka

Kafka是使用Scala/Java语言开发的一款MQ产品。其最大的特点就是高吞吐率，常用于大数据领域的实时计算、日志采集等场景。其没有遵循任何常见的MQ协议，而是使用自研协议。对于Spring Cloud Netflix，其仅支持RabbitMQ与Kafka。

RocketMQ

RocketMQ是使用Java语言开发的一款MQ产品。经过数年阿里双11的考验，性能与稳定性非常高。其没有遵循任何常见的MQ协议，而是使用自研协议。对于Spring Cloud Alibaba，其支持RabbitMQ、Kafka，但提倡使用RocketMQ。

对比

关键词	ACTIVEMQ	RABBITMQ	KAFKA	ROCKETMQ
开发语言	Java	ErLang	Java	Java
单机吞吐量	万级	万级	十万级	十万级
Topic	-	-	百级Topic时会影响系统吞吐量	千级Topic时会影响系统吞吐量
社区活跃度	低	高	高	高

4、MQ常见协议

一般情况下MQ的实现是要遵循一些常规性协议的。常见的协议如下：

JMS

JMS, Java Messaging Service (Java消息服务)。是Java平台上有关MOM (Message Oriented Middleware, 面向消息的中间件 PO/OO/AO) 的技术规范, 它便于消息系统中的Java应用程序进行消息交换, 并且通过提供标准的产生、发送、接收消息的接口, 简化企业应用的开发。ActiveMQ是该协议的典型实现。

STOMP

STOMP, Streaming Text Orientated Message Protocol (面向流文本的消息协议), 是一种MOM设计的简单文本协议。STOMP提供一个可互操作的连接格式, 允许客户端与任意STOMP消息代理 (Broker) 进行交互。ActiveMQ是该协议的典型实现, RabbitMQ通过插件可以支持该协议。

AMQP

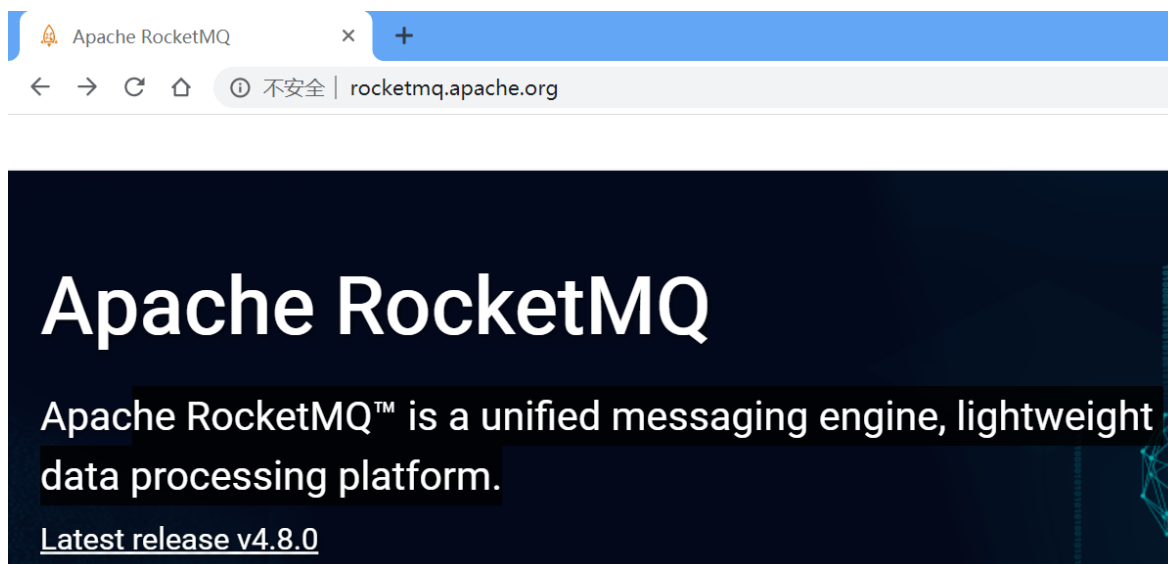
AMQP, Advanced Message Queuing Protocol (高级消息队列协议), 一个提供统一消息服务的应用层标准, 是应用层协议的一个开放标准, 是一种MOM设计。基于此协议的客户端与消息中间件可传递消息, 并不受客户端/中间件不同产品, 不同开发语言等条件的限制。RabbitMQ是该协议的典型实现。

MQTT

MQTT, Message Queuing Telemetry Transport (消息队列遥测传输), 是IBM开发的一个即时通讯协议, 是一种二进制协议, 主要用于服务器和低功耗IoT (物联网) 设备间的通信。该协议支持所有平台, 几乎可以把所有联网物品和外部连接起来, 被用来当做传感器和致动器的通信协议。RabbitMQ通过插件可以支持该协议。

二、RocketMQ概述

1、RocketMQ简介



RocketMQ是一个统一消息引擎、轻量级数据处理平台。

RocketMQ是一款阿里巴巴开源的消息中间件。2016年11月28日，阿里巴巴向 Apache 软件基金会捐赠 RocketMQ，成为 Apache 孵化项目。2017 年 9 月 25 日，Apache 宣布 RocketMQ 孵化成为 Apache 顶级项目（TLP），成为国内首个互联网中间件在 Apache 上的顶级项目。

官网地址：<http://rocketmq.apache.org>

2、RocketMQ发展历程



2007年，阿里开始五彩石项目，Notify作为项目中交易核心消息流转系统，应运而生。Notify系统是RocketMQ的雏形。

2010年，B2B大规模使用ActiveMQ作为阿里的消息内核。阿里急需一个具有海量堆积能力的消息系统。

2011年初，Kafka开源。淘宝中间件团队在对Kafka进行了深入研究后，开发了一款新的MQ，MetaQ。

2012年，MetaQ发展到了v3.0版本，在它基础上进行了进一步的抽象，形成了RocketMQ，然后就将其进行了开源。

2015年，阿里在RocketMQ的基础上，又推出了一款专门针对阿里云上用户的消息系统Aliware MQ。

2016年双十一，RocketMQ承载了**万亿级**消息的流转，跨越了一个新的里程碑。11月28日，阿里巴巴向 Apache 软件基金会捐赠 RocketMQ，成为 Apache 孵化项目。

2017年9月25日，Apache 宣布 RocketMQ孵化成为 Apache 顶级项目（TLP），成为国内首个互联网中间件在 Apache 上的顶级项目。

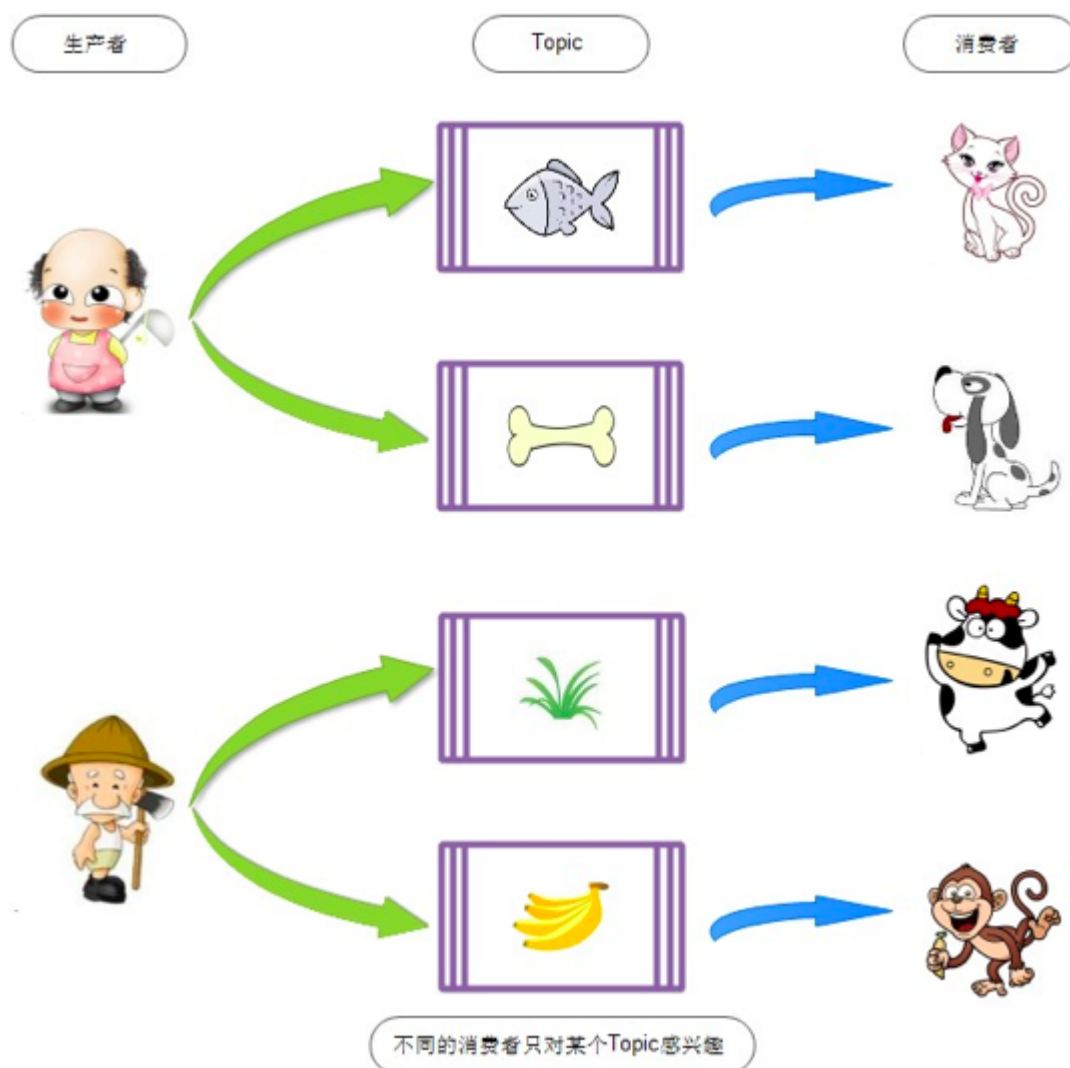
第2章 RocketMQ的安装与启动

一、基本概念

1 消息 (Message)

消息是指，消息系统所传输信息的物理载体，生产和消费数据的最小单位，每条消息必须属于一个主题。

2 主题 (Topic)



Topic表示一类消息的集合，每个主题包含若干条消息，每条消息只能属于一个主题，是RocketMQ进行消息订阅的基本单位。 topic:message 1:n message:topic 1:1

一个生产者可以同时发送多种Topic的消息；而一个消费者只对某种特定的Topic感兴趣，即只可以订阅和消费一种Topic的消息。 producer:topic 1:n consumer:topic 1:1

3 标签 (Tag)

为消息设置的标签，用于同一主题下区分不同类型的消息。来自同一业务单元的消息，可以根据不同业务目的在同一主题下设置不同标签。标签能够有效地保持代码的清晰度和连贯性，并优化RocketMQ提供的查询系统。消费者可以根据Tag实现对不同子主题的不同消费逻辑，实现更好的扩展性。

Topic是消息的一级分类，Tag是消息的二级分类。

Topic: 货物

tag=上海

tag=江苏

tag=浙江

----- 消费者 -----

topic=货物 tag = 上海

topic=货物 tag = 上海浙江

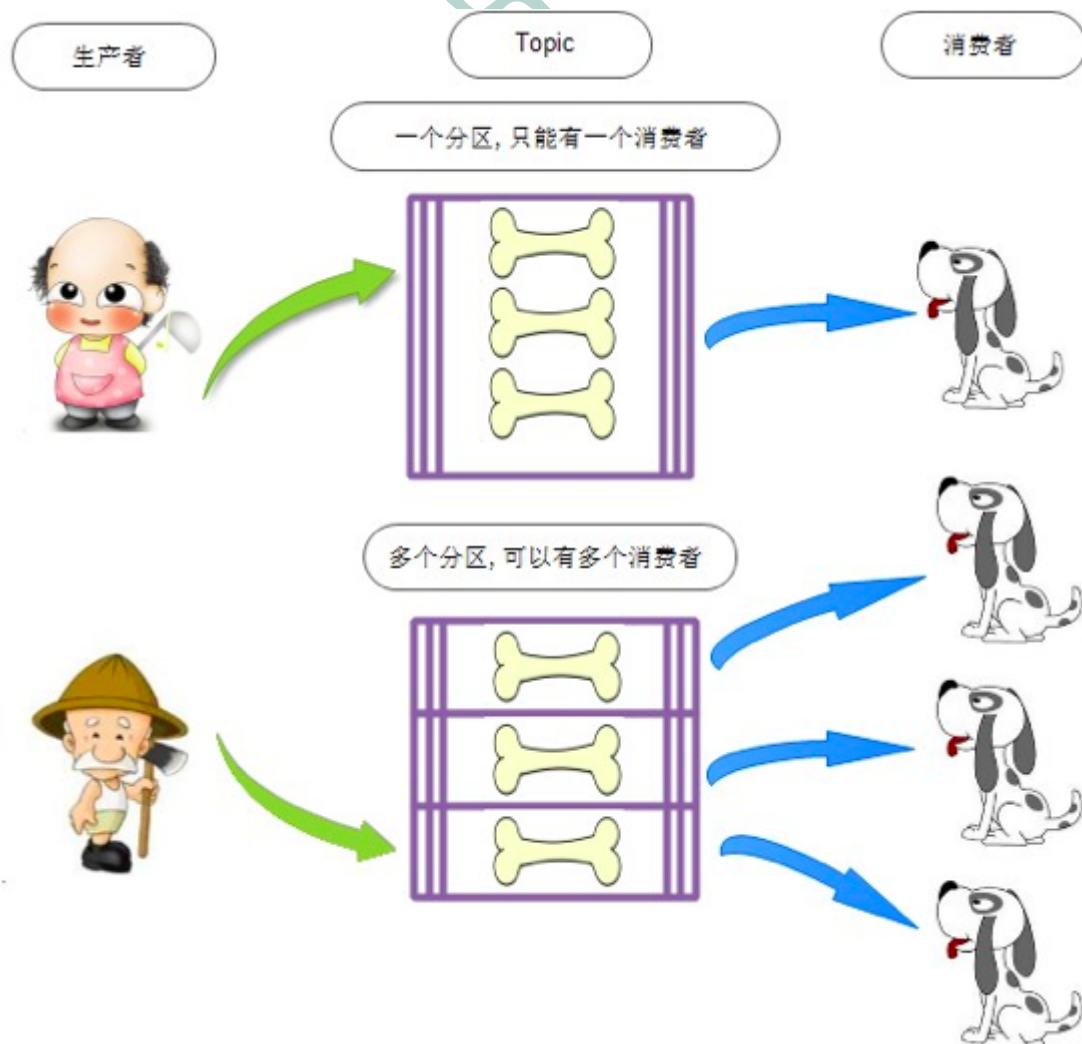
topic=货物 tag = *



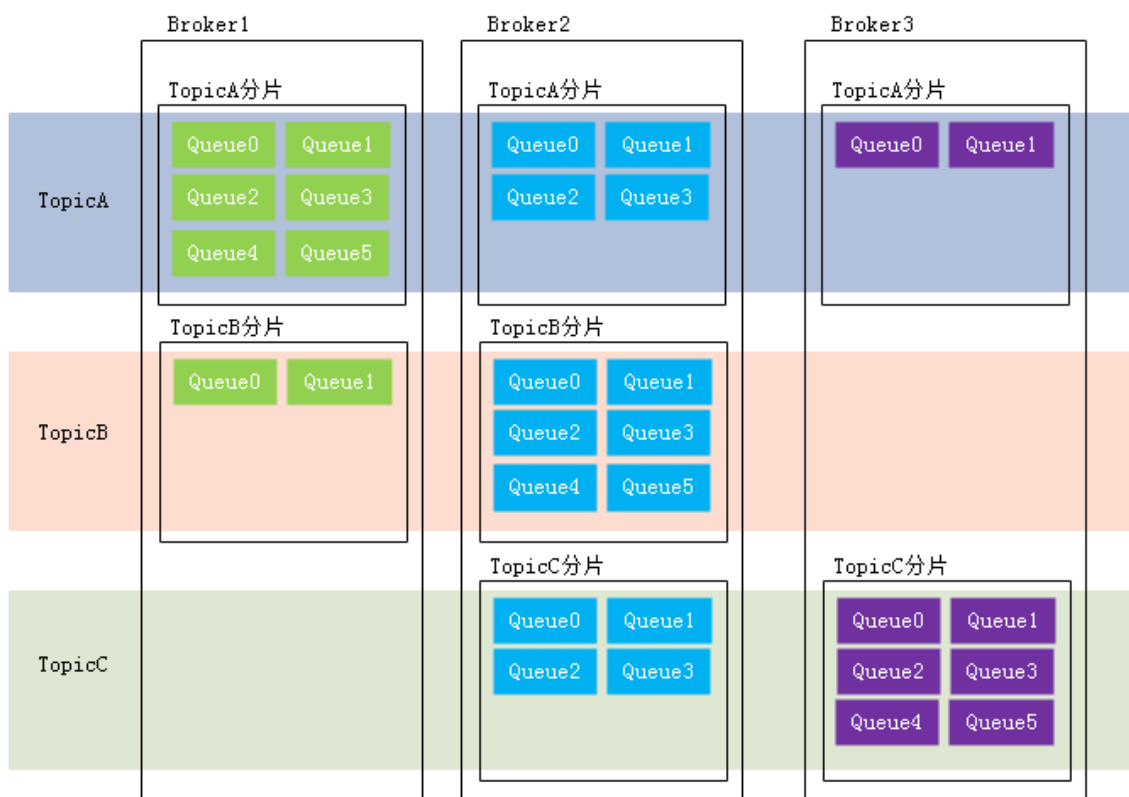
4 队列 (Queue)

存储消息的物理实体。一个Topic中可以包含多个Queue，每个Queue中存放的就是该Topic的消息。一个Topic的Queue也被称为一个Topic中消息的分区 (Partition)。

一个Topic的Queue中的消息只能被一个消费者组中的一个消费者消费。一个Queue中的消息不允许同一个消费者组中的多个消费者同时消费。



在学习参考其它相关资料时，还会看到一个概念：分片 (Sharding)。分片不同于分区。在RocketMQ中，分片指的是存放相应Topic的Broker。每个分片中会创建出相应数量的分区，即Queue，每个Queue的大小都是相同的。

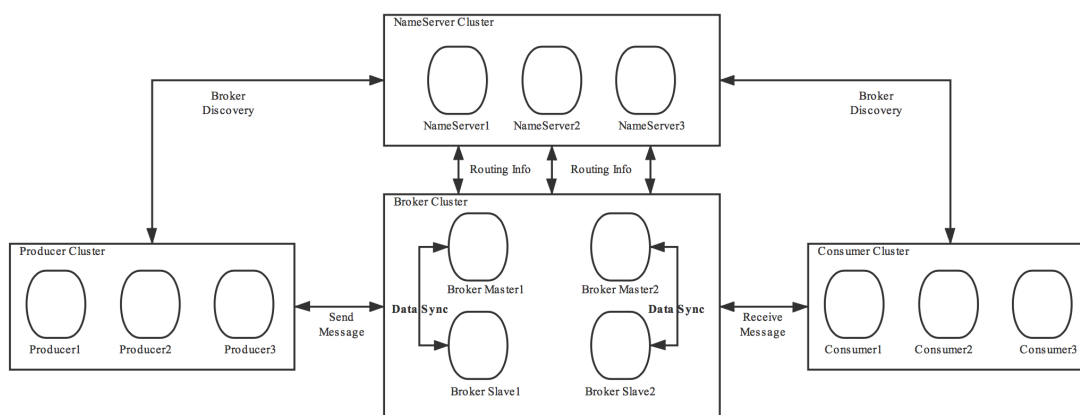


5 消息标识 (MessageId/Key)

RocketMQ中每个消息拥有唯一的MessageId，且可以携带具有业务标识的Key，以方便对消息的查询。不过需要注意的是，MessageId有两个：在生产者send()消息时会自动生成一个MessageId (msgId)，当消息到达Broker后，Broker也会自动生成一个MessageId(offsetMsgId)。msgId、offsetMsgId与key都称为消息标识。

- msgId：由producer端生成，其生成规则为：
producerIp + 进程pid + MessageClientIDSetter类的ClassLoader的hashCode + 当前时间 + AtomicInteger自增计数器
- offsetMsgId：由broker端生成，其生成规则为：brokerIp + 物理分区的offset (Queue中的偏移量)
- key：由用户指定的业务相关的唯一标识

二、系统架构



RocketMQ架构上主要分为四部分构成：

1 Producer

消息生产者，负责生产消息。Producer通过MQ的负载均衡模块选择相应的Broker集群队列进行消息投递，投递的过程支持快速失败并且低延迟。

例如，业务系统产生的日志写入到MQ的过程，就是消息生产的过程

再如，电商平台中用户提交的秒杀请求写入到MQ的过程，就是消息生产的过程

RocketMQ中的消息生产者都是以生产者组（Producer Group）的形式出现的。生产者组是同一类生产者的集合，这类Producer发送相同Topic类型的消息。一个生产者组可以同时发送多个主题的消息。

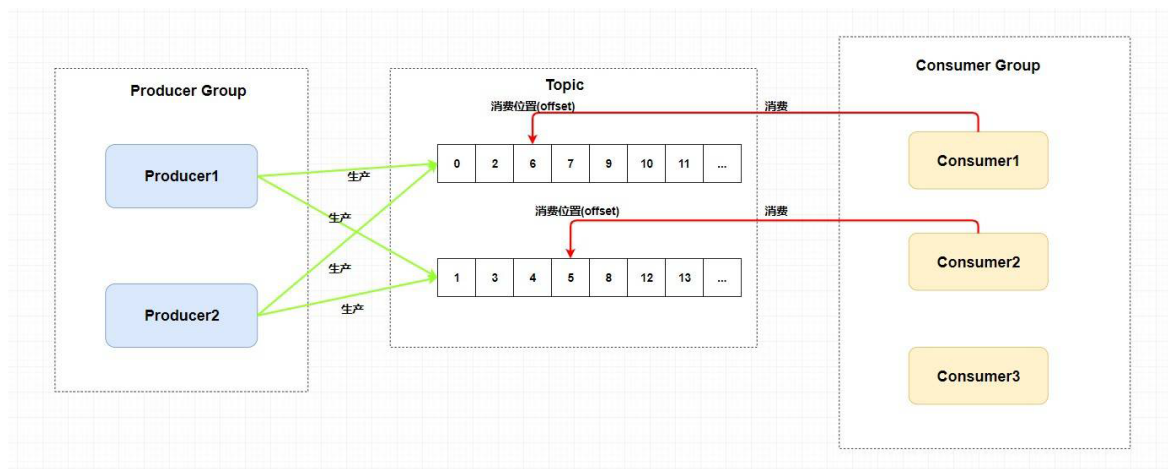
2 Consumer

消息消费者，负责消费消息。一个消息消费者会从Broker服务器中获取到消息，并对消息进行相关业务处理。

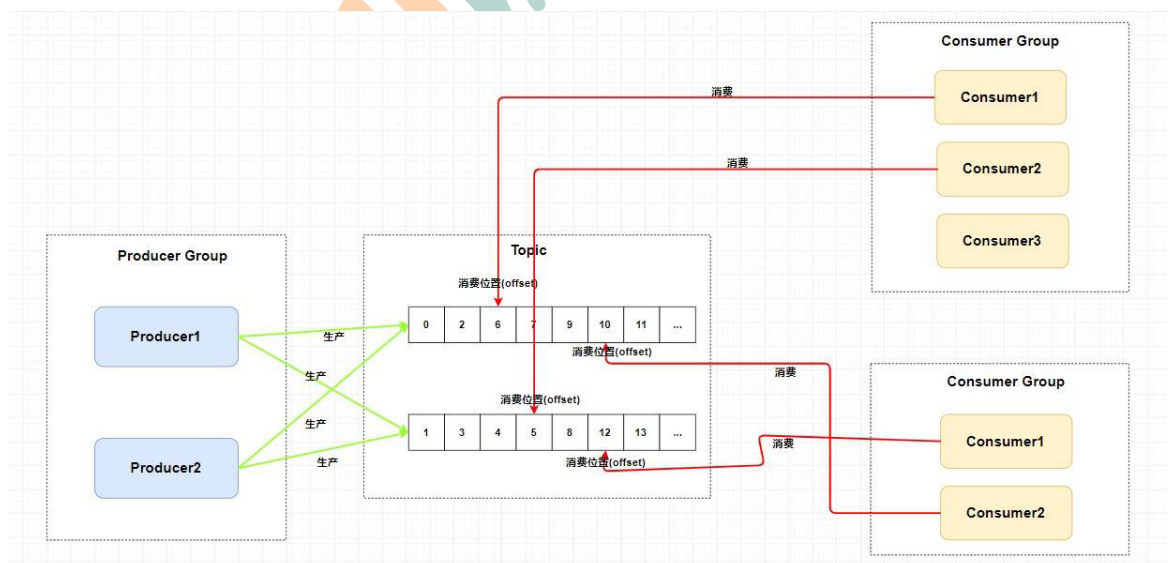
例如，QoS系统从MQ中读取日志，并对日志进行解析处理的过程就是消息消费的过程。

再如，电商平台的业务系统从MQ中读取到秒杀请求，并对请求进行处理的过程就是消息消费的过程。

RocketMQ中的消息消费者都是以消费者组（Consumer Group）的形式出现的。消费者组是同一类消费者的集合，这类Consumer消费的是同一个Topic类型的消息。消费者组使得在消息消费方面，实现**负载均衡**（将一个Topic中的不同的Queue平均分配给同一个Consumer Group的不同的Consumer，注意，并不是将消息负载均衡）和**容错**（一个Consumer挂了，该Consumer Group中的其它Consumer可以接着消费原Consumer消费的Queue）的目标变得非常容易。



消费者组中Consumer的数量应该小于等于订阅Topic的Queue数量。如果超出Queue数量，则多出的Consumer将不能消费消息。



不过，一个Topic类型的消息可以被多个消费者组同时消费。

注意，

- 1) 消费者组只能消费一个Topic的消息，不能同时消费多个Topic消息
- 2) 一个消费者组中的消费者必须订阅完全相同的Topic

3 Name Server

功能介绍

NameServer是一个Broker与Topic路由的注册中心，支持Broker的动态注册与发现。

RocketMQ的思想来自于Kafka，而Kafka是依赖于Zookeeper的。所以，在RocketMQ的早期版本，即在MetaQ v1.0与v2.0版本中，也是依赖于Zookeeper的。从MetaQ v3.0，即RocketMQ开始去掉了Zookeeper依赖，使用了自己的NameServer。

主要包括两个功能：

- **Broker管理**：接受Broker集群的注册信息并且保存下来作为路由信息的基本数据；提供心跳检测机制，检查Broker是否还存活。
- **路由信息管理**：每个NameServer中都保存着Broker集群的整个路由信息和用于客户端查询的队列信息。Producer和Consumer通过NameServer可以获取整个Broker集群的路由信息，从而进行消息的投递和消费。

路由注册

NameServer通常也是以集群的方式部署，不过，NameServer是无状态的，即NameServer集群中的各个节点间是无差异的，各节点间相互不进行信息通讯。那各节点中的数据是如何进行数据同步的呢？在Broker节点启动时，轮询NameServer列表，与每个NameServer节点建立长连接，发起注册请求。在NameServer内部维护着一个Broker列表，用来动态存储Broker的信息。

注意，这是与其它像zk、Eureka、Nacos等注册中心不同的地方。

这种NameServer的无状态方式，有什么优缺点：

优点：NameServer集群搭建简单，扩容简单。

缺点：对于Broker，必须明确指出所有NameServer地址。否则未指出的将不会去注册。也正因为如此，NameServer并不能随便扩容。因为，若Broker不重新配置，新增的NameServer对于Broker来说是不可见的，其不会向这个NameServer进行注册。

Broker节点为了证明自己是活着的，为了维护与NameServer间的长连接，会将最新的信息以**心跳包**的方式上报给NameServer，每30秒发送一次心跳。心跳包中包含 BrokerId、Broker地址(IP+Port)、Broker名称、Broker所属集群名称等等。NameServer在接收到心跳包后，会更新心跳时间戳，记录这个Broker的最新存活时间。

路由剔除

由于Broker关机、宕机或网络抖动等原因，NameServer没有收到Broker的心跳，NameServer可能会将其从Broker列表中剔除。

NameServer中有一个定时任务，每隔10秒就会扫描一次Broker表，查看每一个Broker的最新心跳时间戳距离当前时间是否超过120秒，如果超过，则会判定Broker失效，然后将其从Broker列表中剔除。

扩展：对于RocketMQ日常运维工作，例如Broker升级，需要停掉Broker的工作。OP需要怎么做？

OP需要将Broker的读写权限禁掉。一旦client(Consumer或Producer)向broker发送请求，都会收到broker的NO_PERMISSION响应，然后client会进行对其它Broker的重试。

当OP观察到这个Broker没有流量后，再关闭它，实现Broker从NameServer的移除。

OP：运维工程师

SRE：Site Reliability Engineer，现场可靠性工程师

路由发现

RocketMQ的路由发现采用的是Pull模型。当Topic路由信息出现变化时，NameServer不会主动推送给客户端，而是客户端定时拉取主题最新的路由。默认客户端每30秒会拉取一次最新的路由。

扩展：

1) Push模型：推送模型。其实时性较好，是一个“发布-订阅”模型，需要维护一个长连接。而长连接的维护是需要资源成本的。该模型适合于的场景：

- 实时性要求较高
- Client数量不多，Server数据变化较频繁

2) Pull模型：拉取模型。存在的问题是，实时性较差。

3) Long Polling模型：长轮询模型。其是对Push与Pull模型的整合，充分利用了这两种模型的优势，屏蔽了它们的劣势。

客户端NameServer选择策略

这里的客户端指的是Producer与Consumer

客户端在配置时必须写上NameServer集群的地址，那么客户端到底连接的是哪个NameServer节点呢？客户端首先会生产一个随机数，然后再与NameServer节点数量取模，此时得到的就是所要连接的节点索引，然后就会进行连接。如果连接失败，则会采用round-robin策略，逐个尝试着去连接其它节点。

首先采用的是 **随机策略** 进行的选择，失败后采用的是 **轮询策略**。

扩展：Zookeeper Client是如何选择Zookeeper Server的？

简单来说就是，经过两次Shuffle，然后选择第一台Zookeeper Server。

详细说就是，将配置文件中的zk server地址进行第一次shuffle，然后随机选择一个。这个选择出的一般都是一个hostname。然后获取到该hostname对应的所有ip，再对这些ip进行第二次shuffle，从shuffle过的结果中取第一个server地址进行连接。

4 Broker

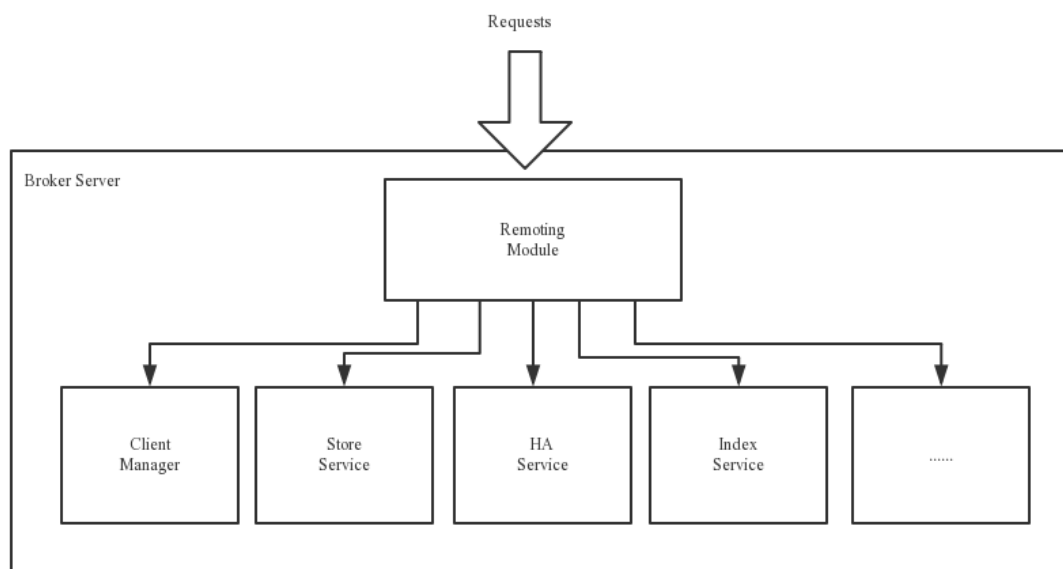
功能介绍

Broker充当着消息中转角色，负责存储消息、转发消息。Broker在RocketMQ系统中负责接收并存储从生产者发送来的消息，同时为消费者的拉取请求作准备。Broker同时也存储着消息相关的元数据，包括消费者组消费进度偏移offset、主题、队列等。

Kafka 0.8版本之后，offset是存放在Broker中的，之前版本是存放在Zookeeper中的。

模块构成

下图为Broker Server的功能模块示意图。



Remoting Module: 整个Broker的实体，负责处理来自clients端的请求。而这个Broker实体则由以下模块构成。

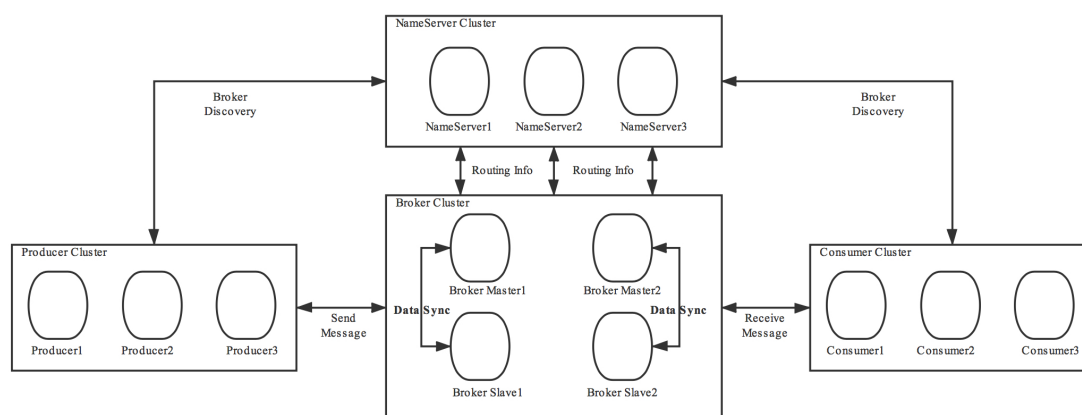
Client Manager: 客户端管理器。负责接收、解析客户端(Producer/Consumer)请求，管理客户端。例如，维护Consumer的Topic订阅信息

Store Service: 存储服务。提供方便简单的API接口，处理消息存储到物理硬盘和消息查询功能。

HA Service: 高可用服务，提供Master Broker 和 Slave Broker之间的数据同步功能。

Index Service: 索引服务。根据特定的Message key，对投递到Broker的消息进行索引服务，同时也提供根据Message Key对消息进行快速查询的功能。

集群部署



为了增强Broker性能与吞吐量，Broker一般都是以集群形式出现的。各集群节点中可能存放着相同Topic的不同Queue。不过，这里有个问题，如果某Broker节点宕机，如何保证数据不丢失呢？其解决方案是，将每个Broker集群节点进行横向扩展，即将Broker节点再建为一个HA集群，解决单点问题。

Broker节点集群是一个主从集群，即集群中具有Master与Slave两种角色。Master负责处理读写操作请求，Slave负责对Master中的数据进行备份。当Master挂掉了，Slave则会自动切换为Master去工作。所以这个Broker集群是主备集群。一个Master可以包含多个Slave，但一个Slave只能隶属于一个Master。Master与Slave的对应关系是通过指定相同的BrokerName、不同的BrokerId来确定的。BrokerId为0表示Master，非0表示Slave。每个Broker与NameServer集群中的所有节点建立长连接，定时注册Topic信息到所有NameServer。

5 工作流程

具体流程

- 1) 启动NameServer，NameServer启动后开始监听端口，等待Broker、Producer、Consumer连接。
- 2) 启动Broker时，Broker会与所有的NameServer建立并保持长连接，然后每30秒向NameServer定时发送心跳包。
- 3) 发送消息前，可以先创建Topic，创建Topic时需要指定该Topic要存储在哪些Broker上，当然，在创建Topic时也会将Topic与Broker的关系写入到NameServer中。不过，这步是可选的，也可以在发送消息时自动创建Topic。
- 4) Producer发送消息，启动时先跟NameServer集群中的其中一台建立长连接，并从NameServer中获取路由信息，即当前发送的Topic消息的Queue与Broker的地址（IP+Port）的映射关系。然后根据算法策略从队选择一个Queue，与队列所在的Broker建立长连接从而向Broker发消息。当然，在获取到路由信息后，Producer会首先将路由信息缓存到本地，再每30秒从NameServer更新一次路由信息。
- 5) Consumer跟Producer类似，跟其中一台NameServer建立长连接，获取其所订阅Topic的路由信息，然后根据算法策略从路由信息中获取到其所要消费的Queue，然后直接跟Broker建立长连接，开始消费其中的消息。Consumer在获取到路由信息后，同样也会每30秒从NameServer更新一次路由信息。不过不同于Producer的是，Consumer还会向Broker发送心跳，以确保Broker的存活状态。

Topic的创建模式

手动创建Topic时，有两种模式：

- 集群模式：该模式下创建的Topic在该集群中，所有Broker中的Queue数量是相同的。
- Broker模式：该模式下创建的Topic在该集群中，每个Broker中的Queue数量可以不同。

自动创建Topic时，默认采用的是Broker模式，会为每个Broker默认创建4个Queue。

读/写队列

从物理上来讲，读/写队列是同一个队列。所以，不存在读/写队列数据同步问题。读/写队列是逻辑上进行区分的概念。一般情况下，读/写队列数量是相同的。

例如，创建Topic时设置的写队列数量为8，读队列数量为4，此时系统会创建8个Queue，分别是0 1 2 3 4 5 6 7。Producer会将消息写入到这8个队列，但Consumer只会消费0 1 2 3这4个队列中的消息，4 5 6 7中的消息是不会被消费到的。

再如，创建Topic时设置的写队列数量为4，读队列数量为8，此时系统会创建8个Queue，分别是0 1 2 3 4 5 6 7。Producer会将消息写入到0 1 2 3这4个队列，但Consumer只会消费0 1 2 3 4 5 6 7这8个队列中的消息，但是4 5 6 7中是没有消息的。此时假设Consumer Group中包含两个Consumer，Consumer1消费0 1 2 3，而Consumer2消费4 5 6 7。但实际情况是，Consumer2是没有消息可消费的。

也就是说，当读/写队列数量设置不同时，总是有问题的。那么，为什么要这样设计呢？

其这样设计的目的是为了，方便Topic的Queue的扩容。

例如，原来创建的Topic中包含16个Queue，如何能够使其Queue扩容为8个，还不会丢失消息？可以动态修改写队列数量为8，读队列数量不变。此时新的消息只能写入到前8个队列，而消费都消费的却是16个队列中的数据。当发现后8个Queue中的消息消费完毕后，就可以再将读队列数量动态设置为8。整个扩容过程，没有丢失任何消息。

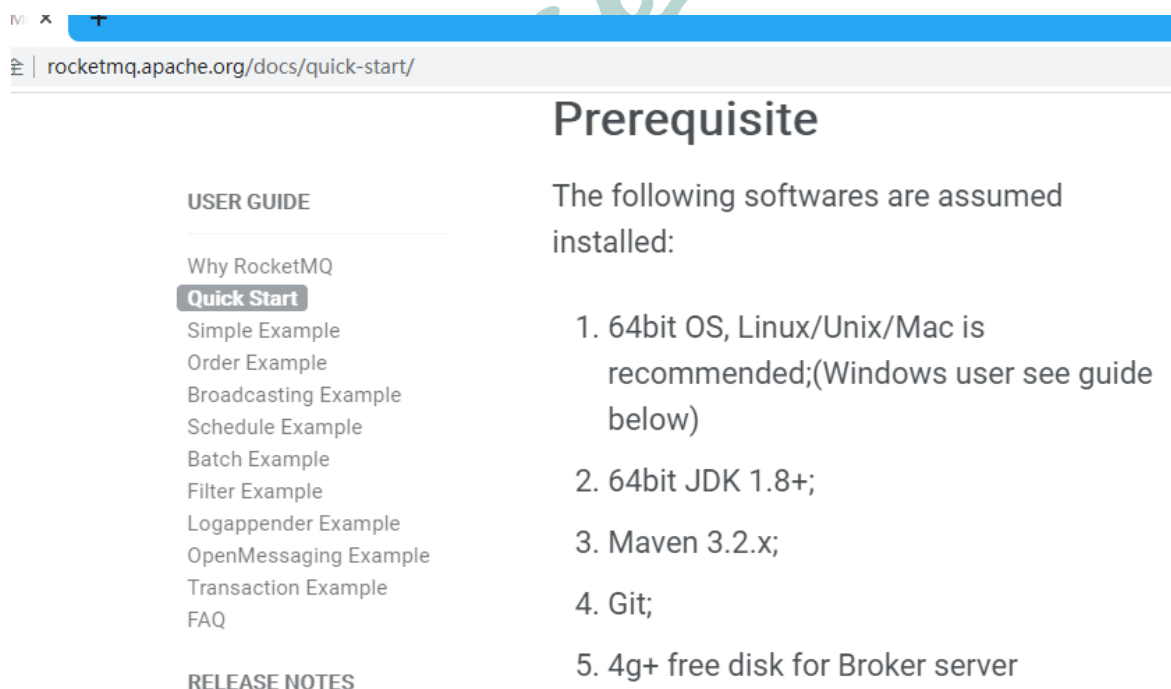
perm用于设置对当前创建Topic的操作权限：2表示只写，4表示只读，6表示读写。

三、单机安装与启动

1 准备工作

软硬件需求

系统要求是64位的，JDK要求是1.8及其以上版本的。



The screenshot shows the 'Prerequisite' section of the RocketMQ Quick Start guide. On the left, there is a sidebar with a 'Quick Start' button highlighted. The main content area lists five prerequisites: 1. 64bit OS, Linux/Unix/Mac is recommended; (Windows user see guide below) 2. 64bit JDK 1.8+; 3. Maven 3.2.x; 4. Git; 5. 4g+ free disk for Broker server.

USER GUIDE

- Why RocketMQ
- Quick Start**
- Simple Example
- Order Example
- Broadcasting Example
- Schedule Example
- Batch Example
- Filter Example
- Logappender Example
- OpenMessaging Example
- Transaction Example
- FAQ

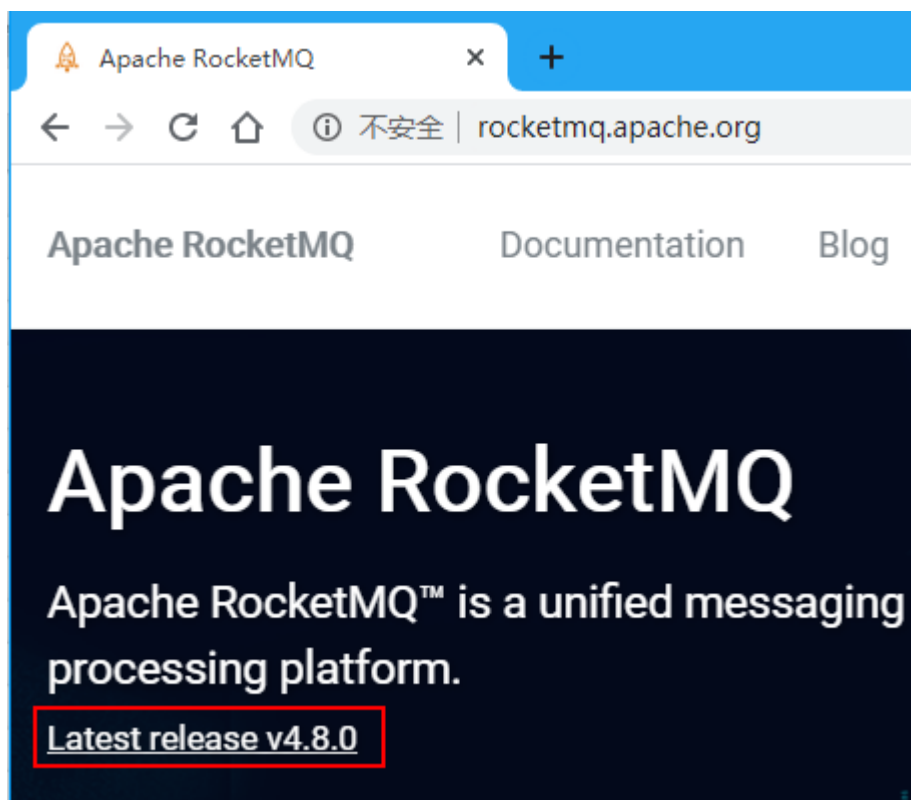
RELEASE NOTES

Prerequisite

The following softwares are assumed installed:

1. 64bit OS, Linux/Unix/Mac is recommended;(Windows user see guide below)
2. 64bit JDK 1.8+;
3. Maven 3.2.x;
4. Git;
5. 4g+ free disk for Broker server

下载RocketMQ安装包



Download the 4.8.0 release

- Source: [rocketmq-all-4.8.0-source-release.zip](#) [PGP] [SHA512]
- Binary: [rocketmq-all-4.8.0-bin-release.zip](#) [PGP] [SHA512]

将下载的安装包上传到Linux。

```
✓ mqos [x]
[root@mq0S tools]# ll
总用量 183636
-rw-r--r--. 1 root root 174157387 1月 18 2018 jdk-8u161-linux-x64.rpm
-rw-r--r-- 1 root root 13881969 1月 12 17:43 rocketmq-all-4.8.0-bin-release.zip
[root@mq0S tools]#
```

解压。

```
✓ mqos [x]
[root@mq0S rocketmq-all-4.8.0-bin-release]# ll
总用量 40
drwxr-xr-x 2 root root 102 12月 9 19:46 benchmark
drwxr-xr-x 3 root root 4096 12月 4 14:26 bin
drwxr-xr-x 6 root root 211 12月 4 14:26 conf
drwxr-xr-x 2 root root 4096 12月 9 19:46 lib
-rw-r--r-- 1 root root 17336 10月 23 10:29 LICENSE
-rw-r--r-- 1 root root 1338 12月 4 14:26 NOTICE
-rw-r--r-- 1 root root 5132 12月 4 14:26 README.md
[root@mq0S rocketmq-all-4.8.0-bin-release]#
```

2、修改初始内存

修改runserver.sh

使用vim命令打开bin/runserver.sh文件。现将这些值修改为如下：

```
✓ mqos [x]
    JAVA_OPT="${JAVA_OPT} -Xlog:gc*:file=${GC_LOG_DIR}/rmq
filecount=5,filesize=30M"
    fi
}

choose_gc_log_directory

    JAVA_OPT="${JAVA_OPT} -server -Xms256m -Xmx256m -Xmn128m -XX:
etaspaceSize=320m"
    choose_gc_options
```

修改runbroker.sh

使用vim命令打开bin/runbroker.sh文件。现将这些值修改为如下：

```
✓ mqos [x]

choose_gc_log_directory

    JAVA_OPT="${JAVA_OPT} -server -Xms256m -Xmx256m -Xmn128m"
    JAVA_OPT="${JAVA_OPT} -XX:+UseG1GC -XX:G1HeapRegionsize=16m
InitiatingHeapOccupancyPercent=30 -XX:SoftRefLRUPolicyMSPer
```

3、启动

启动NameServer

```
1 nohup sh bin/mqnamesrv &
2 tail -f ~/logs/rocketmqlogs/namesrv.log
```

```

mqos
[root@mqOS rocketmq-all-4.8.0-bin-release]# nohup sh bin/mqnamesrv &
[1] 3407
[root@mqOS rocketmq-all-4.8.0-bin-release]# nohup: 忽略输入并把输出追加到"nohup.out"
再敲一次回车
[root@mqOS rocketmq-all-4.8.0-bin-release]# tail -f ~/logs/rocketmqlogs/namesrv.log
2021-01-13 09:59:00 INFO main - tls.client.authServer = false
2021-01-13 09:59:00 INFO main - tls.client.trustCertPath = null
2021-01-13 09:59:01 INFO main - Using OpenSSL provider
2021-01-13 09:59:01 INFO main - SSLContext created for server
2021-01-13 09:59:01 INFO main - Try to start service thread:FileWatchService started:fa
ll
2021-01-13 09:59:01 INFO NettyEventExecutor - NettyEventExecutor service started
2021-01-13 09:59:01 INFO FileWatchService - FileWatchService service started
2021-01-13 09:59:01 INFO main - The Name Server boot success. serializeType=JSON
2021-01-13 10:00:01 INFO NSScheduledThread1 - -----
2021-01-13 10:00:01 INFO NSScheduledThread1 - configTable SIZE: 0

```

启动broker

```

1 nohup sh bin/mqbroker -n localhost:9876 &
2 tail -f ~/logs/rocketmqlogs/broker.log

```

```

mqos
[root@mqOS rocketmq-all-4.8.0-bin-release]# nohup sh bin/mqbroker -n localhost:9876 &
[5] 13452
[4] 退出 143 nohup sh bin/mqbroker -n localhost:9876
[root@mqOS rocketmq-all-4.8.0-bin-release]# nohup: 忽略输入并把输出追加到"nohup.out"
[root@mqOS rocketmq-all-4.8.0-bin-release]# tail -f ~/logs/rocketmqlogs/broker.log
2021-01-13 13:06:12 INFO main - Try to start service thread:FileWatchService started:false lastThread
2021-01-13 13:06:12 INFO main - Try to start service thread:PullRequestHoldService started:false last
2021-01-13 13:06:12 INFO FileWatchService - FileWatchService service started
2021-01-13 13:06:12 INFO PullRequestHoldService - PullRequestHoldService service started
2021-01-13 13:06:12 INFO main - Try to start service thread:TransactionalMessageCheckService started:
2021-01-13 13:06:12 INFO brokerOutApi_thread_1 - register broker[0]to name server localhost:9876 OK
2021-01-13 13:06:12 INFO main - The broker[mqOS, 192.168.59.160:10911] boot success. serializeType=JSO
2021-01-13 13:06:22 INFO BrokerControllerScheduledThread1 - dispatch behind commit log 0 bytes
2021-01-13 13:06:22 INFO BrokerControllerScheduledThread1 - slave fall behind master: 646007 bytes
2021-01-13 13:06:22 INFO brokerOutApi_thread_2 - register broker[0]to name server localhost:9876 OK

```

4、发送/接收消息测试

发送消息

```

1 export NAMESRV_ADDR=localhost:9876
2 sh bin/tools.sh org.apache.rocketmq.example.quickstart.Producer

```

接收消息

```

1 sh bin/tools.sh org.apache.rocketmq.example.quickstart.Consumer

```

5、关闭Server

无论是关闭name server还是broker，都是使用bin/mqshutdown命令。

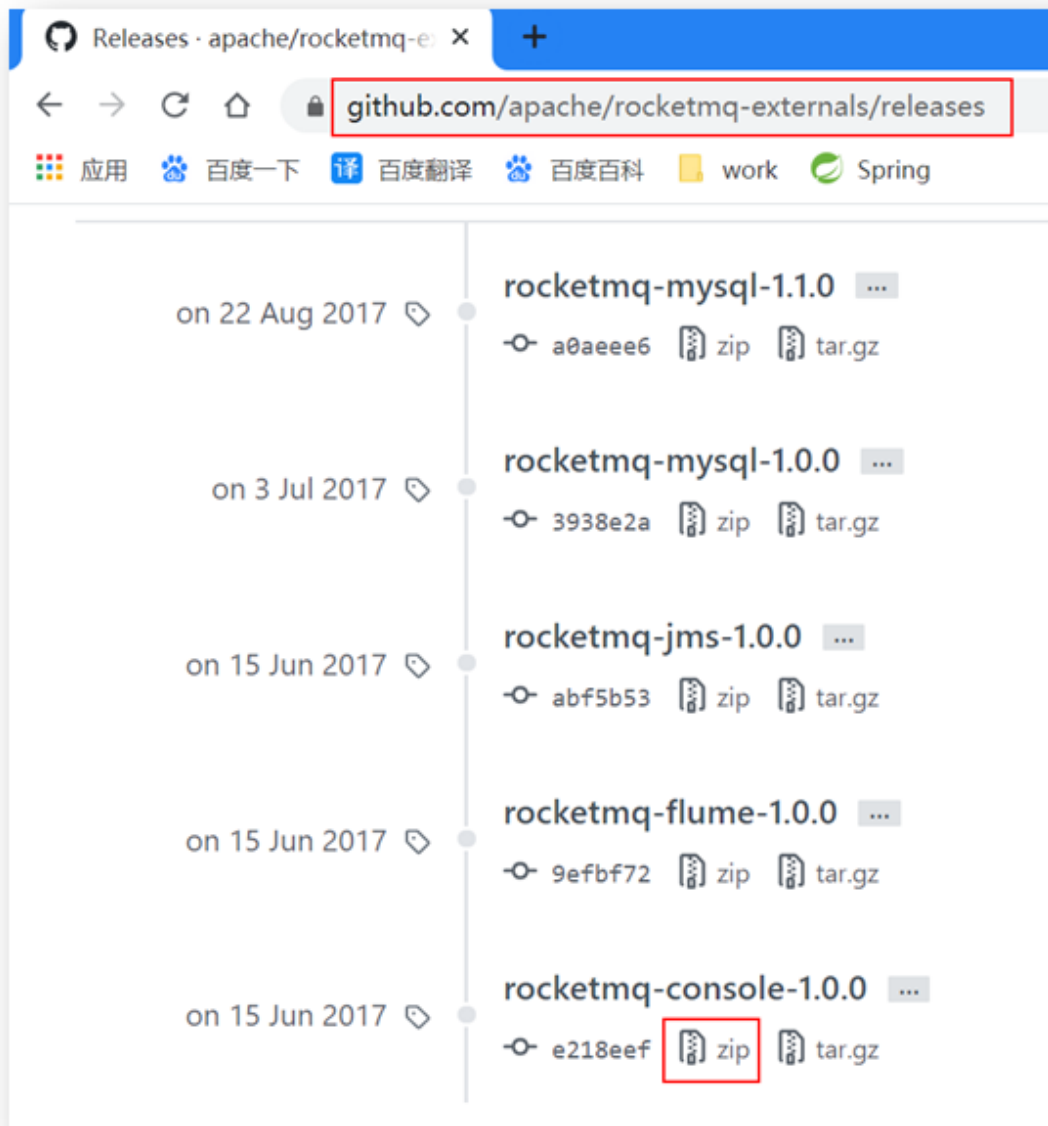
```
1 [root@mqOS rocketmq]# sh bin/mqshutdown broker
2 The mqbroker(1740) is running...
3 Send shutdown request to mqbroker(1740) OK
4
5 [root@mqOS rocketmq]# sh bin/mqshutdown namesrv
6 The mqnamesrv(1692) is running...
7 Send shutdown request to mqnamesrv(1692) OK
8 [2]+ 退出 143  nohup sh bin/mqbroker -n localhost:9876
```

四、控制台的安装与启动

RocketMQ有一个可视化的dashboard，通过该控制台可以直观的查看到很多数据。

1 下载

下载地址：<https://github.com/apache/rocketmq-externals/releases>



2 修改配置

修改其src/main/resources中的application.properties配置文件。

- 原来的端口号为8080，修改为一个不常用的
- 指定RocketMQ的name server地址



```

1 server.contextPath=
2 server.port=7000
3 #spring.application.index=true
4 spring.application.name=rocketmq-console
5 spring.http.encoding.charset=UTF-8
6 spring.http.encoding.enabled=true
7 spring.http.encoding.force=true
8 logging.config=classpath:logback.xml
9 #if this value is empty,use env value rocketmq.config.namesrvAddr
10 rocketmq.config.namesrvAddr=mqOS:9876
11 #if you use rocketmq version < 3.5.8, rocketmq.config.isVIPChannel=
12 rocketmq.config.isVIPChannel=

```

3 添加依赖

在解压目录rocketmq-console的pom.xml中添加如下JAXB依赖。

JAXB, Java Architecture for Xml Binding, 用于XML绑定的Java技术, 是一个业界标准, 是一项可以根据XML Schema生成Java类的技术。

```

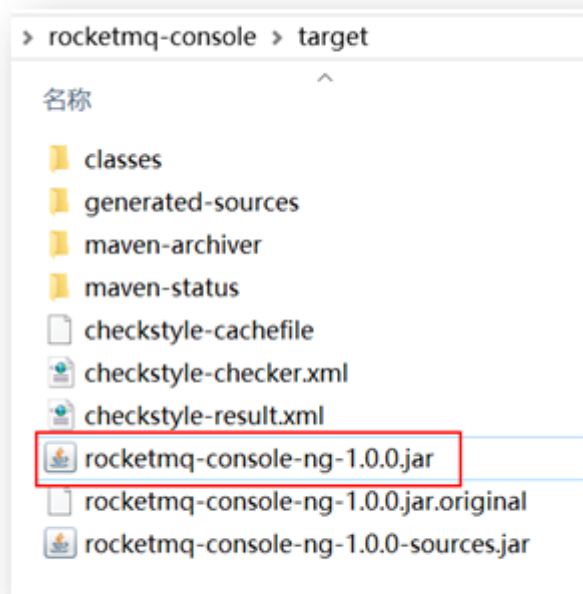
1 <dependency>
2     <groupId>javax.xml.bind</groupId>
3     <artifactId>jaxb-api</artifactId>
4     <version>2.3.0</version>
5 </dependency>
6 <dependency>
7     <groupId>com.sun.xml.bind</groupId>
8     <artifactId>jaxb-impl</artifactId>
9     <version>2.3.0</version>
10 </dependency>
11 <dependency>
12     <groupId>com.sun.xml.bind</groupId>
13     <artifactId>jaxb-core</artifactId>
14     <version>2.3.0</version>
15 </dependency>
16 <dependency>
17     <groupId>javax.activation</groupId>
18     <artifactId>activation</artifactId>
19     <version>1.1.1</version>
20 </dependency>

```

4 打包

在rocketmq-console目录下运行maven的打包命令。

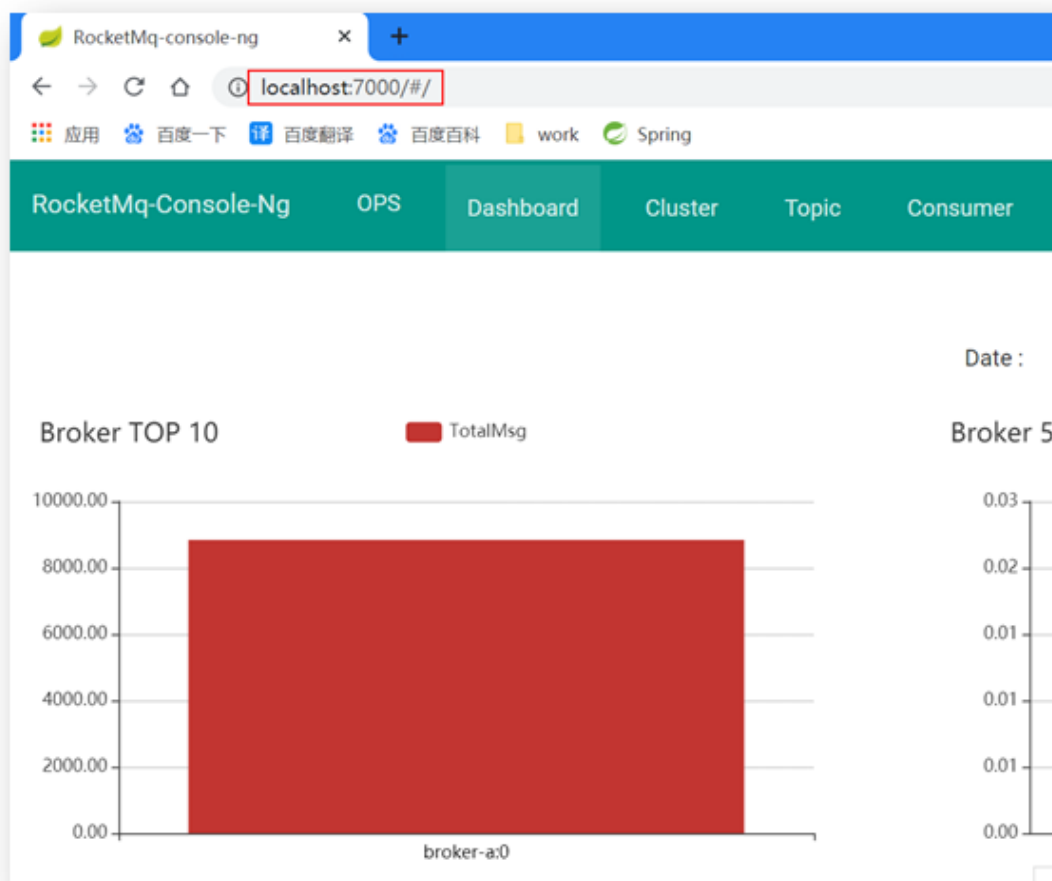
```
管理员: C:\Windows\System32\cmd.exe
D:\notes15\alibaba\rocketmq-console>mvn clean package -Dmaven.test.skip=true
```



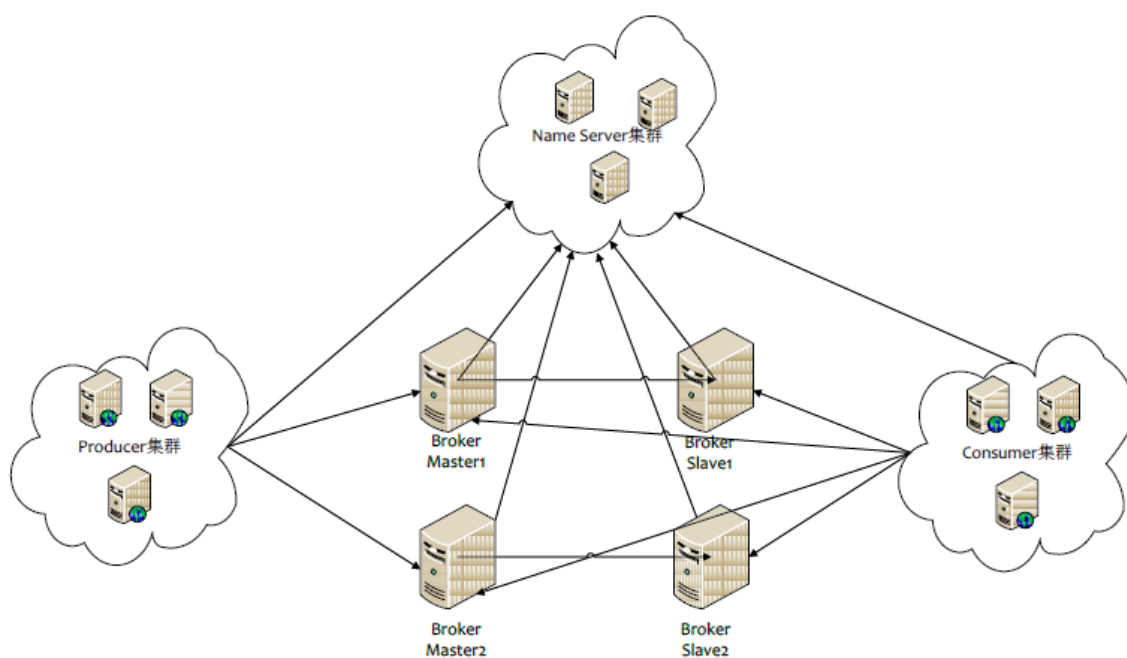
5 启动

```
管理员: C:\Windows\System32\cmd.exe
D:\notes15\alibaba\rocketmq-console\target>java -jar rocketmq-console-ng-1.0.0.jar
```

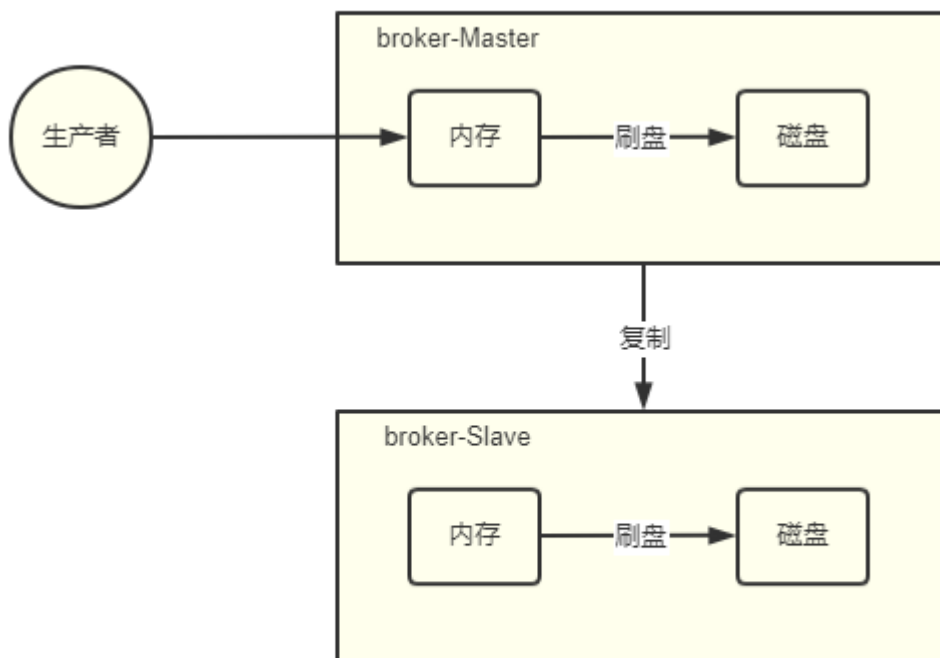
6 访问



五、集群搭建理论



1 数据复制与刷盘策略



复制与刷盘的区别示意图

复制策略

复制策略是Broker的Master与Slave间的数据同步方式。分为同步复制与异步复制：

- 同步复制：消息写入master后，master会等待slave同步数据成功后才向producer返回成功ACK
- 异步复制：消息写入master后，master立即向producer返回成功ACK，无需等待slave同步数据成功

异步复制策略会降低系统的写入延迟，RT变小，提高了系统的吞吐量

刷盘策略

刷盘策略指的是broker中消息的落盘方式，即消息发送到broker内存后消息持久化到磁盘的方式。分为同步刷盘与异步刷盘：

- 同步刷盘：当消息持久化到broker的磁盘后才算是消息写入成功。
- 异步刷盘：当消息写入到broker的内存后即表示消息写入成功，无需等待消息持久化到磁盘。

1) 异步刷盘策略会降低系统的写入延迟，RT变小，提高了系统的吞吐量

2) 消息写入到Broker的内存，一般是写入到了PageCache

3) 对于异步刷盘策略，消息会写入到PageCache后立即返回成功ACK。但并不会立即做落盘操作，而是当PageCache到达一定量时会自动进行落盘。

2 Broker集群模式

根据Broker集群中各个节点间关系的不同，Broker集群可以分为以下几类：

单Master

只有一个broker（其本质上就不能称为集群）。这种方式也只能是在测试时使用，生产环境下不能使用，因为存在单点问题。

多Master

broker集群仅由多个master构成，不存在Slave。同一Topic的各个Queue会平均分布在各个master节点上。

- 优点：配置简单，单个Master宕机或重启维护对应用无影响，在磁盘配置为RAID10时，即使机器宕机不可恢复情况下，由于RAID10磁盘非常可靠，消息也不会丢（异步刷盘丢失少量消息，同步刷盘一条不丢），性能最高；
- 缺点：单台机器宕机期间，这台机器上未被消费的消息在机器恢复之前不可订阅（不可消费），消息实时性会受到影响。

以上优点的前提是，这些Master都配置了RAID磁盘阵列。如果没有配置，一旦出现某Master宕机，则会发生大量消息丢失的情况。

多Master多Slave模式-异步复制

broker集群由多个master构成，每个master又配置了多个slave（在配置了RAID磁盘阵列的情况下，一个master一般配置一个slave即可）。master与slave的关系是主备关系，即master负责处理消息的读写请求，而slave仅负责消息的备份与master宕机后的角色切换。

异步复制即前面所讲的 [复制策略](#) 中的 [异步复制策略](#)，即消息写入master成功后，master立即向producer返回成功ACK，无需等待slave同步数据成功。

该模式的最大特点之一是，当master宕机后slave能够 [自动切换](#) 为master。不过由于slave从master的同步具有短暂的延迟（毫秒级），所以当master宕机后，这种异步复制方式可能会存在少量消息的丢失问题。

Slave从Master同步的延迟越短，其可能丢失的消息就越少

对于Master的RAID磁盘阵列，若使用的也是异步复制策略，同样也存在延迟问题，同样也可能丢失消息。但RAID阵列的秘诀是微秒级的（因为是由硬盘支持的），所以其丢失的数据量会更少。

多Master多Slave模式-同步双写

该模式是多Master多Slave模式的 [同步复制](#) 实现。所谓 [同步双写](#)，指的是消息写入master成功后，master会等待slave同步数据成功后才向producer返回成功ACK，即master与slave都要写入成功后才会返回成功ACK，也即 [双写](#)。

该模式与 [异步复制模式](#) 相比，优点是消息的安全性更高，不存在消息丢失的情况。但单个消息的RT略高，从而导致性能要略低（大约低10%）。

该模式存在一个大的问题：对于目前的版本，Master宕机后，Slave [不会自动切换](#) 到Master。

最佳实践

一般会为Master配置RAID10磁盘阵列，然后再为其配置一个Slave。即利用了RAID10磁盘阵列的高效、安全性，又解决了可能会影响订阅的问题。

1) RAID 磁盘阵列的效率要高于Master-Slave集群。因为RAID是硬件支持的。也正因为如此，所以RAID阵列的搭建成本较高。

2) 多Master+RAID阵列，与多Master多Slave集群的区别是什么？

- 多Master+RAID阵列，其仅仅可以保证数据不丢失，即不影响消息写入，但其可能会影响到消息的订阅。但其执行效率要远高于 [多Master多Slave集群](#)
- 多Master多Slave集群，其不仅可以保证数据不丢失，也不会影响消息写入。其运行效率要低于 [多Master+RAID阵列](#)

六、磁盘阵列RAID（补充）

1 RAID历史

1988年美国加州大学伯克利分校的 D. A. Patterson 教授等首次在论文“A Case of Redundant Array of Inexpensive Disks”中提出了 RAID 概念，即 [廉价冗余磁盘阵列](#)（Redundant Array of Inexpensive Disks）。由于当时大容量磁盘比较昂贵，RAID 的基本思想是将多个容量较小、相对廉价的磁盘进行有机组合，从而以较低的成本获得与昂贵大容量磁盘相当的容量、性能、可靠性。随着磁盘成本和价格的不断降低，“廉价”已经毫无意义。因此，RAID 咨询委员会（RAID Advisory Board, RAB）决定用“独立”替代“廉价”，于时 RAID 变成了 [独立磁盘冗余阵列](#)（Redundant Array of Independent Disks）。但这仅仅是名称的变化，实质内容没有改变。

内存: 32m 6.4G (IBM 10.1G)

2 RAID等级

RAID 这种设计思想很快被业界接纳，RAID 技术作为高性能、高可靠的存储技术，得到了非常广泛的应用。RAID 主要利用镜像、数据条带和数据校验三种技术来获取高性能、可靠性、容错能力和扩展性，根据对这三种技术的使用策略和组合架构，可以把 RAID 分为不同的等级，以满足不同数据应用的需求。

D. A. Patterson 等的论文中定义了 RAID0 ~ RAID6 原始 RAID 等级。随后存储厂商又不断推出 RAID7、RAID10、RAID01、RAID50、RAID53、RAID100 等 RAID 等级，但这些并无统一的标准。目前业界与学术界公认的标准是 RAID0 ~ RAID6，而在实际应用领域中使用最多的 RAID 等级是 RAID0、RAID1、RAID3、RAID5、RAID6 和 RAID10。

RAID 每一个等级代表一种实现方法和技术，等级之间并无高低之分。在实际应用中，应当根据用户的数据应用特点，综合考虑可用性、性能和成本来选择合适的 RAID 等级，以及具体的实现方式。

3 关键技术

镜像技术

镜像技术是一种冗余技术，为磁盘提供数据备份功能，防止磁盘发生故障而造成数据丢失。对于 RAID 而言，采用镜像技术最典型地的用法就是，同时在磁盘阵列中产生两个完全相同的数据副本，并且分布在两个不同的磁盘上。镜像提供了完全的数据冗余能力，当一个数据副本失效不可用时，外部系统仍可正常访问另一副本，不会对应用系统运行和性能产生影响。而且，镜像不需要额外的计算和校验，故障修复非常快，直接复制即可。镜像技术可以从多个副本进行并发读取数据，提供更高的读 I/O 性能，但不能并行写数据，写多个副本通常会导致一定的 I/O 性能下降。

镜像技术提供了非常高的数据安全性，其代价也是非常昂贵的，需要至少双倍的存储空间。高成本限制了镜像的广泛应用，主要应用于至关重要的数据保护，这种场合下的数据丢失可能会造成非常巨大的损失。

数据条带技术

数据条带化技术是一种自动将 I/O 操作负载均衡到多个物理磁盘上的技术。更具体地说就是，将一块连续的数据分成很多小部分并把它们分别存储到不同磁盘上。这就能使多个进程可以并发访问数据的多个不同部分，从而获得最大程度上的 I/O 并行能力，极大地提升性能。

数据校验技术

数据校验技术是指，RAID 要在写入数据的同时进行校验计算，并将得到的校验数据存储存储在 RAID 成员磁盘中。校验数据可以集中保存在某个磁盘或分散存储在多个不同磁盘中。当其中一部分数据出错时，就可以对剩余数据和校验数据进行反校验计算重建丢失的数据。

数据校验技术相对于镜像技术的优势在于节省大量开销，但由于每次数据读写都要进行大量的校验运算，对计算机的运算速度要求很高，且必须使用硬件 RAID 控制器。在数据重建恢复方面，校验技术比镜像技术复杂得多且慢得多。

4 RAID 分类

从实现角度看，RAID 主要分为软 RAID、硬 RAID 以及混合 RAID 三种。

软 RAID

所有功能均有操作系统和 CPU 来完成，没有独立的 RAID 控制处理芯片和 I/O 处理芯片，效率自然最低。

硬 RAID

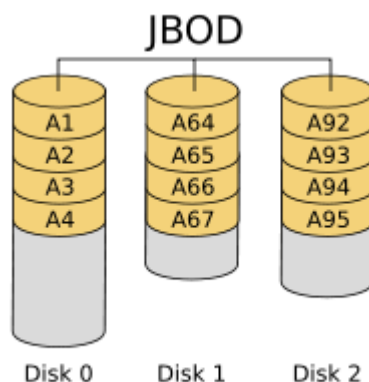
配备了专门的 RAID 控制处理芯片和 I/O 处理芯片以及阵列缓冲，不占用 CPU 资源。效率很高，但成本也很高。

混合 RAID

具备 RAID 控制处理芯片，但没有专门的I/O 处理芯片，需要 CPU 和驱动程序来完成。性能和成本在软 RAID 和硬 RAID 之间。

5 常见RAID等级详解

JBOD



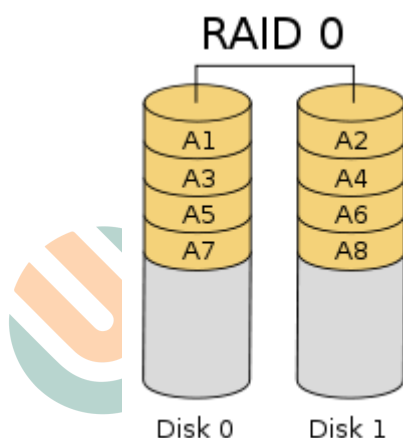
JBOD，Just a Bunch of Disks，磁盘簇。表示一个没有控制软件提供协调控制的磁盘集合，这是 RAID 区别与 JBOD 的主要因素。JBOD 将多个物理磁盘串联起来，提供一个巨大的逻辑磁盘。

JBOD 的数据存放机制是由第一块磁盘开始按顺序往后存储，当前磁盘存储空间用完后，再依次往后面的磁盘存储数据。JBOD 存储性能完全等同于单块磁盘，而且也不提供数据安全保护。

其只是简单提供一种扩展存储空间的机制，JBOD可用存储容量等于所有成员磁盘的存储空间之和

JBOD 常指磁盘柜，而不论其是否提供 RAID 功能。不过，JBOD并非官方术语，官方称为Spanning。

RAID0



RAID0 是一种简单的、无数据校验的**数据条带化技术**。实际上不是一种真正的 RAID，因为它并不提供任何形式的冗余策略。RAID0 将所在磁盘条带化后组成大容量的存储空间，将数据分散存储在所有磁盘上，以独立访问方式实现多块磁盘的并读访问。

理论上讲，一个由 n 块磁盘组成的 RAID0，它的读写性能是单个磁盘性能的 n 倍，但由于总线带宽等多种因素的限制，实际的性能提升低于理论值。由于可以并发执行 I/O 操作，总线带宽得到充分利用。再加上不需要进行数据校验，**RAID0 的性能在所有 RAID 等级中是最高的**。

RAID0 具有低成本、高读写性能、100% 的高存储空间利用率等优点，但是它不提供数据冗余保护，一旦数据损坏，将无法恢复。

应用场景：对数据的顺序读写要求不高，对数据的安全性和可靠性要求不高，但对系统性能要求很高的场景。

RAID0与JBOD相同点：

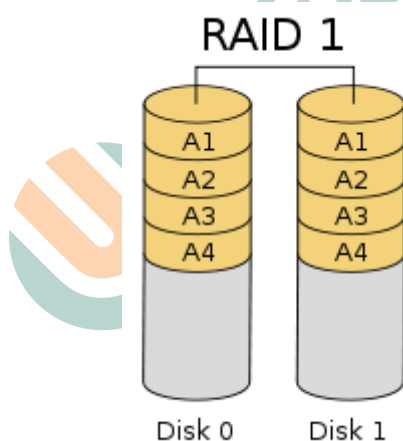
- 1) 存储容量：都是成员磁盘容量总和
- 2) 磁盘利用率，都是100%，即都没有做任何的数据冗余备份

RAID0与JBOD不同点：

JBOD：数据是顺序存放的，一个磁盘存满后才会开始存放 to 下一个磁盘

RAID：各个磁盘中的数据写入是并行的，是通过数据条带技术写入的。其读写性能是JBOD的 n 倍

RAID1

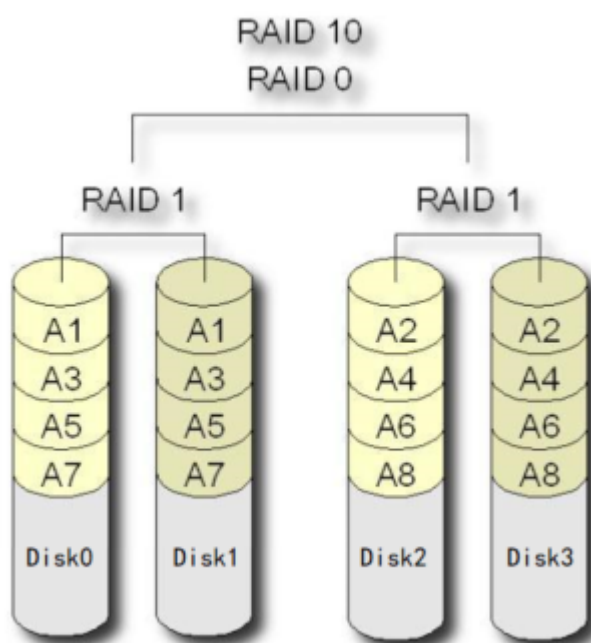


RAID1 就是一种**镜像技术**，它将数据完全一致地分别写到工作磁盘和镜像磁盘，它的磁盘空间利用率为 50%。RAID1 在数据写入时，响应时间会有所影响，但是读数据的时候没有影响。RAID1 提供了最佳的数据保护，一旦工作磁盘发生故障，系统将自动切换到镜像磁盘，不会影响使用。

RAID1 是为了增强数据安全性使两块磁盘数据呈现完全镜像，从而达到安全性好、技术简单、管理方便。RAID1 拥有完全容错的能力，但实现成本高。

应用场景：对顺序读写性能要求较高，或对数据安全性要求较高的场景。

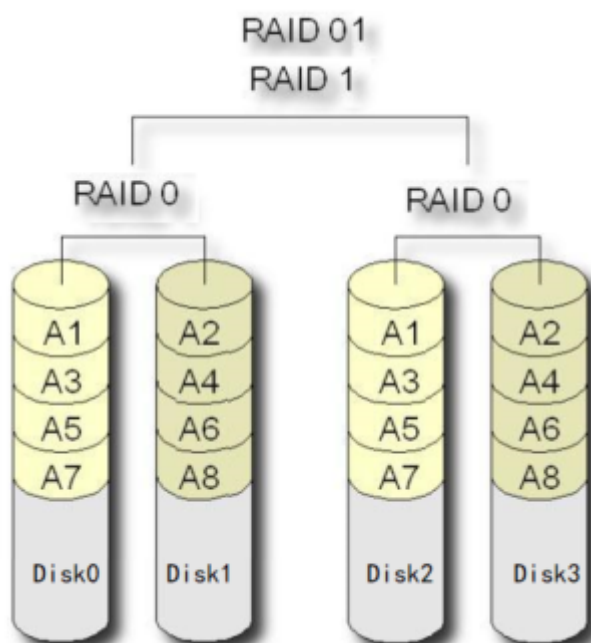
RAID10



RAID10是一个RAID1与RAID0的组合体，所以它继承了RAID0的快速和RAID1的安全。

简单来说就是，先做条带，再做镜像。发即将进来的数据先分散到不同的磁盘，再将磁盘中的数据做镜像。

RAID01



RAID01是一个RAID0与RAID1的组合体，所以它继承了RAID0的快速和RAID1的安全。

简单来说就是，先做 **镜像** 再做 **条带**。即将进来的数据先做镜像，再将镜像数据写入到与之前数据不同的磁盘，即再做条带。

RAID10要比RAID01的容错率再高，所以生产环境下一般是不使用RAID01的。

七、集群搭建实践

1 集群架构

这里要搭建一个双主双从异步复制的Broker集群。为了方便，这里使用了两台主机来完成集群的搭建。这两台主机的功能与broker角色分配如下表。

序号	主机名/IP	IP	功能	BROKER角色
1	rocketmqOS1	192.168.59.164	NameServer + Broker	Master1 + Slave2
2	rocketmqOS2	192.168.59.165	NameServer + Broker	Master2 + Slave1

2 克隆生成rocketmqOS1

克隆rocketmqOS主机，并修改配置。指定主机名为rocketmqOS1。

3 修改rocketmqOS1配置文件

配置文件位置

要修改的配置文件在rocketMQ解压目录的conf/2m-2s-async目录中。

```
✓ mqos1 [root@mqos1 2m-2s-async]# ll
总用量 16
-rw-r--r-- 1 root root 981 1月 27 11:28 broker-a.properties
-rw-r--r-- 1 root root 922 10月 23 10:29 broker-a-s.properties
-rw-r--r-- 1 root root 929 10月 23 10:29 broker-b.properties
-rw-r--r-- 1 root root 922 10月 23 10:29 broker-b-s.properties
[root@mqos1 2m-2s-async]#
```

修改broker-a.properties

将该配置文件内容修改为如下：

```
1 # 指定整个broker集群的名称，或者说是RocketMQ集群的名称
2 brokerClusterName=DefaultCluster
3 # 指定master-slave集群的名称。一个RocketMQ集群可以包含多个master-slave集群
4 brokerName=broker-a
5 # master的brokerId为0
6 brokerId=0
```

```

7 # 指定删除消息存储过期文件的时间为凌晨4点
8 deletewhen=04
9 # 指定未发生更新的消息存储文件的保留时长为48小时，48小时后过期，将会被删除
10 fileReservedTime=48
11 # 指定当前broker为异步复制master
12 brokerRole=ASYNC_MASTER
13 # 指定刷盘策略为异步刷盘
14 flushDiskType=ASYNC_FLUSH
15 # 指定Name Server的地址
16 namesrvAddr=192.168.59.164:9876;192.168.59.165:9876

```

修改broker-b-s.properties

将该配置文件内容修改为如下：

```

1 brokerClusterName=DefaultCluster
2 # 指定这是另外一个master-slave集群
3 brokerName=broker-b
4 # slave的brokerId为非0
5 brokerId=1
6 deletewhen=04
7 fileReservedTime=48
8 # 指定当前broker为slave
9 brokerRole=SLAVE
10 flushDiskType=ASYNC_FLUSH
11 namesrvAddr=192.168.59.164:9876;192.168.59.165:9876
12 # 指定Broker对外提供服务的端口，即Broker与producer与consumer通信的端口。默认
    10911。由于当前主机同时充当着master1与slave2，而前面的master1使用的是默认端口。这
    里需要将这两个端口加以区分，以区分出master1与slave2
13 listenPort=11911
14 # 指定消息存储相关的路径。默认路径为~/store目录。由于当前主机同时充当着master1与
    slave2，master1使用的是默认路径，这里就需要再指定一个不同路径
15 storePathRootDir=~/.store-s
16 storePathCommitLog=~/.store-s/commitlog
17 storePathConsumeQueue=~/.store-s/consumequeue
18 storePathIndex=~/.store-s/index
19 storeCheckpoint=~/.store-s/checkpoint
20 abortFile=~/.store-s/abort

```

其它配置

除了以上配置外，这些配置文件中还可以设置其它属性。

```

1 #指定整个broker集群的名称，或者说是RocketMQ集群的名称
2 brokerClusterName=rocket-MS
3 #指定master-slave集群的名称。一个RocketMQ集群可以包含多个master-slave集群
4 brokerName=broker-a

```

```
5  #0 表示 Master, >0 表示 slave
6  brokerId=0
7  #nameServer地址, 分号分割
8  namesrvAddr=nameserver1:9876;nameserver2:9876
9  #默认为新建Topic所创建的队列数
10 defaultTopicQueueNums=4
11 #是否允许 Broker 自动创建Topic, 建议生产环境中关闭
12 autoCreateTopicEnable=true
13 #是否允许 Broker 自动创建订阅组, 建议生产环境中关闭
14 autoCreateSubscriptionGroup=true
15 #Broker对外提供服务的端口, 即Broker与producer与consumer通信的端口
16 listenPort=10911
17 #HA高可用监听端口, 即Master与Slave间通信的端口, 默认值为listenPort+1
18 haListenPort=10912
19 #指定删除消息存储过期文件的时间为凌晨4点
20 deleteWhen=04
21 #指定未发生更新的消息存储文件的保留时长为48小时, 48小时后过期, 将会被删除
22 fileReservedTime=48
23 #指定commitLog目录中每个文件的大小, 默认1G
24 mappedFileSizeCommitLog=1073741824
25 #指定ConsumeQueue的每个Topic的每个Queue文件中可以存放的消息数量, 默认30w条
26 mappedFileSizeConsumeQueue=300000
27 #在清除过期文件时, 如果该文件被其他线程所占 (引用数大于0, 比如读取消息), 此时会阻止
    此次删除任务, 同时在第一次试图删除该文件时记录当前时间戳。该属性则表示从第一次拒绝删除
    后开始计时, 该文件最多可以保留的时长。在此时间内若引用数仍不为0, 则删除仍会被拒绝。不过
    时间到后, 文件将被强制删除
28 destroyMappedFileIntervalForcibly=120000
29 #指定commitlog、consumequeue所在磁盘分区的最小使用率, 超过该值, 则需立即清除过期文
    件
30 diskMaxUsedSpaceRatio=88
31 #指定store目录的路径, 默认在当前用户主目录中
32 storePathRootDir=/usr/local/rocketmq-all-4.5.0/store
33 #commitLog目录路径
34 storePathCommitLog=/usr/local/rocketmq-all-4.5.0/store/commitlog
35 #consumequeue目录路径
36 storePathConsumeQueue=/usr/local/rocketmq-all-4.5.0/store/consumequeue
37 #index目录路径
38 storePathIndex=/usr/local/rocketmq-all-4.5.0/store/index
39 #checkpoint文件路径
40 storeCheckpoint=/usr/local/rocketmq-all-4.5.0/store/checkpoint
41 #abort文件路径
42 abortFile=/usr/local/rocketmq-all-4.5.0/store/abort
43 #指定消息的最大大小
44 maxMessageSize=65536
45 #Broker的角色
46 # - ASYNC_MASTER 异步复制Master
47 # - SYNC_MASTER 同步双写Master
48 # - SLAVE
```

```
49 brokerRole=SYNC_MASTER
50 #刷盘策略
51 # - ASYNC_FLUSH 异步刷盘
52 # - SYNC_FLUSH 同步刷盘
53 flushDiskType=SYNC_FLUSH
54 #发消息线程池数量
55 sendMessageThreadPoolNums=128
56 #拉消息线程池数量
57 pullMessageThreadPoolNums=128
58 #强制指定本机IP，需要根据每台机器进行修改。官方介绍可为空，系统默认自动识别，但多网卡
    时IP地址可能读取错误
59 brokerIP1=192.168.3.105
```

4 克隆生成rocketmqOS2

克隆rocketmqOS1主机，并修改配置。指定主机名为rocketmqOS2。

5 修改rocketmqOS2配置文件

对于rocketmqOS2主机，同样需要修改rocketMQ解压目录的conf目录的子目录2m-2s-async中的两个配置文件。

修改broker-b.properties

将该配置文件内容修改为如下：

```
1 brokerClusterName=DefaultCluster
2 brokerName=broker-b
3 brokerId=0
4 deleteWhen=04
5 fileReservedTime=48
6 brokerRole=ASYNC_MASTER
7 flushDiskType=ASYNC_FLUSH
8 namesrvAddr=192.168.59.164:9876;192.168.59.165:9876
```

修改broker-a-s.properties

将该配置文件内容修改为如下：

```
1 brokerClusterName=DefaultCluster
2 brokerName=broker-a
3 brokerId=1
4 deleteWhen=04
5 fileReservedTime=48
6 brokerRole=SLAVE
7 flushDiskType=ASYNC_FLUSH
```

```
8 namesrvAddr=192.168.59.164:9876;192.168.59.165:9876
9 listenPort=11911
10 storePathRootDir=~/.store-s
11 storePathCommitLog=~/.store-s/commitlog
12 storePathConsumeQueue=~/.store-s/consumequeue
13 storePathIndex=~/.store-s/index
14 storeCheckpoint=~/.store-s/checkpoint
15 abortFile=~/.store-s/abort
```

6 启动服务器

启动NameServer集群

分别启动rocketmqOS1与rocketmqOS2两个主机中的NameServer。启动命令完全相同。

```
1 nohup sh bin/mqnamesrv &
2 tail -f ~/logs/rocketmqlogs/namesrv.log
```

启动两个Master

分别启动rocketmqOS1与rocketmqOS2两个主机中的broker master。注意，它们指定所要加载的配置文件是不同的。

```
1 nohup sh bin/mqbroker -c conf/2m-2s-async/broker-a.properties &
2 tail -f ~/logs/rocketmqlogs/broker.log
```

```
1 nohup sh bin/mqbroker -c conf/2m-2s-async/broker-b.properties &
2 tail -f ~/logs/rocketmqlogs/broker.log
```

启动两个Slave

分别启动rocketmqOS1与rocketmqOS2两个主机中的broker slave。注意，它们指定所要加载的配置文件是不同的。

```
1 nohup sh bin/mqbroker -c conf/2m-2s-async/broker-b-s.properties &
2 tail -f ~/logs/rocketmqlogs/broker.log
```

```
1 nohup sh bin/mqbroker -c conf/2m-2s-async/broker-a-s.properties &
2 tail -f ~/logs/rocketmqlogs/broker.log
```

八、mqadmin命令

在mq解压目录的bin目录下有一个mqadmin命令，该命令是一个运维指令，用于对mq的主题，集群，broker 等信息进行管理。

1 修改bin/tools.sh

在运行mqadmin命令之前，先要修改mq解压目录下bin/tools.sh配置的JDK的ext目录位置。本机的ext目录在 `/usr/java/jdk1.8.0_161/jre/lib/ext`。

使用vim命令打开tools.sh文件，并在JAVA_OPT配置的-Djava.ext.dirs这一行的后面添加ext的路径。

```
#####  
# JVM Configuration  
#####  
JAVA_OPT="{JAVA_OPT} -server -Xms1g -Xmx1g -Xmn256m -XX:MetaspaceSize=128m -XX:MaxMetaspaceSize=128m"  
JAVA_OPT="{JAVA_OPT} -Djava.ext.dirs={BASE_DIR}/lib:{JAVA_HOME}/jre/lib/ext:{JAVA_HOME}/lib/ext:/usr/java/jdk1.8.0_161/jre/lib/ext"  
JAVA_OPT="{JAVA_OPT} -cp {CLASSPATH}"
```

```
1  JAVA_OPT="{JAVA_OPT} -server -Xms1g -Xmx1g -Xmn256m -  
   XX:MetaspaceSize=128m -XX:MaxMetaspaceSize=128m"  
2  JAVA_OPT="{JAVA_OPT} -  
   Djava.ext.dirs={BASE_DIR}/lib:{JAVA_HOME}/jre/lib/ext:{JAVA_HOME}/lib/  
   ext:/usr/java/jdk1.8.0_161/jre/lib/ext"  
3  JAVA_OPT="{JAVA_OPT} -cp {CLASSPATH}"
```

2 运行mqadmin

直接运行该命令，可以看到其可以添加的commands。通过这些commands可以完成很多的功能。

```
1  [root@mqOS rocketmq-all-4.8.0-bin-release]# ./bin/mqadmin  
2  The most commonly used mqadmin commands are:  
3      updateTopic          Update or create topic  
4      deleteTopic          Delete topic from broker and NameServer.  
5      updateSubGroup       Update or create subscription group  
6      deleteSubGroup       Delete subscription group from broker.  
7      updateBrokerConfig   Update broker's config  
8      updateTopicPerm      Update topic perm  
9      topicRoute           Examine topic route info  
10     topicStatus          Examine topic Status info  
11     topicClusterList     get cluster info for topic  
12     brokerStatus         Fetch broker runtime status data  
13     queryMsgById         Query Message by Id  
14     queryMsgByKey        Query Message by Key  
15     queryMsgByUniqueKey   Query Message by Unique key  
16     queryMsgByOffset     Query Message by offset  
17     QueryMsgTraceById    query a message trace  
18     printMsg             Print Message Detail  
19     printMsgByQueue      Print Message Detail  
20     sendMsgStatus        send msg to broker.  
21     brokerConsumeStats   Fetch broker consume stats data  
22     producerConnection   Query producer's socket connection and client  
    version  
23     consumerConnection   Query consumer's socket connection, client  
    version and subscription  
24     consumerProgress     Query consumers's progress, speed
```

25	consumerStatus	Query consumer's internal data structure
26	cloneGroupOffset	clone offset from other group.
27	clusterList	List all of clusters
28	topicList	Fetch all topic list from name server
29	updateKvConfig	Create or update KV config.
30	deleteKvConfig	Delete KV config.
31	wipeWritePerm	wipe write perm of broker in all name server
32	resetOffsetByTime	Reset consumer offset by timestamp(without client restart).
33	updateOrderConf	Create or update or delete order conf
34	cleanExpiredCQ	Clean expired ConsumeQueue on broker.
35	cleanUnusedTopic	Clean unused topic on broker.
36	startMonitoring	Start Monitoring
37	statsAll	Topic and Consumer tps stats
38	allocateMQ	Allocate MQ
39	checkMsgSendRT	check message send response time
40	clusterRT	List All clusters Message Send RT
41	getNamesrvConfig	Get configs of name server.
42	updateNamesrvConfig	Update configs of name server.
43	getBrokerConfig	Get broker config by cluster or special broker!
44	queryCq	Query cq command.
45	sendMessage	Send a message
46	consumeMessage	Consume message
47	updateAclConfig	Update acl config yaml file in broker
48	deleteAccessConfig	Delete Acl Config Account in broker
49	clusterAclConfigVersion	List all of acl config version information in cluster
50	updateGlobalWhiteAddr	Update global white address for acl Config File in broker
51	getAccessConfigSubCommand	List all of acl config information in cluster

3 该命令的官网详解

该命令在官网中有详细的用法解释。

<https://github.com/apache/rocketmq/blob/master/docs/cn/operation.md>

- [RocketMQ部署\(Cluster Deployment\)](#) - 介绍RocketMQ的集群部署方式。

- [集群部署\(Cluster Deployment\)](#): 介绍RocketMQ的集群部署方式。

5. 运维管理

- [集群部署\(Operation\)](#): 介绍单Master模式、多Master模式、多Master多slave模式等RocketMQ集群各种形式的部署方法以及运维工具mqadmin的使用方式。

6. API Reference (待补充)

- [DefaultMQProducer API Reference](#)

2 mqadmin管理工具

注意:

1. 执行命令方法: `./mqadmin {command} {args}`
2. 几乎所有命令都需要配置-n表示NameServer地址, 格式为ip:port
3. 几乎所有命令都可以通过-h获取帮助
4. 如果既有Broker地址 (-b) 配置项又有clusterName (-c) 配置项, 则优先以Broker地址执行命令, 如果不配置Broker地址, 则对集群中所有主机执行命令, 只支持一个Broker地址。-b格式为ip:port, port默认是10911
5. 在tools下可以看到很多命令, 但并不是所有命令都能使用, 只有在MQAdminStartup中初始化的命令才能使用, 你也可以修改这个类, 增加或自定义命令
6. 由于版本更新问题, 少部分命令可能未及时更新, 遇到错误请直接阅读相关命令源码

2.1 Topic相关

名称	含义	命令选项	说明
		-b	Broker 地址, 表示 topic 所在 Broker, 只支持单台Broker, 地址为ip:port
		-c	cluster 名称, 表示 topic 所在集群 (集群可通过 clusterList 查询)
		-h-	打印帮助

第3章 RocketMQ工作原理

一、消息的生产

1 消息的生产过程

Producer可以将消息写入到某Broker中的某Queue中, 其经历了如下过程:

- Producer发送消息之前, 会先向NameServer发出获取消息Topic的路由信息 的请求
- NameServer返回该Topic的 路由表 及 Broker列表
- Producer根据代码中指定的Queue选择策略, 从Queue列表中选出一个队列, 用于后续存储消息
- Producer对消息做一些特殊处理, 例如, 消息本身超过4M, 则会对其进行压缩
- Producer向选择出的Queue所在的Broker发出RPC请求, 将消息发送到选择出的Queue

路由表: 实际是一个Map, key为Topic名称, value是一个QueueData实例列表。QueueData并不是一个Queue对应一个QueueData, 而是一个Broker中该Topic的所有Queue对应一个QueueData。即, 只要涉及到该Topic的Broker, 一个Broker对应一个QueueData。QueueData中包含brokerName。简单来说, 路由表的key为Topic名称, value则为所有涉及该Topic的BrokerName列表。

Broker列表：其实际也是一个Map。key为brokerName，value为BrokerData。一个Broker对应一个BrokerData实例，对吗？不对。一套brokerName名称相同的Master-Slave小集群对应一个BrokerData。BrokerData中包含brokerName及一个map。该map的key为brokerId，value为该broker对应的地址。brokerId为0表示该broker为Master，非0表示Slave。

2 Queue选择算法

对于无序消息，其Queue选择算法，也称为消息投递算法，常见的有两种：

轮询算法

默认选择算法。该算法保证了每个Queue中可以均匀的获取到消息。

该算法存在一个问题：由于某些原因，在某些Broker上的Queue可能投递延迟较严重。从而导致Producer的缓存队列中出现较大的消息积压，影响消息的投递性能。

最小投递延迟算法

该算法会统计每次消息投递的时间延迟，然后根据统计出的结果将消息投递到时间延迟最小的Queue。如果延迟相同，则采用轮询算法投递。该算法可以有效提升消息的投递性能。

该算法也存在一个问题：消息在Queue上的分配不均匀。投递延迟小的Queue其可能会存在大量的消息。而对该Queue的消费者压力会增大，降低消息的消费能力，可能会导致MQ中消息的堆积。

二、消息的存储

RocketMQ中的消息存储在本地文件系统中，这些相关文件默认在当前用户主目录下的store目录中。

✓ rocketmqos ✓ rocketmqos (1)

```
[root@rocketmq0S store]# ll
总用量 8
-rw-r--r-- 1 root root 0 7月 1 17:43 abort
-rw-r--r-- 1 root root 4096 7月 1 17:44 checkpoint
drwxr-xr-x 2 root root 34 6月 26 10:35 commitlog
drwxr-xr-x 2 root root 246 7月 1 17:44 config
drwxr-xr-x 3 root root 23 6月 26 10:28 consumequeue
drwxr-xr-x 2 root root 31 6月 26 10:28 index
-rw-r--r-- 1 root root 4 7月 1 17:43 lock
[root@rocketmq0S store]#
```

- abort：该文件在Broker启动后会自动创建，正常关闭Broker，该文件会自动消失。若在没有启动Broker的情况下，发现这个文件是存在的，则说明之前Broker的关闭是非正常关闭。
- checkpoint：其中存储着commitlog、consumequeue、index文件的最后刷盘时间戳
- commitlog：其中存放着commitlog文件，而消息是写在commitlog文件中的
- config：存放着Broker运行期间的一些配置数据
- consumequeue：其中存放着consumequeue文件，队列就存放在这个目录中
- index：其中存放着消息索引文件indexFile
- lock：运行期间使用到的全局资源锁

1 commitlog文件

说明：在很多资料中commitlog目录中的文件简单就称为commitlog文件。但在源码中，该文件被命名为mappedFile。

目录与文件

commitlog目录中存放着很多的mappedFile文件，当前Broker中的所有消息都是落盘到这些mappedFile文件中的。mappedFile文件大小为1G（小于等于1G），文件名由20位十进制数构成，表示当前文件的第一条消息的起始位移偏移量。

第一个文件名一定是20位0构成的。因为第一个文件的第一条消息的偏移量commitlog offset为0

当第一个文件放满时，则会自动生成第二个文件继续存放消息。假设第一个文件大小是1073741820字节（1G = 1073741824字节），则第二个文件名就是00000000001073741824。

以此类推，第n个文件名应该是前n-1个文件大小之和。

一个Broker中所有mappedFile文件的commitlog offset是连续的

需要注意的是，一个Broker中仅包含一个commitlog目录，所有的mappedFile文件都是存放在该目录中的。即无论当前Broker中存放着多少Topic的消息，这些消息都是被顺序写入到了mappedFile文件中的。也就是说，这些消息在Broker中存放时并没有被按照Topic进行分类存放。

mappedFile文件是顺序读写的文件，所有其访问效率很高

无论是SSD磁盘还是SATA磁盘，通常情况下，顺序存取效率都会高于随机存取。

消息单元

Commitlog Offset	n	MsgLen	PhysicalOffset	Body	BodyLength	BornHost	BornTimestamp	Topic	QueueId	QueueOffset	
	L1	MsgLen	PhysicalOffset	Body	BodyLength	BornHost	BornTimestamp	Topic	QueueId	QueueOffset	
	L2	MsgLen	PhysicalOffset	Body	BodyLength	BornHost	BornTimestamp	Topic	QueueId	QueueOffset	
	L3	MsgLen	PhysicalOffset	Body	BodyLength	BornHost	BornTimestamp	Topic	QueueId	QueueOffset	
	L4	MsgLen	PhysicalOffset	Body	BodyLength	BornHost	BornTimestamp	Topic	QueueId	QueueOffset	
	L5	MsgLen	PhysicalOffset	Body	BodyLength	BornHost	BornTimestamp	Topic	QueueId	QueueOffset	
	L6	MsgLen	PhysicalOffset	Body	BodyLength	BornHost	BornTimestamp	Topic	QueueId	QueueOffset	
	L7	MsgLen	PhysicalOffset	Body	BodyLength	BornHost	BornTimestamp	Topic	QueueId	QueueOffset	

commitlog中名称为n的mappedFile文件内部消息存放结构示意图

mappedFile文件内容由一个个的消息单元构成。每个消息单元中包含消息总长度MsgLen、消息的物理位置physicalOffset、消息体内容Body、消息体长度BodyLength、消息主题Topic、Topic长度TopicLength、消息生产者BornHost、消息发送时间戳BornTimestamp、消息所在的队列QueueId、消息在Queue中存储的偏移量QueueOffset等近20余项消息相关属性。

需要注意到，消息单元中是包含Queue相关属性的。所以，我们在后续的学习中，就需要十分留意commitlog与queue间的关系是什么？

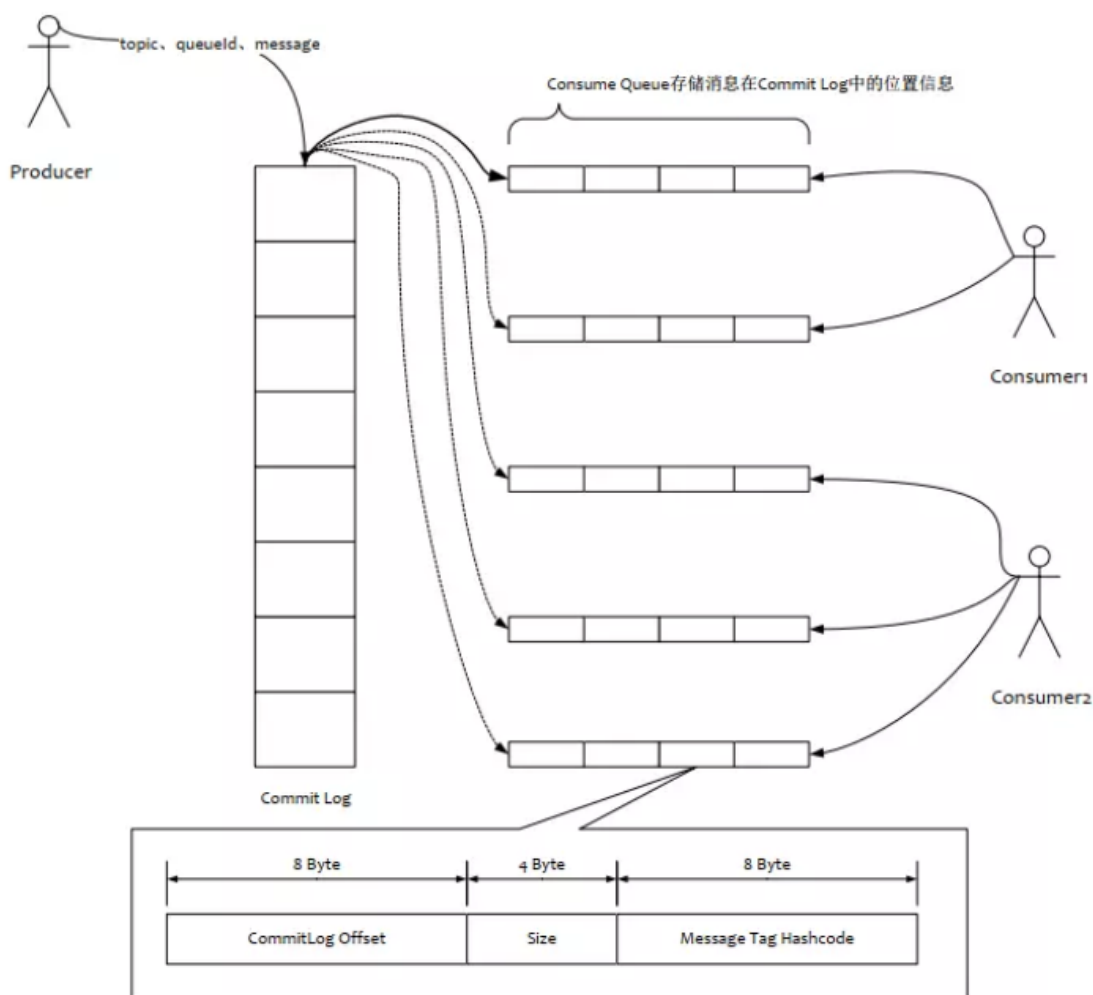
一个mappedFile文件中第m+1个消息单元的commitlog offset偏移量

$$L(m+1) = L(m) + \text{MsgLen}(m) \quad (m \geq 0)$$

2 consumequeue

```
✓ mqos [red square]
[root@mqOS ~]# ll store/consumequeue/
总用量 0
drwxr-xr-x 6 root root 42 1月 13 13:09 aaaxxaaa
drwxr-xr-x 6 root root 42 1月 13 12:13 abc000
drwxr-xr-x 6 root root 42 1月 13 12:11 abcXxx
drwxr-xr-x 6 root root 42 1月 17 13:58 myTopic
drwxr-xr-x 6 root root 42 1月 13 12:03 persons
drwxr-xr-x 6 root root 42 1月 13 12:09 RMQ_SYS_TRACE_TOPIC
drwxr-xr-x 3 root root 15 1月 28 17:27 SCHEDULE_TOPIC_XXXX
drwxr-xr-x 6 root root 42 1月 17 13:57 someTopic
drwxr-xr-x 6 root root 42 1月 17 13:56 someTopicTest
drwxr-xr-x 6 root root 42 1月 29 10:23 Topic1
drwxr-xr-x 6 root root 42 1月 13 10:44 TopicTest
drwxr-xr-x 6 root root 42 1月 17 13:55 TopicTest1
[root@mqOS ~]#
[root@mqOS ~]# ll store/consumequeue/persons
总用量 0
drwxr-xr-x 2 root root 34 1月 13 12:03 0
drwxr-xr-x 2 root root 34 1月 13 12:03 1
drwxr-xr-x 2 root root 34 1月 13 12:03 2
drwxr-xr-x 2 root root 34 1月 13 12:03 3
[root@mqOS ~]#
[root@mqOS ~]# ll store/consumequeue/persons/0
总用量 4
-rw-r--r-- 1 root root 6000000 1月 13 12:05 00000000000000000000
[root@mqOS ~]#
```

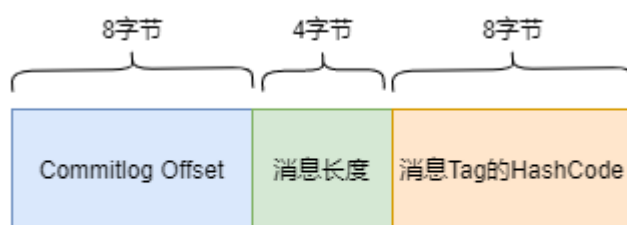
目录与文件



为了提高效率，会为每个Topic在~/store/consumequeue中创建一个目录，目录名为Topic名称。在该Topic目录下，会再为每个该Topic的Queue建立一个目录，目录名为queueId。每个目录中存放着若干consumequeue文件，consumequeue文件是commitlog的索引文件，可以根据consumequeue定位到具体的消息。

consumequeue文件名也由20位数字构成，表示当前文件的第一个索引条目的起始位移偏移量。与mappedFile文件名不同的是，其后续文件名是固定的。因为consumequeue文件大小是固定不变的。

索引条目

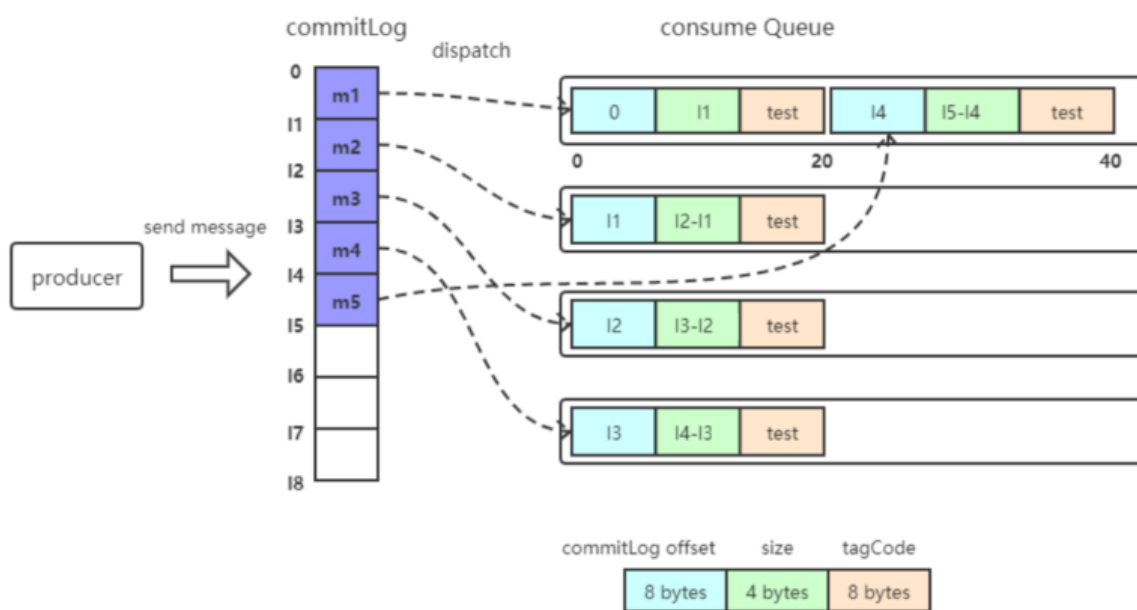


consumequeue文件的一个索引条目的结构示意图

每个consumequeue文件可以包含30w个索引条目，每个索引条目包含了三个消息重要属性：消息在mappedFile文件中的偏移量CommitLog Offset、消息长度、消息Tag的hashCode值。这三个属性占20个字节，所以每个文件的大小是固定的30w * 20字节。

一个consumequeue文件中所有消息的Topic一定是相同的。但每条消息的Tag可能是不同的。

3 对文件的读写



消息写入

一条消息进入到Broker后经历了以下几个过程才最终被持久化。

- Broker根据queueId，获取到该消息对应索引条目要在consumequeue目录中的写入偏移量，即QueueOffset
- 将queueId、queueOffset等数据，与消息一起封装为消息单元
- 将消息单元写入到commitlog
- 同时，形成消息索引条目
- 将消息索引条目分发到相应的consumequeue

消息拉取

- 当Consumer来拉取消息时会经历以下几个步骤：
 - Consumer获取到其要消费消息所在Queue的**消费偏移量offset**，计算出其要消费消息的**消息offset**

消费offset即消费进度，consumer对某个Queue的消费offset，即消费到了该Queue的第几条消息

消息offset = 消费offset + 1

- Consumer向Broker发送拉取请求，其中会包含其要拉取消息的Queue、消息offset及消息Tag。
- Broker计算在该consumequeue中的queueOffset。

$queueOffset = 消息offset * 20 字节$

- 从该queueOffset处开始向后查找第一个指定Tag的索引条目。
- 解析该索引条目前8个字节，即可定位到该消息在commitlog中的commitlog offset
- 从对应commitlog offset中读取消息单元，并发送给Consumer

性能提升

RocketMQ中，无论是消息本身还是消息索引，都是存储在磁盘上的。其不会影响消息的消费吗？当然不会。其实RocketMQ的性能在目前的MQ产品中性能是非常高的。因为系统通过一系列相关机制大大提升了性能。

首先，RocketMQ对文件的读写操作是通过mmap零拷贝进行的，将对文件的操作转化为直接对内存地址进行操作，从而极大地提高了文件的读写效率。

其次，consumequeue中的数据是顺序存放的，还引入了PageCache的预读取机制，使得对consumequeue文件的读取几乎接近于内存读取，即使在有消息堆积情况下也不会影响性能。

PageCache 机制，页缓存机制，是OS对文件的缓存机制，用于加速对文件的读写操作。一般来说，程序对文件进行顺序读写的速度几乎接近于内存读写速度，主要原因是由于OS使用PageCache 机制对读写访问操作进行性能优化，将一部分的内存用作PageCache。

- 写操作：OS会先将数据写入到PageCache中，随后会以异步方式由pdflush (page dirty flush) 内核线程将Cache中的数据刷盘到物理磁盘
- 读操作：若用户要读取数据，其首先会从PageCache中读取，若没有命中，则OS在从物理磁盘上加载该数据到PageCache的同时，也会顺序对其相邻数据块中的数据进行预读取。

RocketMQ中可能会影响性能的是对commitlog文件的读取。因为对commitlog文件来说，读取消息时会产生大量的随机访问，而随机访问会严重影响性能。不过，如果选择合适的系统IO调度算法，比如设置调度算法为Deadline（采用SSD固态硬盘的话），随机读的性能也会有所提升。

4 与Kafka的对比

RocketMQ的很多思想来源于Kafka，其中commitlog与consumequeue就是。

RocketMQ中的commitlog目录与consumequeue的结合就类似于Kafka中的partition分区目录。mappedFile文件就类似于Kafka中的segment段。

Kafka 中的Topic的消息被分割为一个或多个partition。partition是一个物理概念，对应到系统上就是topic目录下的一个或多个目录。每个partition中包含的文件称为segment，是具体存放消息的文件。

Kafka 中消息存放的目录结构是：topic目录下有partition目录，partition目录下有segment文件

Kafka 中没有二级分类标签Tag这个概念

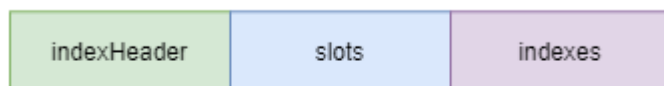
Kafka中无需索引文件。因为生产者是将消息直接写在了partition中的，消费者也是直接从partition中读取数据的

三、indexFile

除了通过通常的指定Topic进行消息消费外，RocketMQ还提供了根据key进行消息查询的功能。该查询是通过store目录中的index子目录中的indexFile进行索引实现的快速查询。当然，这个indexFile中的索引数据是在 **包含了key的消息** 被发送到Broker时写入的。如果消息中没有包含key，则不会写入。

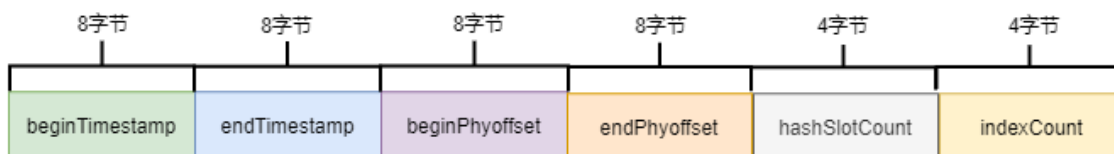
1 索引条目结构

每个Broker中会包含一组indexFile，每个indexFile都是以一个 **时间戳** 命名的（这个indexFile被创建时的时间戳）。每个indexFile文件由三部分构成：indexHeader，slots槽位，indexes索引数据。每个indexFile文件中包含500w个slot槽。而每个slot槽又可能会挂载很多的index索引单元。



indexFile文件结构示意图

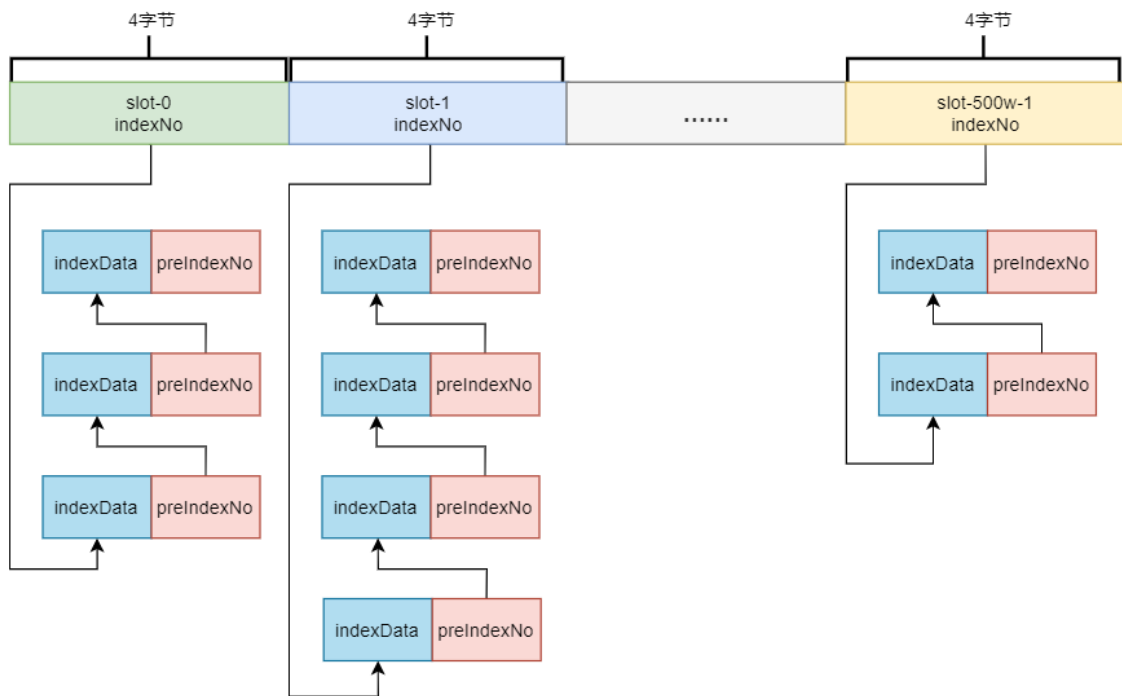
indexHeader固定40个字节，其中存放着如下数据：



indexHeader结构示意图

- beginTimestamp：该indexFile中第一条消息的存储时间
- endTimestamp：该indexFile中最后一条消息存储时间
- beginPhyoffset：该indexFile中第一条消息在commitlog中的偏移量commitlog offset
- endPhyoffset：该indexFile中最后一条消息在commitlog中的偏移量commitlog offset
- hashSlotCount：已经填充有index的slot数量（并不是每个slot槽下都挂载有index索引单元，这里统计的是所有挂载了index索引单元的slot槽的数量）
- indexCount：该indexFile中包含的索引单元个数（统计出当前indexFile中所有slot槽下挂载的所有index索引单元的数量之和）

indexFile中最复杂的是Slots与Indexes间的关系。在实际存储时，Indexes是在Slots后面的，但为了便于理解，将它们的关系展示为如下形式：



slots与indexes关系结构示意图

key的hash值 % 500w的结果即为slot槽位，然后将该slot值修改为该index索引单元的indexNo，根据这个indexNo可以计算出该index单元在indexFile中的位置。不过，该取模结果的重复率是很高的，为了解决该问题，在每个index索引单元中增加了preIndexNo，用于指定该slot中当前index索引单元的前一个index索引单元。而slot中始终存放的是其下最新的index索引单元的indexNo，这样的话，只要找到了slot就可以找到其最新的index索引单元，而通过这个index索引单元就可以找到其之前的所有index索引单元。

indexNo是一个在indexFile中的流水号，从0开始依次递增。即在一个indexFile中所有indexNo是以此递增的。indexNo在index索引单元中是没有体现的，其是通过indexes中依次数出来的。

index索引单元默写20个字节，其中存放着以下四个属性：



index索引单元结构示意图

- keyHash：消息中指定的业务key的hash值
- phyOffset：当前key对应的消息在commitlog中的偏移量commitlog offset
- timeDiff：当前key对应消息的存储时间与当前indexFile创建时间的的时间差
- preIndexNo：当前slot下当前index索引单元的前一个index索引单元的indexNo

2 indexFile的创建

indexFile的文件名为当前文件被创建时的时间戳。这个时间戳有什么用处呢？

根据业务key进行查询时，查询条件除了key之外，还需要指定一个要查询的时间戳，表示要查询不大于该时间戳的最新的消息，即查询指定时间戳之前存储的最新消息。这个时间戳文件名可以简化查询，提高查询效率。具体后面会详细讲解。

indexFile文件是何时创建的？其创建的条件（时机）有两个：

- 当第一条带key的消息发送来后，系统发现没有indexFile，此时会创建第一个indexFile文件
- 当一个indexFile中挂载的index索引单元数量超出2000w个时，会创建新的indexFile。当带key的消息发送到来后，系统会找到最新的indexFile，并从其indexHeader的最后4字节中读取到indexCount。若 $\text{indexCount} \geq 2000w$ 时，会创建新的indexFile。

由于可以推算出，一个indexFile的最大大小是： $(40 + 500w * 4 + 2000w * 20)$ 字节

3 查询流程

当消费者通过业务key来查询相应的消息时，其需要经过一个相对较复杂的查询流程。不过，在分析查询流程之前，首先要清楚几个定位计算式子：

1 计算指定消息key的slot槽位序号：
2 $\text{slot槽位序号} = \text{key的hash} \% 500w$ (式子1)

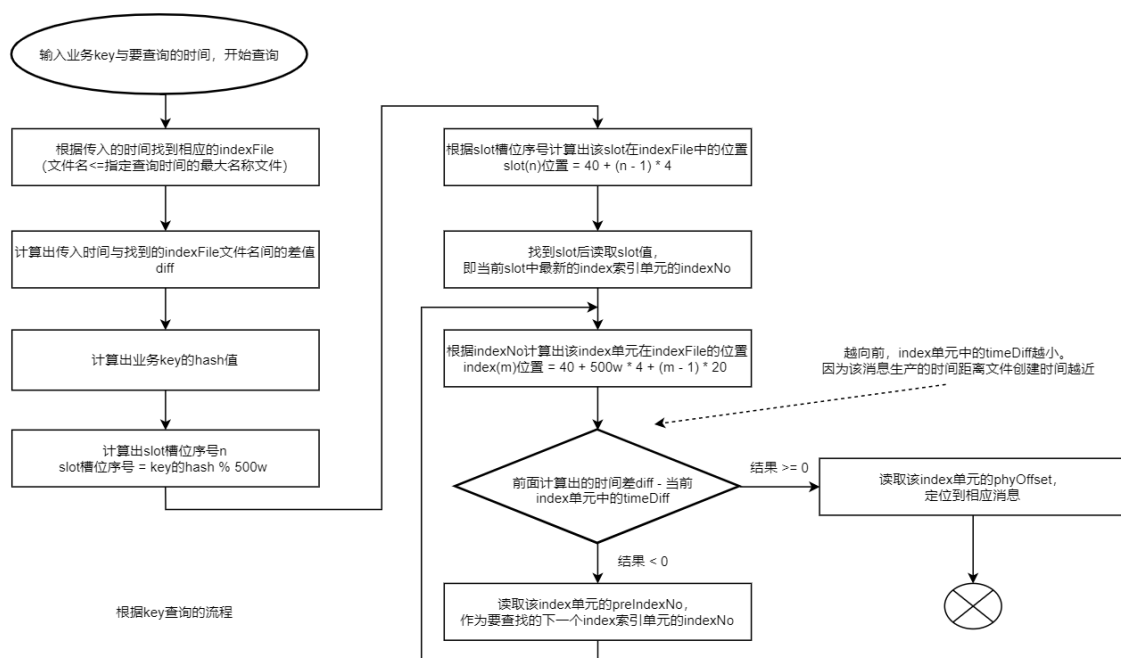
1 计算槽位序号为n的slot在indexFile中的起始位置：
2 $\text{slot}(n)\text{位置} = 40 + (n - 1) * 4$ (式子2)

1 计算indexNo为m的index在indexFile中的位置：
2 $\text{index}(m)\text{位置} = 40 + 500w * 4 + (m - 1) * 20$ (式子3)

40为indexFile中indexHeader的字节数

$500w * 4$ 是所有slots所占的字节数

具体查询流程如下：



四、消息的消费

消费者从Broker中获取消息的方式有两种：pull拉取方式和push推动方式。消费者组对于消息消费的模式又分为两种：集群消费Clustering和广播消费Broadcasting。

1 获取消费类型

拉取式消费

Consumer主动从Broker中拉取消息，主动权由Consumer控制。一旦获取了批量消息，就会启动消费过程。不过，该方式的实时性较弱，即Broker中有了新的消息时消费者并不能及时发现并消费。

由于拉取时间间隔是由用户指定的，所以在设置该间隔时需要注意平稳：间隔太短，空请求比例会增加；间隔太长，消息的实时性太差

推送式消费

该模式下Broker收到数据后会主动推送给Consumer。该获取方式一般实时性较高。

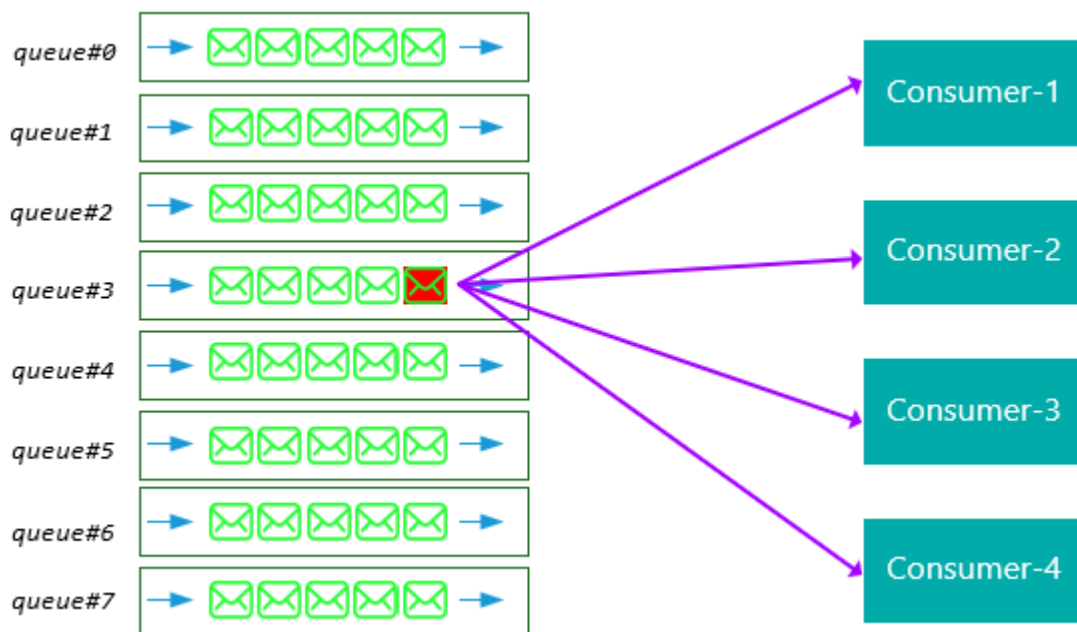
该获取方式是典型的发布-订阅模式，即Consumer向其关联的Queue注册了监听器，一旦发现有新的消息到来就会触发回调的执行，回调方法是Consumer去Queue中拉取消息。而这些都是基于Consumer与Broker间的长连接的。长连接的维护是需要消耗系统资源的。

对比

- pull：需要应用去实现对关联Queue的遍历，实时性差；但便于应用控制消息的拉取
- push：封装了对关联Queue的遍历，实时性强，但会占用较多的系统资源

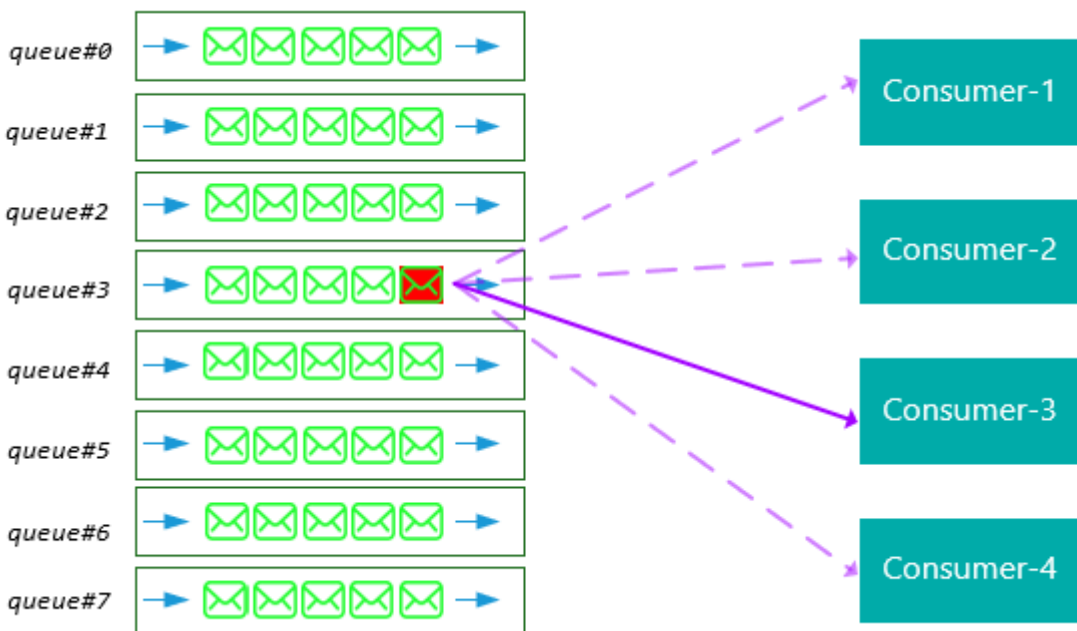
2 消费模式

广播消费



广播消费模式下，相同Consumer Group的每个Consumer实例都接收同一个Topic的全量消息。即每条消息都会被发送到Consumer Group中的每个Consumer。

集群消费



集群消费模式下，相同Consumer Group的每个Consumer实例平均分摊同一个Topic的消息。即每条消息只会被发送到Consumer Group中的某个Consumer。

消息进度保存

- 广播模式：消费进度保存在consumer端。因为广播模式下consumer group中每个consumer都会消费所有消息，但它们的消费进度是不同。所以consumer各自保存各自的消费进度。
- 集群模式：消费进度保存在broker中。consumer group中的所有consumer共同消费同一个Topic中的消息，同一条消息只会被消费一次。消费进度会参与到了消费的负载均衡中，故消费进度是需要共享的。下图是broker中存放的各个Topic的各个Queue的消费进度。

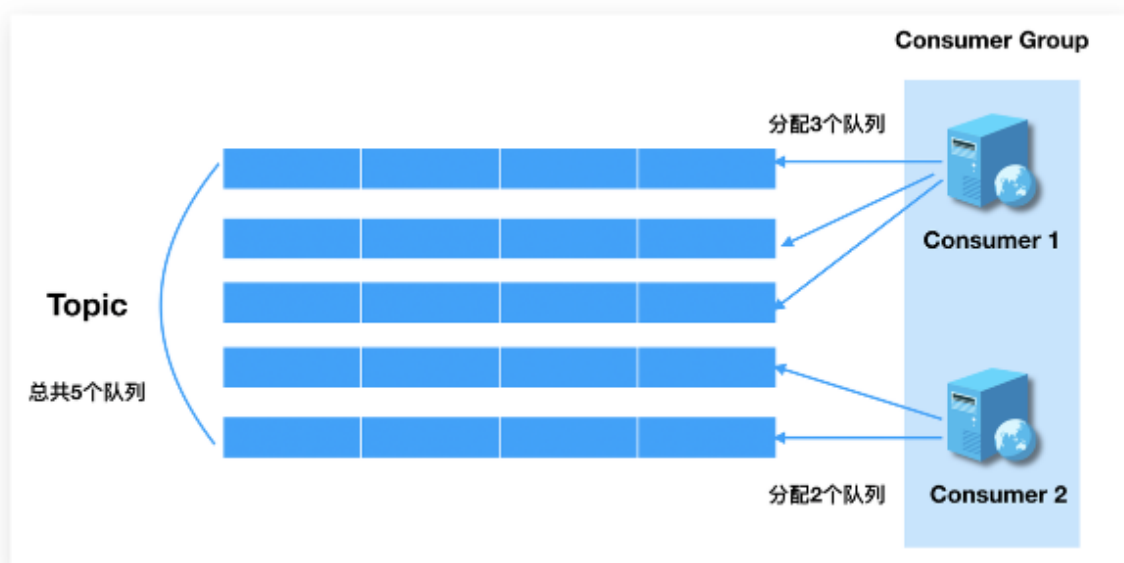
```
✓ mqos [root@mq0S store]# cat config/consumerOffset.json
{
  "offsetTable": {
    "%RETRY%cg@cg": {0:0},
    "somTopic@cg": {0:0,1:1,2:0,3:1},
    "someTopic@myConsumerGroup": {0:8,1:7,2:7,3:8},
    "persons@persons-group": {0:6,1:7,2:6,3:6},
    "%RETRY%persons-group@persons-group": {0:0},
    "abcXxx@persons-group": {0:7,1:6,2:3,3:3},
    "abc000@persons-group": {0:8,1:11,2:11,3:8},
    "someTopic@cg": {0:31,1:34,2:36,3:35},
    "TopicTest@please_rename_unique_group_name_4": {0:294},
    "RMQ_SYS_TRANS_HALF_TOPIC@CID_RMQ_SYS_TRANS": {0:18},
    "%RETRY%please_rename_unique_group_name_4@please_ren",
    "aaaxxxaaa@persons-group": {0:3,1:5,2:4,3:2}
```

3 Rebalance机制

Rebalance机制讨论的前提是：集群消费。

什么是Rebalance

Rebalance即再均衡，指的是，将一个Topic下的多个Queue在同一个Consumer Group中的多个Consumer间进行重新分配的过程。



Rebalance机制的本意是为了提升消息的 **并行消费能力**。例如，一个Topic下5个队列，在只有1个消费者的情况下，这个消费者将负责消费这5个队列的消息。如果此时我们增加一个消费者，那么就可以给其中一个消费者分配2个队列，给另一个分配3个队列，从而提升消息的并行消费能力。

Rebalance限制

由于一个队列最多分配给一个消费者，因此当某个消费者组下的消费者实例数量 **大于** 队列的数量时，多余的消费者实例将分配不到任何队列。

Rebalance危害

Rebalance的在提升消费能力的同时，也带来一些问题：

消费暂停：在只有一个Consumer时，其负责消费所有队列；在新增了一个Consumer后会触发Rebalance的发生。此时原Consumer就需要暂停部分队列的消费，等到这些队列分配给新的Consumer后，这些暂停消费的队列才能继续被消费。

消费重复：Consumer 在消费新分配给自己的队列时，必须接着之前Consumer 提交的消费进度的offset继续消费。然而默认情况下，offset是异步提交的，这个异步性导致提交到Broker的offset与Consumer 实际消费的消息并不一致。这个不一致的差值就是可能会重复消费的消息。

同步提交：consumer提交了其消费完毕的一批消息的offset给broker后，需要等待broker的成功ACK。当收到ACK后，consumer才会继续获取并消费下一批消息。在等待ACK期间，consumer是阻塞的。

异步提交：consumer提交了其消费完毕的一批消息的offset给broker后，不需要等待broker的成功ACK。consumer可以直接获取并消费下一批消息。

对于一次性读取消息的数量，需要根据具体业务场景选择一个相对均衡的是很有必要的。因为数量过大，系统性能提升了，但产生重复消费的消息数量可能会增加；数量过小，系统性能会下降，但被重复消费的消息数量可能会减少。

消费突刺：由于Rebalance可能导致重复消费，如果需要重复消费的消息过多，或者因为Rebalance暂停时间过长从而导致积压了部分消息。那么有可能会在Rebalance结束之后瞬间需要消费很多消息。

Rebalance产生的原因

导致Rebalance产生的原因，无非就两个：消费者所订阅Topic的Queue数量发生变化，或消费者组中消费者的数量发生变化。

1) Queue数量发生变化的场景：

Broker扩容或缩容

Broker升级运维

Broker与NameServer间的网络异常

Queue扩容或缩容

2) 消费者数量发生变化的场景：

Consumer Group扩容或缩容

Consumer升级运维

Consumer与NameServer间网络异常

Rebalance过程

在Broker中维护着多个Map集合，这些集合中动态存放着当前Topic中Queue的信息、Consumer Group中Consumer实例的信息。一旦发现消费者所订阅的Queue数量发生变化，或消费者组中消费者的数量发生变化，立即向Consumer Group中的每个实例发出Rebalance通知。

TopicConfigManager：key是topic名称，value是TopicConfig。TopicConfig中维护着该Topic中所有Queue的数据。

ConsumerManager：key是Consumer Group Id，value是ConsumerGroupInfo。
ConsumerGroupInfo中维护着该Group中所有Consumer实例数据。

ConsumerOffsetManager：key为Topic与订阅该Topic的Group的组合，即topic@group，
value是一个内层Map。内层Map的key为QueueId，内层Map的value为该Queue的消费进度offset。

Consumer实例在接收到通知后会采用Queue分配算法自己获取到相应的Queue，即由Consumer实例自主进行Rebalance。

与Kafka对比

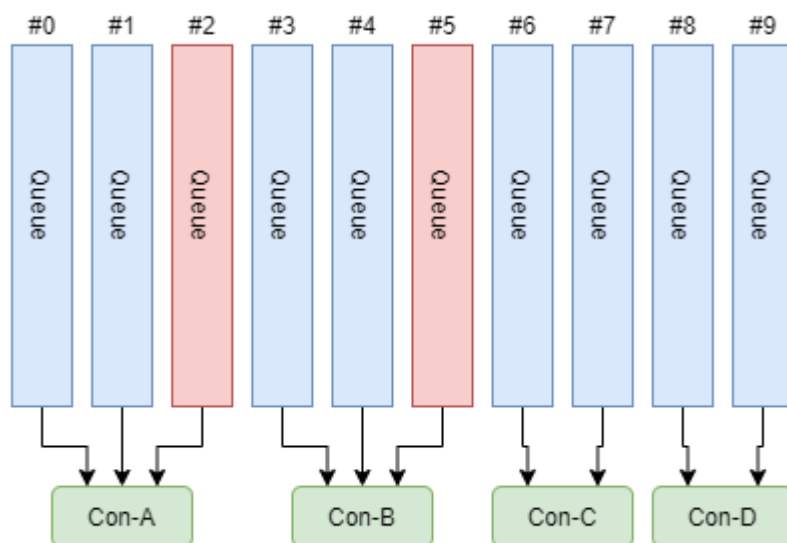
在Kafka中，一旦发现出现了Rebalance条件，Broker会调用Group Coordinator来完成Rebalance。Coordinator是Broker中的一个进程。Coordinator会在Consumer Group中选出一个Group Leader。由这个Leader根据自己本身组情况完成Partition分区的再分配。这个再分配结果会上报给Coordinator，并由Coordinator同步给Group中的所有Consumer实例。

Kafka中的Rebalance是由Consumer Leader完成的。而RocketMQ中的Rebalance是由每个Consumer自身完成的，Group中不存在Leader。

4 Queue分配算法

一个Topic中的Queue只能由Consumer Group中的一个Consumer进行消费，而一个Consumer可以同时消费多个Queue中的消息。那么Queue与Consumer间的配对关系是如何确定的，即Queue要分配给哪个Consumer进行消费，也是有算法策略的。常见的有四种策略。这些策略是通过在创建Consumer时的构造器传进去的。

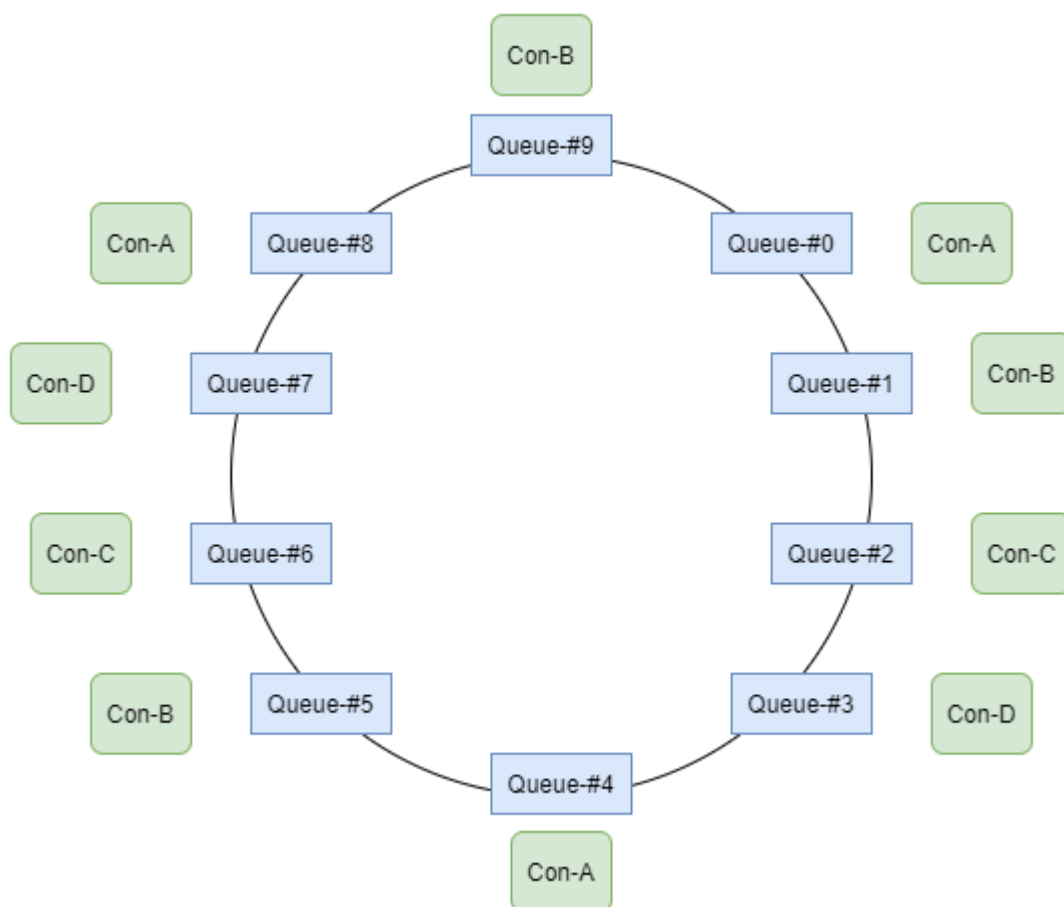
平均分配策略



该算法是要根据 $avg = QueueCount / ConsumerCount$ 的计算结果进行分配的。如果能够整除，则按顺序将avg个Queue逐个分配Consumer；如果不能整除，则将多余出的Queue按照Consumer顺序逐个分配。

该算法即，先计算好每个Consumer应该分得几个Queue，然后再依次将这些数量的Queue逐个分配个Consumer。

环形平均策略

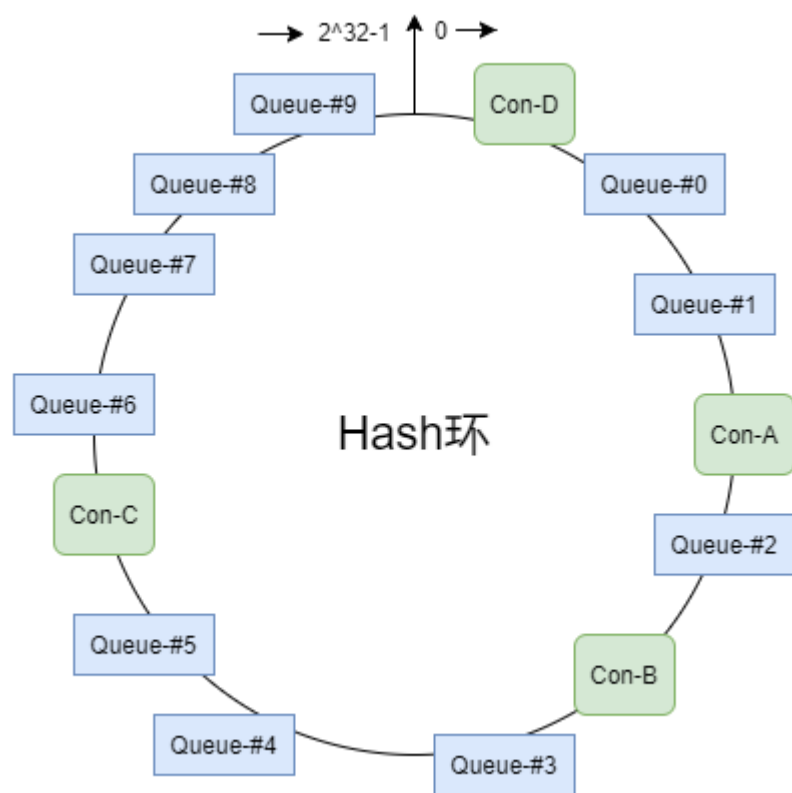


环形平均算法是指，根据消费者的顺序，依次在由queue队列组成的环形图中逐个分配。

■ 该算法不用事先计算每个Consumer需要分配几个Queue，直接一个一个分即可。

一致性hash策略

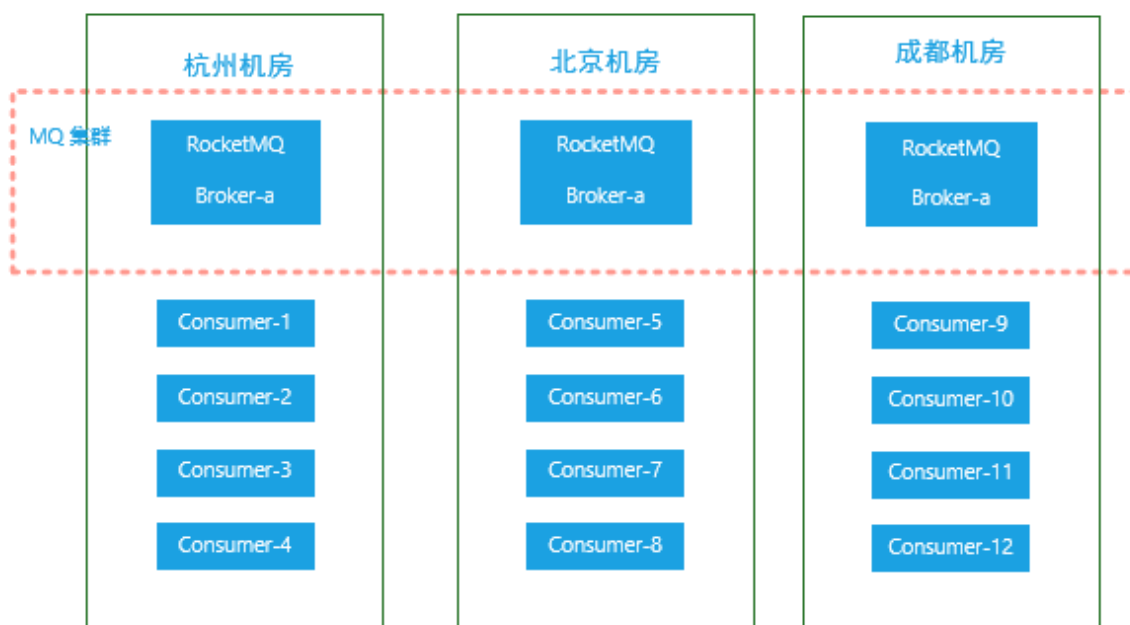




该算法会将consumer的hash值作为Node节点存放到hash环上，然后将queue的hash值也放到hash环上，通过顺时针方向，距离queue最近的那个consumer就是该queue要分配的consumer。

该算法存在的问题：分配不均。

同机房策略



该算法会根据queue的部署机房位置和consumer的位置，过滤出当前consumer相同机房的queue。然后按照平均分配策略或环形平均策略对同机房queue进行分配。如果没有同机房queue，则按照平均分配策略或环形平均策略对所有queue进行分配。

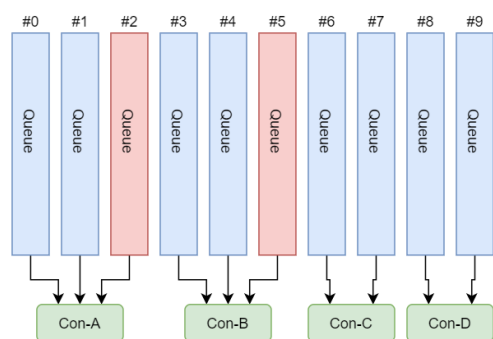
对比

一致性hash算法存在的问题：

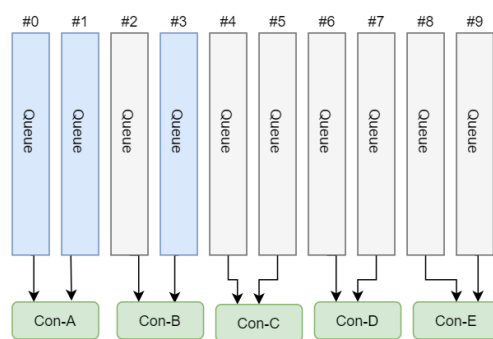
两种平均分配策略的分配效率较高，一致性hash策略的较低。因为一致性hash算法较复杂。另外，一致性hash策略分配的结果也很大可能上存在不平均的情况。

一致性hash算法存在的意义：

其可以有效减少由于消费者组扩容或缩容所带来的大量的Rebalance。

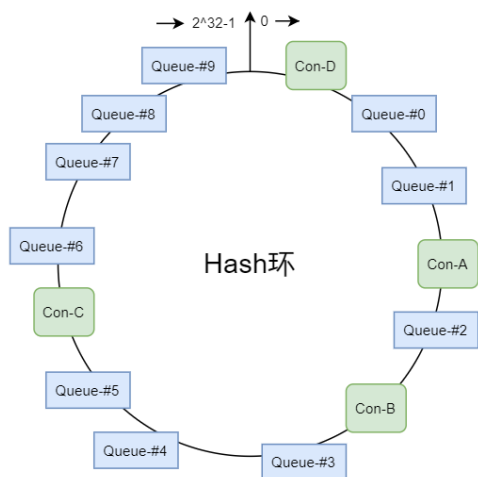


扩容前的分配情况

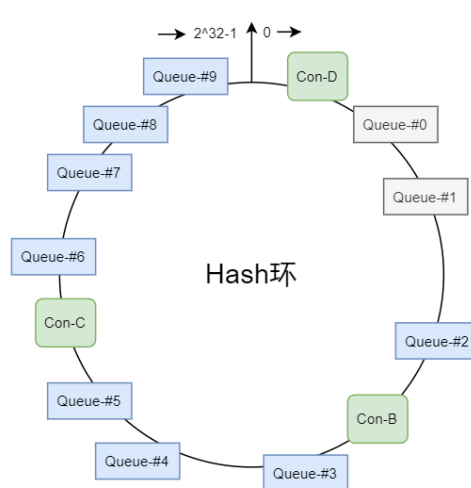


Rebalance比例占70%

扩容后的分配情况



缩容前的分配情况



Rebalance比例占20%

缩容后的分配情况

一致性hash算法的应用场景：

Consumer数量变化较频繁的场景。

5 至少一次原则

RocketMQ有一个原则：每条消息必须要被 **成功消费** 一次。

那么什么是成功消费呢？Consumer在消费完消息后会向其 **消费进度记录器** 提交其消费消息的offset，offset被成功记录到记录器中，那么这条消费就被成功消费了。

什么是消费进度记录器？

对于广播消费模式来说，Consumer本身就是消费进度记录器。

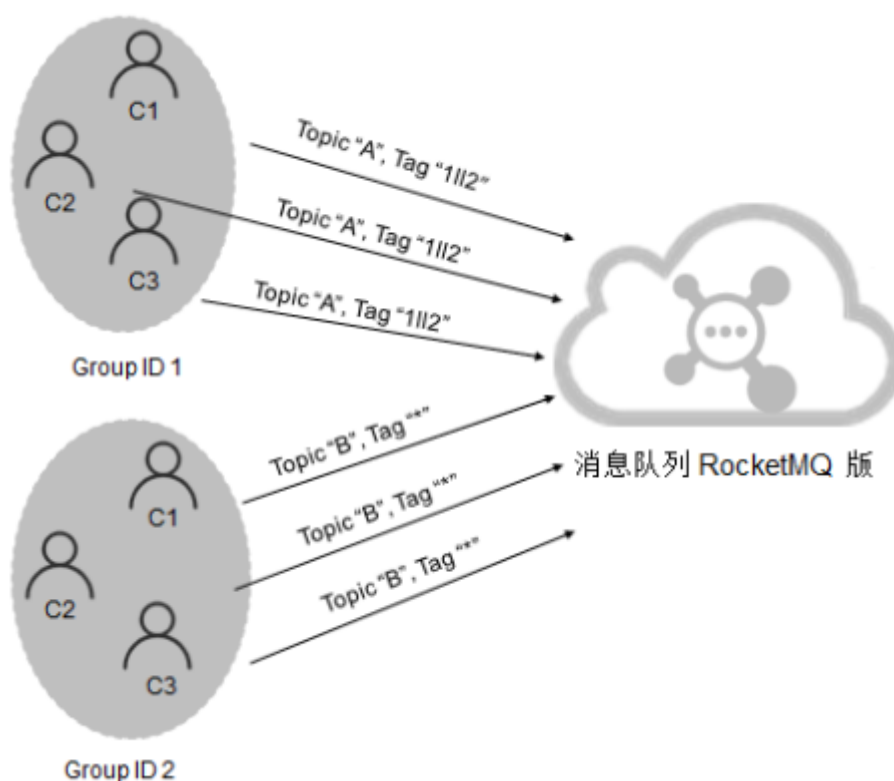
对于集群消费模式来说，Broker是消费进度记录器。

五、订阅关系的一致性

订阅关系的一致性指的是，同一个消费者组（Group ID相同）下所有Consumer实例所订阅的Topic与Tag及对消息的处理逻辑必须完全一致。否则，消息消费的逻辑就会混乱，甚至导致消息丢失。

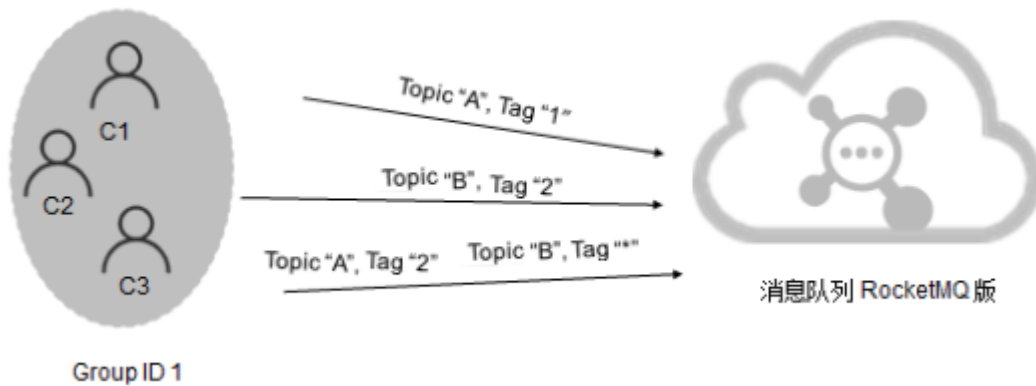
1 正确订阅关系

多个消费者组订阅了多个Topic，并且每个消费者组里的多个消费者实例的订阅关系保持了一致。



2 错误订阅关系

一个消费者组订阅了多个Topic，但是该消费者组里的多个Consumer实例的订阅关系并没有保持一致。



订阅了不同Topic

该例中的错误在于，同一个消费者组中的两个Consumer实例订阅了不同的Topic。

Consumer实例1-1：（订阅了topic为jodie_test_A，tag为所有的消息）

```

1 Properties properties = new Properties();
2 properties.put(PropertyKeyConst.GROUP_ID, "GID_jodie_test_1");
3 Consumer consumer = ONSFactory.createConsumer(properties);
4 consumer.subscribe("jodie_test_A", "*", new MessageListener() {
5     public Action consume(Message message, ConsumeContext context) {
6         System.out.println(message.getMsgID());
7         return Action.CommitMessage;
8     }
9 });

```

Consumer实例1-2：（订阅了topic为jodie_test_B，tag为所有的消息）

```

1 Properties properties = new Properties();
2 properties.put(PropertyKeyConst.GROUP_ID, "GID_jodie_test_1");
3 Consumer consumer = ONSFactory.createConsumer(properties);
4 consumer.subscribe("jodie_test_B", "*", new MessageListener() {
5     public Action consume(Message message, ConsumeContext context) {
6         System.out.println(message.getMsgID());
7         return Action.CommitMessage;
8     }
9 });

```

订阅了不同Tag

该例中的错误在于，同一个消费者组中的两个Consumer订阅了相同Topic的不同Tag。

Consumer实例2-1：（订阅了topic为jodie_test_A，tag为TagA的消息）

```

1 Properties properties = new Properties();
2 properties.put(PropertyKeyConst.GROUP_ID, "GID_jodie_test_2");
3 Consumer consumer = ONSFactory.createConsumer(properties);
4 consumer.subscribe("jodie_test_A", "TagA", new MessageListener() {
5     public Action consume(Message message, ConsumeContext context) {
6         System.out.println(message.getMsgID());
7         return Action.CommitMessage;
8     }
9 });

```

Consumer实例2-2：（订阅了topic为jodie_test_A, tag为所有的消息）

```

1 Properties properties = new Properties();
2 properties.put(PropertyKeyConst.GROUP_ID, "GID_jodie_test_2");
3 Consumer consumer = ONSFactory.createConsumer(properties);
4 consumer.subscribe("jodie_test_A", "*", new MessageListener() {
5     public Action consume(Message message, ConsumeContext context) {
6         System.out.println(message.getMsgID());
7         return Action.CommitMessage;
8     }
9 });

```

订阅了不同数量的Topic

该例中的错误在于，同一个消费者组中的两个Consumer订阅了不同数量的Topic。

Consumer实例3-1：（该Consumer订阅了两个Topic）

```

1 Properties properties = new Properties();
2 properties.put(PropertyKeyConst.GROUP_ID, "GID_jodie_test_3");
3 Consumer consumer = ONSFactory.createConsumer(properties);
4 consumer.subscribe("jodie_test_A", "TagA", new MessageListener() {
5     public Action consume(Message message, ConsumeContext context) {
6         System.out.println(message.getMsgID());
7         return Action.CommitMessage;
8     }
9 });
10 consumer.subscribe("jodie_test_B", "TagB", new MessageListener() {
11     public Action consume(Message message, ConsumeContext context) {
12         System.out.println(message.getMsgID());
13         return Action.CommitMessage;
14     }
15 });

```

Consumer实例3-2：（该Consumer订阅了一个Topic）

```

1 Properties properties = new Properties();
2 properties.put(PropertyKeyConst.GROUP_ID, "GID_jodie_test_3");
3 Consumer consumer = ONSFactory.createConsumer(properties);
4 consumer.subscribe("jodie_test_A", "TagB", new MessageListener() {
5     public Action consume(Message message, ConsumeContext context) {
6         System.out.println(message.getMsgID());
7         return Action.CommitMessage;
8     }
9 });

```

六、offset管理

这里的offset指的是Consumer的消费进度offset。

消费进度offset是用来记录每个Queue的不同消费组的消费进度的。根据消费进度记录器的不同，可以分为两种模式：本地模式和远程模式。

1 offset本地管理模式

当消费模式为**广播消费**时，offset使用本地模式存储。因为每条消息会被所有的消费者消费，每个消费者管理自己的消费进度，各个消费者之间不存在消费进度的交集。

Consumer在广播消费模式下offset相关数据以json的形式持久化到Consumer本地磁盘文件中，默认文件路径为当前用户主目录下的 `.rocketmq_offsets/${clientId}/${group}/Offsets.json`。其中`${clientId}`为当前消费者id，默认为`ip@DEFAULT`；`${group}`为消费者组名称。

2 offset远程管理模式

当消费模式为**集群消费**时，offset使用远程模式管理。因为所有Consumer实例对消息采用的是均衡消费，所有Consumer共享Queue的消费进度。

Consumer在集群消费模式下offset相关数据以json的形式持久化到Broker磁盘文件中，文件路径为当前用户主目录下的 `store/config/consumerOffset.json`。

Broker启动时会加载这个文件，并写入到一个双层Map（ConsumerOffsetManager）。外层map的key为`topic@group`，value为内层map。内层map的key为`queueId`，value为`offset`。当发生Rebalance时，新的Consumer会从该Map中获取到相应的数据来继续消费。

集群模式下offset采用远程管理模式，主要是为了保证Rebalance机制。

3 offset用途

消费者是如何从最开始持续消费消息的？消费者要消费的第一条消息的起始位置是用户自己通过`consumer.setConsumeFromWhere()`方法指定的。

在Consumer启动后，其要消费的第一条消息的起始位置常用的有三种，这三种位置可以通过枚举类型常量设置。这个枚举类型为ConsumeFromWhere。

```
public enum ConsumeFromWhere {  
    CONSUME_FROM_LAST_OFFSET,  
  
    @Deprecated  
    CONSUME_FROM_LAST_OFFSET_AND_FROM_MIN_WHEN_BOOT_FIRST,  
    @Deprecated  
    CONSUME_FROM_MIN_OFFSET,  
    @Deprecated  
    CONSUME_FROM_MAX_OFFSET,  
    CONSUME_FROM_FIRST_OFFSET,  
    CONSUME_FROM_TIMESTAMP,  
}
```

CONSUME_FROM_LAST_OFFSET: 从queue的当前最后一条消息开始消费

CONSUME_FROM_FIRST_OFFSET: 从queue的第一条消息开始消费

CONSUME_FROM_TIMESTAMP: 从指定的具体时间戳位置的消息开始消费。这个具体时间戳是通过另外一个语句指定的。

`consumer.setConsumeTimestamp("20210701080000")` yyyyMMddHHmmss

当消费完一批消息后，Consumer会提交其消费进度offset给Broker，Broker在收到消费进度后会将其更新到那个双层Map（ConsumerOffsetManager）及consumerOffset.json文件中，然后向该Consumer进行ACK，而ACK内容中包含三项数据：当前消费队列的最小offset（minOffset）、最大offset（maxOffset）、及下次消费的起始offset（nextBeginOffset）。

4 重试队列

```

[root@mq0S config]# cat consumerOffset.json
{
  "offsetTable":{
    "%RETRY%cg@cg":{0:0
    },
    "somTopic@cg":{0:0,1:1,2:0,3:1
    },
    "someTopic@myConsumerGroup":{0:8,1:7,2:7,3:8
    },
    "persons@persons-group":{0:6,1:7,2:6,3:6
    },
    "%RETRY%persons-group@persons-group":{0:0
    },
    "abcxxx@persons-group":{0:7,1:6,2:3,3:3
    },
    "abc000@persons-group":{0:8,1:11,2:11,3:8
    },
    "someTopic@cg":{0:29,1:31,2:33,3:33
    },
    "TopicTest@please_rename_unique_group_name_4":{0:294,1:
    },
    "RMQ_SYS_TRANS_HALF_TOPIC@CID_RMQ_SYS_TRANS":{0:18
    },
    "%RETRY%please_rename_unique_group_name_4@please_rename
    },
    "aaaxxxaaa@persons-group":{0:3,1:5,2:4,3:2
    },
    "%RETRY%myConsumerGroup@myConsumerGroup":{0:0
    },
    "RMQ_SYS_TRANS_OP_HALF_TOPIC@CID_RMQ_SYS_TRANS":{0:12
    }
  }
}[root@mq0S config]#

```

当rocketMQ对消息的消费出现异常时，会将发生异常的消息的offset提交到Broker中的重试队列。系统在发生消息消费异常时会为当前的topic@group创建一个重试队列，该队列以%RETRY%开头，到达重试时间后进行消费重试。

5 offset的同步提交与异步提交

集群消费模式下，Consumer消费完消息后会向Broker提交消费进度offset，其提交方式分为两种：

同步提交：消费者在消费完一批消息后会向broker提交这些消息的offset，然后等待broker的成功响应。若在等待超时之前收到了成功响应，则继续读取下一批消息进行消费（从ACK中获取nextBeginOffset）。若没有收到响应，则会重新提交，直到获取到响应。而在这个等待过程中，消费者是阻塞的。其严重影响了消费者的吞吐量。

异步提交：消费者在消费完一批消息后向broker提交offset，但无需等待Broker的成功响应，可以继续读取并消费下一批消息。这种方式增加了消费者的吞吐量。但需要注意，broker在收到提交的offset后，还是会向消费者进行响应的。可能还没有收到ACK，此时Consumer会从Broker中直接获取nextBeginOffset。

七、消费幂等

1 什么是消费幂等

当出现消费者对某条消息重复消费的情况时，重复消费的结果与消费一次的结果是相同的，并且多次消费并未对业务系统产生任何负面影响，那么这个消费过程就是消费幂等的。

■ 幂等：若某操作执行多次与执行一次对系统产生的影响是相同的，则称该操作是幂等的。

在互联网应用中，尤其在网络不稳定的情况下，消息很有可能会出现重复发送或重复消费。如果重复的消息可能会影响业务处理，那么就应该对消息做幂等处理。

2 消息重复的场景分析

什么情况下可能会出现消息被重复消费呢？最常见的有以下三种情况：

发送时消息重复

当一条消息已被成功发送到Broker并完成持久化，此时出现了网络闪断，从而导致Broker对Producer应答失败。如果此时Producer意识到消息发送失败并尝试再次发送消息，此时Broker中就可能会出现两条内容相同并且Message ID也相同的消息，那么后续Consumer就一定会消费两次该消息。

消费时消息重复

消息已投递到Consumer并完成业务处理，当Consumer给Broker反馈应答时网络闪断，Broker没有接收到消费成功响应。为了保证消息 **至少被消费一次** 的原则，Broker将在网络恢复后再次尝试投递之前已被处理过的消息。此时消费者就会收到与之前处理过的内容相同、Message ID也相同的消息。

Rebalance时消息重复

当Consumer Group中的Consumer数量发生变化时，或其订阅的Topic的Queue数量发生变化时，会触发Rebalance，此时Consumer可能会收到曾经被消费过的消息。

3 通用解决方案

两要素

幂等解决方案的设计中涉及到两项要素：幂等令牌，与唯一性处理。只要充分利用好这两要素，就可以设计出好的幂等解决方案。

- 幂等令牌：是生产者和消费者两者中的既定协议，通常指具备唯一业务标识的字符串。例如，订单号、流水号。一般由Producer随着消息一同发送来的。
- 唯一性处理：服务端通过采用一定的算法策略，保证同一个业务逻辑不会被重复执行成功多次。例如，对同一笔订单的多次支付操作，只会成功一次。

解决方案

对于常见的系统，幂等性操作的通用性解决方案是：

1. 首先通过缓存去重。在缓存中如果已经存在了某幂等令牌，则说明本次操作是重复性操作；若缓存没有命中，则进入下一步。

2. 在唯一性处理之前，先在数据库中查询幂等令牌作为索引的数据是否存在。若存在，则说明本次操作为重复性操作；若不存在，则进入下一步。
3. 在同一事务中完成三项操作：唯一性处理后，将幂等令牌写入到缓存，并将幂等令牌作为唯一索引的数据写入到DB中。

第1步已经判断过是否是重复性操作了，为什么第2步还要再次判断？能够进入第2步，说明已经不是重复操作了，第2次判断是否重复？

当然不重复。一般缓存中的数据是具有有效期的。缓存中数据的有效期一旦过期，就是发生缓存穿透，使请求直接就到达了DBMS。

解决方案举例

以支付场景为例：

1. 当支付请求到达后，首先在Redis缓存中获取key为 **支付流水号** 的缓存value。若value不空，则说明本次支付是重复操作，业务系统直接返回调用侧重复支付标识；若value为空，则进入下一步操作
2. 到DBMS中根据 **支付流水号** 查询是否存在相应实例。若存在，则说明本次支付是重复操作，业务系统直接返回调用侧重复支付标识；若不存在，则说明本次操作是首次操作，进入下一步完成唯一性处理
3. 在分布式事务中完成三项操作：
 - 完成支付任务
 - 将当前 **支付流水号** 作为key，任意字符串作为value，通过set(key, value, expireTime)将数据写入到Redis缓存
 - 将当前 **支付流水号** 作为主键，与其它相关数据共同写入到DBMS

4 消费幂等的实现

消费幂等的解决方案很简单：为消息指定不会重复的唯一标识。因为Message ID有可能出现重复的情况，所以真正安全的幂等处理，不建议以Message ID作为处理依据。最好的方式是以业务唯一标识作为幂等处理的关键依据，而业务的唯一标识可以通过消息Key设置。

以支付场景为例，可以将消息的Key设置为订单号，作为幂等处理的依据。具体代码示例如下：

```
1 Message message = new Message();
2 message.setKey("ORDERID_100");
3 SendResult sendResult = producer.send(message);
```

消费者收到消息时可以根据消息的Key即订单号来实现消费幂等：

```

1 consumer.registerMessageListener(new MessageListenerConcurrently() {
2     @Override
3     public ConsumeConcurrentlyStatus consumeMessage(List<MessageExt>
4 msgs,
5     ConsumeConcurrentlyContext
6 context) {
7         for(MessageExt msg:msgs){
8             String key = msg.getKeys();
9             // 根据业务唯一标识key做幂等处理
10            // .....
11        }
12        return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
13    }
14 });

```

RocketMQ能够保证消息不丢失，但不能保证消息不重复。

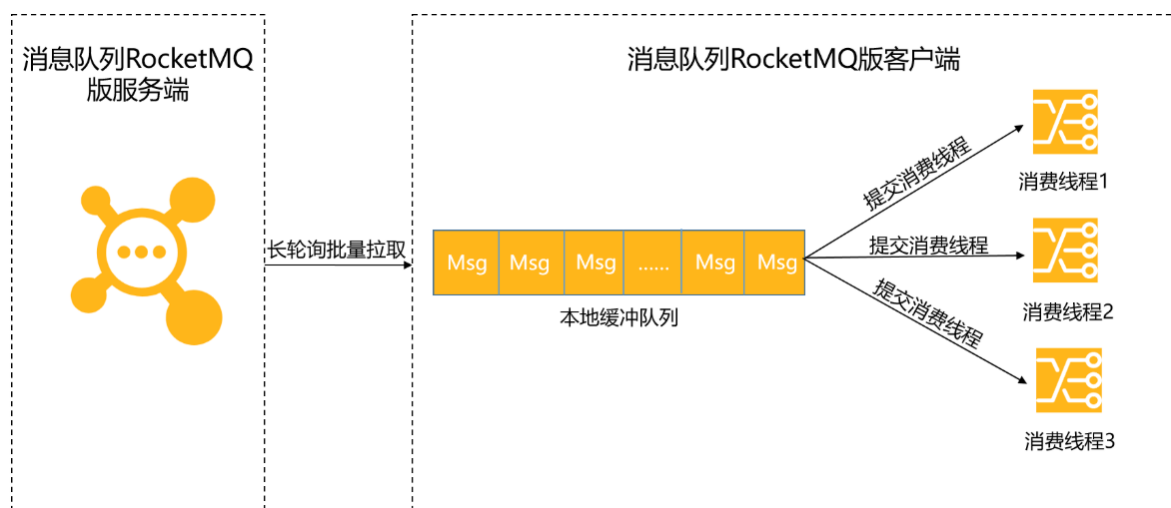
八、消息堆积与消费延迟

1 概念

消息处理流程中，如果Consumer的消费速度跟不上Producer的发送速度，MQ中未处理的消息会越来越多（进的多出的少），这部分消息就被称为**堆积消息**。消息出现堆积进而会造成消息的**消费延迟**。以下场景需要重点关注消息堆积和消费延迟问题：

- 业务系统上下游能力不匹配造成的持续堆积，且无法自行恢复。
- 业务系统对消息的消费实时性要求较高，即使是短暂的堆积造成的消费延迟也无法接受。

2 产生原因分析



Consumer使用长轮询Pull模式消费消息时，分为以下两个阶段：

消息拉取

Consumer通过长轮询Pull模式批量拉取的方式从服务端获取消息，将拉取到的消息缓存到本地缓冲队列中。对于拉取式消费，在内网环境下会有很高的吞吐量，所以这一阶段一般不会成为消息堆积的瓶颈。

一个单线程单分区的低规格主机(Consumer, 4C8G)，其可达到几万的TPS。如果是多个分区多个线程，则可以轻松达到几十万的TPS。

消息消费

Consumer将本地缓存的消息提交到消费线程中，使用业务消费逻辑对消息进行处理，处理完毕后获取到一个结果。这是真正的消息消费过程。此时Consumer的消费能力就完全依赖于消息的**消费耗时**和**消费并发度**了。如果由于业务处理逻辑复杂等原因，导致处理单条消息的耗时较长，则整体的消息吞吐量肯定不会高，此时就会导致Consumer本地缓冲队列达到上限，停止从服务端拉取消息。

结论

消息堆积的主要瓶颈在于客户端的消费能力，而消费能力由**消费耗时**和**消费并发度**决定。注意，消费耗时的优先级要高于消费并发度。即在保证了消费耗时的合理性前提下，再考虑消费并发度问题。

3 消费耗时

影响消息处理时长的主要因素是代码逻辑。而代码逻辑中可能会影响处理时长代码主要有两种类型：**CPU内部计算型代码**和**外部I/O操作型代码**。

通常情况下代码中如果没有复杂的递归和循环的话，内部计算耗时相对外部I/O操作来说几乎可以忽略。所以外部IO型代码是影响消息处理时长的主要症结所在。

外部I/O操作型代码举例：

- 读写外部数据库，例如对远程MySQL的访问
- 读写外部缓存系统，例如对远程Redis的访问
- 下游系统调用，例如Dubbo的RPC远程调用，Spring Cloud的对下游系统的Http接口调用

关于下游系统调用逻辑需要进行提前梳理，掌握每个调用操作预期的耗时，这样做是为了能够判断消费逻辑中IO操作的耗时是否合理。通常消息堆积是由于下游系统出现了**服务异常**或**达到了DBMS容量限制**，导致消费耗时增加。

服务异常，并不仅仅是系统中出现的类似500这样的代码错误，而可能是更加隐蔽的问题。例如，网络带宽问题。

达到了DBMS容量限制，其也会引发消息的消费耗时增加。

4 消费并发度

一般情况下，消费者端的消费并发度由单节点线程数和节点数量共同决定，其值为 **单节点线程数 * 节点数量**。不过，通常需要优先调整单节点的线程数，若单机硬件资源达到了上限，则需要通过横向扩展来提高消费并发度。

单节点线程数，即单个Consumer所包含的线程数量

节点数量，即Consumer Group所包含的Consumer数量

对于普通消息、延时消息及事务消息，并发度计算都是 **单节点线程数 * 节点数量**。但对于顺序消息则是不同的。顺序消息的消费并发度等于 **Topic的Queue分区数量**。

1) 全局顺序消息：该类型消息的Topic只有一个Queue分区。其可以保证该Topic的所有消息被顺序消费。为了保证这个全局顺序性，Consumer Group中在同一时刻只能有一个Consumer的一个线程进行消费。所以其并发度为1。

2) 分区顺序消息：该类型消息的Topic有多个Queue分区。其仅可以保证该Topic的每个Queue分区中的消息被顺序消费，不能保证整个Topic中消息的顺序消费。为了保证这个分区顺序性，每个Queue分区中的消息在Consumer Group中的同一时刻只能有一个Consumer的一个线程进行消费。即，在同一时刻最多会出现多个Queue分区有多个Consumer的多个线程并行消费。所以其并发度为Topic的分区数量。

5 单机线程数计算

对于一台主机中线程池中线程数的设置需要谨慎，不能盲目直接调大线程数，设置过大的线程数反而会带来大量的线程切换的开销。理想环境下单节点的最优线程数计算模型为： $C * (T1 + T2) / T1$ 。

- C：CPU内核数
- T1：CPU内部逻辑计算耗时
- T2：外部IO操作耗时

最优线程数 = $C * (T1 + T2) / T1 = C * T1 / T1 + C * T2 / T1 = C + C * T2 / T1$

注意，该计算出的数值是理想状态下的理论数据，在生产环境中，不建议直接使用。而是根据当前环境，先设置一个比该值小的数值然后观察其压测效果，然后再根据效果逐步调大线程数，直至找到在该环境中性能最佳时的值。

6 如何避免

为了避免在业务使用时出现非预期的消息堆积和消费延迟问题，需要在前期设计阶段对整个业务逻辑进行完善的排查和梳理。其中最重要的就是 **梳理消息的消费耗时** 和 **设置消息消费的并发度**。

梳理消息的消费耗时

通过压测获取消息的消费耗时，并对耗时较高的操作的代码逻辑进行分析。梳理消息的消费耗时需要关注以下信息：

- 消息消费逻辑的计算复杂度是否过高，代码是否存在无限循环和递归等缺陷。

- 消息消费逻辑中的I/O操作是否是必须的，能否用本地缓存等方案规避。
- 消费逻辑中的复杂耗时的操作是否可以做异步化处理。如果可以，是否会造成逻辑错乱。

设置消费并发度

对于消息消费并发度的计算，可以通过以下两步实施：

- 逐步调大单个Consumer节点的线程数，并观测节点的系统指标，得到单个节点最优的消费线程数和消息吞吐量。
- 根据上下游链路的 **流量峰值** 计算出需要设置的节点数

节点数 = 流量峰值 / 单个节点消息吞吐量

九、消息的清理

消息被消费过后会被清理掉吗？不会的。

消息是被顺序存储在commitlog文件的，且消息大小不定长，所以消息的清理是不可能以消息为单位进行清理的，而是以commitlog文件为单位进行清理的。否则会急剧下降清理效率，并实现逻辑复杂。

commitlog文件存在一个 **过期时间**，默认为72小时，即三天。除了用户手动清理外，在以下情况下也会被自动清理，无论文件中的消息是否被消费过：

- 文件过期，且到达 **清理时间点**（默认为凌晨4点）后，自动清理过期文件
- 文件过期，且磁盘空间占用率已达 **过期清理警戒线**（默认75%）后，无论是否达到清理时间点，都会自动清理过期文件
- 磁盘占用率达到 **清理警戒线**（默认85%）后，开始按照设定好的规则清理文件，无论是否过期。默认会从最老的文件开始清理
- 磁盘占用率达到 **系统危险警戒线**（默认90%）后，Broker将拒绝消息写入

需要注意以下几点：

- 1) 对于RocketMQ系统来说，删除一个1G大小的文件，是一个压力巨大的IO操作。在删除过程中，系统性能会骤然下降。所以，其默认清理时间点为凌晨4点，访问量最小的时间。也正因如此，我们要保障磁盘空间的空闲率，不要使系统出现在其它时间点删除commitlog文件的情况。
- 2) 官方建议RocketMQ服务的Linux文件系统采用ext4。因为对于文件删除操作，ext4要比ext3性能更好

第4章 RocketMQ应用

一、普通消息

1 消息发送分类

Producer对于消息的发送方式也有多种选择，不同的方式会产生不同的系统效果。

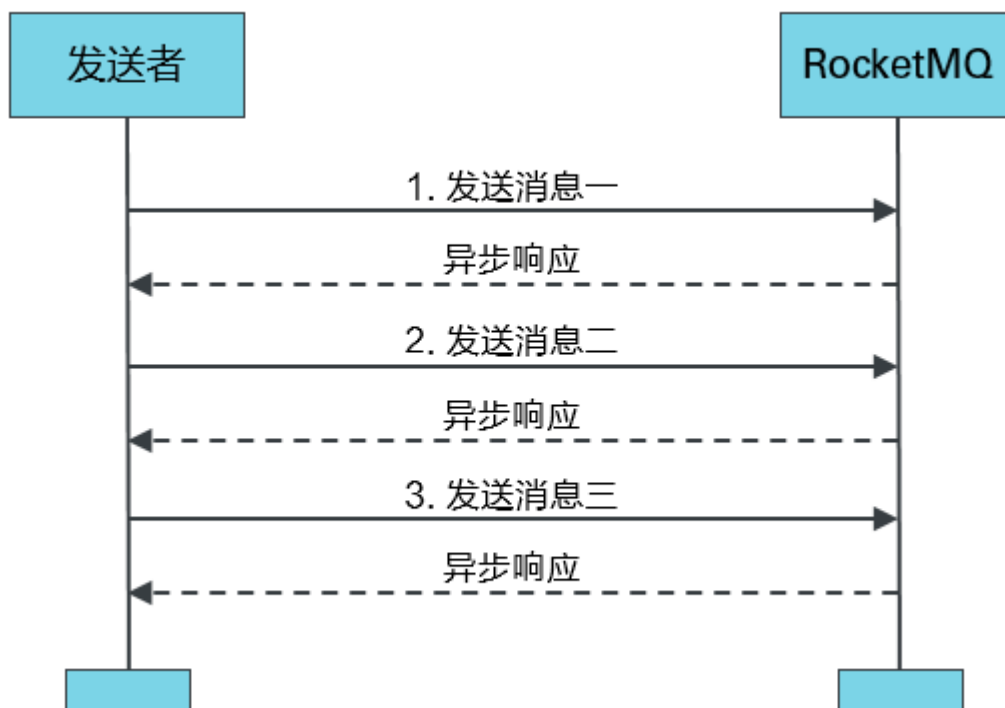
同步发送消息

同步发送消息是指，Producer发出一条消息后，会在收到MQ返回的ACK之后才发下一条消息。该方式的消息可靠性最高，但消息发送效率太低。



异步发送消息

异步发送消息是指，Producer发出消息后无需等待MQ返回ACK，直接发送下一条消息。该方式的消息可靠性可以得到保障，消息发送效率也可以。



单向发送消息

单向发送消息是指，Producer仅负责发送消息，不等待、不处理MQ的ACK。该发送方式时MQ也不返回ACK。该方式的消息发送效率最高，但消息可靠性较差。



2 代码举例

创建工程

创建一个Maven的Java工程rocketmq-test。

导入依赖

导入rocketmq的client依赖。

```
1     <properties>
2         <project.build.sourceEncoding>UTF-
8</project.build.sourceEncoding>
3         <maven.compiler.source>1.8</maven.compiler.source>
4         <maven.compiler.target>1.8</maven.compiler.target>
5     </properties>
6
7     <dependencies>
8         <dependency>
9             <groupId>org.apache.rocketmq</groupId>
10            <artifactId>rocketmq-client</artifactId>
11            <version>4.8.0</version>
12        </dependency>
13    </dependencies>
```

定义同步消息发送生产者

```
1 public class SyncProducer {
2     public static void main(String[] args) throws Exception {
3         // 创建一个producer, 参数为Producer Group名称
4         DefaultMQProducer producer = new DefaultMQProducer("pg");
5         // 指定nameServer地址
6         producer.setNamesrvAddr("rocketmqOS:9876");
7         // 设置当发送失败时重试发送的次数, 默认为2次
8         producer.setRetryTimesWhenSendFailed(3);
9         // 设置发送超时时限为5s, 默认3s
10        producer.setSendMessageTimeout(5000);
11
12        // 开启生产者
13        producer.start();
14
15        // 生产并发送100条消息
16        for (int i = 0; i < 100; i++) {
17            byte[] body = ("Hi," + i).getBytes();
18            Message msg = new Message("someTopic", "someTag", body);
19            // 为消息指定key
20            msg.setKeys("key-" + i);
21            // 发送消息
22            SendResult sendResult = producer.send(msg);
23            System.out.println(sendResult);
```

```

24     }
25     // 关闭producer
26     producer.shutdown();
27 }
28 }

```

```

1 // 消息发送的状态
2 public enum SendStatus {
3     SEND_OK, // 发送成功
4     FLUSH_DISK_TIMEOUT, // 刷盘超时。当Broker设置的刷盘策略为同步刷盘时才可能出现这种异常状态。异步刷盘不会出现
5     FLUSH_SLAVE_TIMEOUT, // Slave同步超时。当Broker集群设置的Master-Slave的复制方式为同步复制时才可能出现这种异常状态。异步复制不会出现
6     SLAVE_NOT_AVAILABLE, // 没有可用的Slave。当Broker集群设置为Master-Slave的复制方式为同步复制时才可能出现这种异常状态。异步复制不会出现
7 }

```

定义异步消息发送生产者

```

1 public class AsyncProducer {
2     public static void main(String[] args) throws Exception {
3         DefaultMQProducer producer = new DefaultMQProducer("pg");
4         producer.setNamesrvAddr("rocketmqOS:9876");
5         // 指定异步发送失败后不进行重试发送
6         producer.setRetryTimesWhenSendAsyncFailed(0);
7         // 指定新创建的Topic的Queue数量为2，默认为4
8         producer.setDefaultTopicQueueNums(2);
9
10        producer.start();
11
12        for (int i = 0; i < 100; i++) {
13            byte[] body = ("Hi," + i).getBytes();
14            try {
15                Message msg = new Message("myTopicA", "myTag", body);
16                // 异步发送。指定回调
17                producer.send(msg, new SendCallback() {
18                    // 当producer接收到MQ发送来的ACK后就会触发该回调方法的执行
19                    @Override
20                    public void onSuccess(SendResult sendResult) {
21                        System.out.println(sendResult);
22                    }
23
24                    @Override
25                    public void onException(Throwable e) {
26                        e.printStackTrace();
27                    }
28                });

```

```

29         } catch (Exception e) {
30             e.printStackTrace();
31         }
32     } // end-for
33
34     // sleep一会儿
35     // 由于采用的是异步发送，所以若这里不sleep，
36     // 则消息还未发送就会将producer给关闭，报错
37     TimeUnit.SECONDS.sleep(3);
38     producer.shutdown();
39 }
40 }

```

定义单向消息发送生产者

```

1 public class OnewayProducer {
2     public static void main(String[] args) throws Exception{
3         DefaultMQProducer producer = new DefaultMQProducer("pg");
4         producer.setNamesrvAddr("rocketmqOS:9876");
5         producer.start();
6
7         for (int i = 0; i < 10; i++) {
8             byte[] body = ("Hi," + i).getBytes();
9             Message msg = new Message("single", "someTag", body);
10            // 单向发送
11            producer.sendOneway(msg);
12        }
13        producer.shutdown();
14        System.out.println("producer shutdown");
15    }
16 }
17

```

定义消息消费者

```

1 public class SomeConsumer {
2
3     public static void main(String[] args) throws MQClientException {
4         // 定义一个pull消费者
5         // DefaultLitePullConsumer consumer = new
DefaultLitePullConsumer("cg");
6         // 定义一个push消费者
7         DefaultMQPushConsumer consumer = new
DefaultMQPushConsumer("cg");
8         // 指定nameServer
9         consumer.setNamesrvAddr("rocketmqOS:9876");
10        // 指定从第一条消息开始消费

```

```
11 consumer.setConsumeFromWhere(ConsumeFromWhere.CONSUME_FROM_FIRST_OFFSET
12 );
13 // 指定消费topic与tag
14 consumer.subscribe("someTopic", "*");
15 // 指定采用“广播模式”进行消费，默认为“集群模式”
16 // consumer.setMessageModel(MessageModel.BROADCASTING);
17
18 // 注册消息监听器
19 consumer.registerMessageListener(new
20 MessageListenerConcurrently() {
21
22     // 一旦broker中有了其订阅的消息就会触发该方法的执行，
23     // 其返回值为当前consumer消费的状态
24     @Override
25     public ConsumeConcurrentlyStatus
26 consumeMessage(List<MessageExt> msgs,
27
28 ConsumeConcurrentlyContext context) {
29     // 逐条消费消息
30     for (MessageExt msg : msgs) {
31         System.out.println(msg);
32     }
33     // 返回消费状态：消费成功
34     return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
35 }
36 });
37
38 // 开启消费者消费
39 consumer.start();
40 System.out.println("Consumer Started");
41 }
42 }
```



二、顺序消息

1 什么是顺序消息

顺序消息指的是，严格按照消息的 **发送顺序** 进行 **消费** 的消息(FIFO)。

默认情况下生产者会把消息以Round Robin轮询方式发送到不同的Queue分区队列；而消费消息时会从多个Queue上拉取消息，这种情况下的发送和消费是不能保证顺序的。如果将消息仅发送到同一个Queue中，消费时也只从这个Queue上拉取消息，就严格保证了消息的顺序性。

2 为什么需要顺序消息

例如，现在有TOPIC **ORDER_STATUS** (订单状态)，其下有4个Queue队列，该Topic中的不同消息用于描述当前订单的不同状态。假设订单有状态：**未支付**、**已支付**、**发货中**、**发货成功**、**发货失败**。

根据以上订单状态，生产者从 **时序** 上可以生成如下几个消息：

订单T0000001:未支付 --> **订单T0000001:已支付** --> **订单T0000001:发货中** --> **订单T0000001:发货失败**

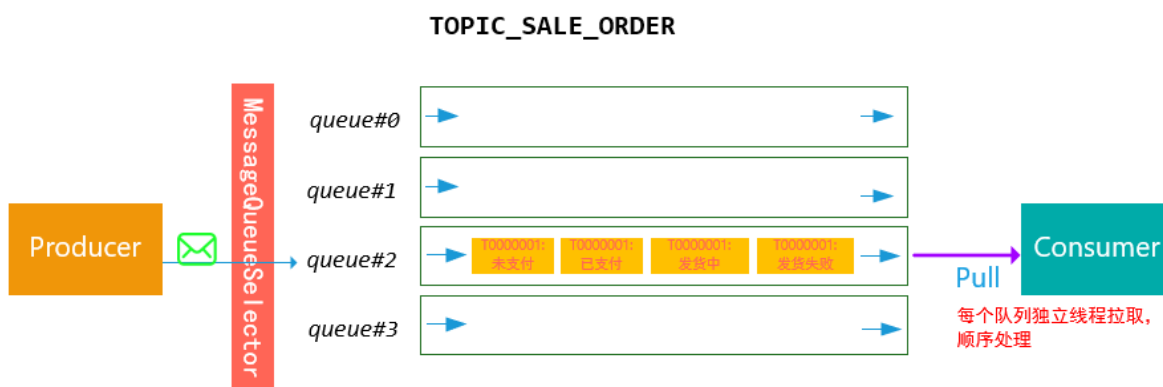
消息发送到MQ中之后，Queue的选择如果采用轮询策略，消息在MQ的存储可能如下：



消费消息的可能顺序

正确	订单T0000001:未支付	订单T0000001:已支付	订单T0000001:发货中	订单T0000001:发货失败
偏差	订单T0000001:已支付	订单T0000001:未支付	订单T0000001:发货失败	订单T0000001:发货中
偏差	订单T0000001:发货失败	订单T0000001:发货中	订单T0000001:未支付	订单T0000001:已支付
偏差	订单T0000001:发货失败	订单T0000001:未支付	订单T0000001:发货中	订单T0000001:已支付

这种情况下，我们希望Consumer消费消息的顺序和我们发送是一致的，然而上述MQ的投递和消费方式，我们无法保证顺序是正确的。对于顺序异常的消息，Consumer即使设置有一定的状态容错，也不能完全处理好这么多种随机出现组合情况。



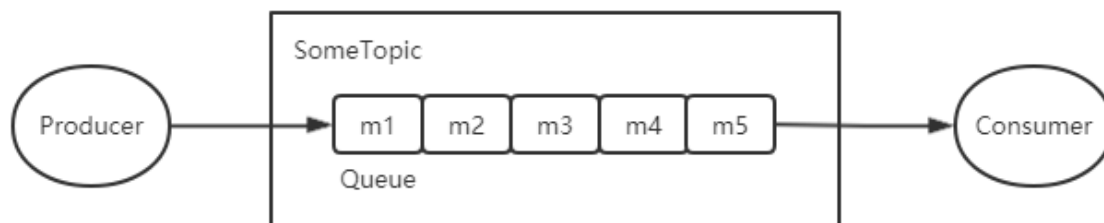
定义策略，顺序投递到特定的队列中

基于上述的情况，可以设计如下方案：对于相同订单号的消息，通过一定的策略，将其放置在一个Queue中，然后消费者再采用一定的策略（例如，一个线程独立处理一个queue，保证处理消息的顺序性），能够保证消费的顺序性。

3 有序性分类

根据有序范围的不同，RocketMQ可以严格地保证两种消息的有序性：**分区有序**与**全局有序**。

全局有序



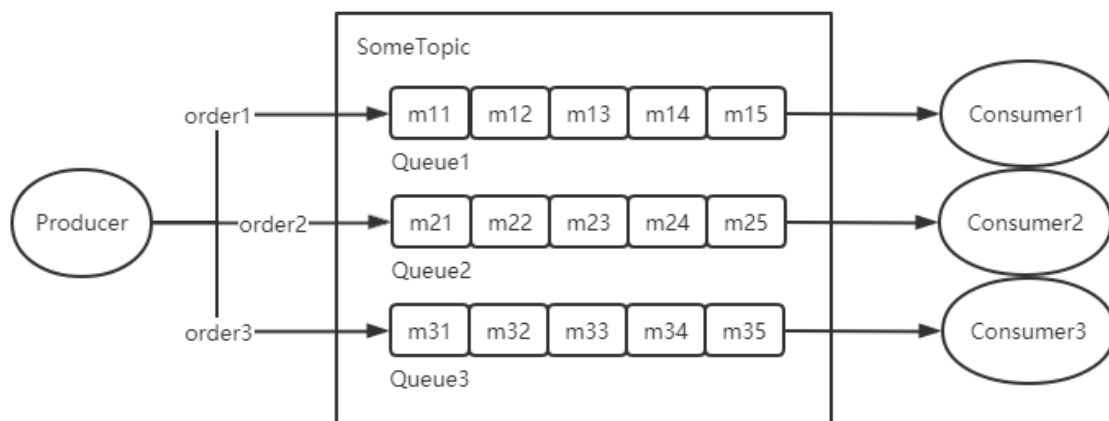
全局有序示意图

当发送和消费参与的Queue只有一个时所保证的有序是整个Topic中消息的顺序，称为**全局有序**。

在创建Topic时指定Queue的数量。有三种指定方式：

- 1) 在代码中创建Producer时，可以指定其自动创建的Topic的Queue数量
- 2) 在RocketMQ可视化控制台中手动创建Topic时指定Queue数量
- 3) 使用mqadmin命令手动创建Topic时指定Queue数量

分区有序



分区有序示意图

如果有多个Queue参与，其仅可保证在该Queue分区队列上的消息顺序，则称为**分区有序**。

如何实现Queue的选择？在定义Producer时我们可以指定消息队列选择器，而这个选择器是我们自己实现了MessageQueueSelector接口定义的。

在定义选择器的选择算法时，一般需要使用选择key。这个选择key可以是消息key也可以是其它数据。但无论谁做选择key，都不能重复，都是唯一的。

一般性的选择算法是，让选择key（或其hash值）与该Topic所包含的Queue的数量取模，其结果即为选择出的Queue的QueueId。

取模算法存在一个问题：不同选择key与Queue数量取模结果可能会是相同的，即不同选择key的消息可能会出现在相同的Queue，即同一个Consumer可能会消费到不同选择key的消息。这个问题如何解决？一般性的作法是，从消息中获取到选择key，对其进行判断。若是当前Consumer需要消费的消息，则直接消费，否则，什么也不做。这种做法要求选择key要能够随着消息一起被Consumer获取到。此时使用消息key作为选择key是比较好的做法。

以上做法会不会出现如下新的问题呢？不属于那个Consumer的消息被拉取走了，那么应该消费该消息的Consumer是否还能再消费该消息呢？同一个Queue中的消息不可能被同一个Group中的不同Consumer同时消费。所以，消费同一个Queue的不同选择key的消息的Consumer一定属于不同的Group。而不同的Group中的Consumer间的消费是相互隔离的，互不影响的。

4 代码举例

```
1 public class OrderedProducer {
2     public static void main(String[] args) throws Exception {
3         DefaultMQProducer producer = new DefaultMQProducer("pg");
4         producer.setNamesrvAddr("rocketmqos:9876");
5         producer.start();
6
7         for (int i = 0; i < 100; i++) {
8             Integer orderId = i;
9             byte[] body = ("Hi," + i).getBytes();
10            Message msg = new Message("Topica", "TagA", body);
11            SendResult sendResult = producer.send(msg, new
12            MessageQueueSelector() {
13                @Override
14                public MessageQueue select(List<MessageQueue> mqs,
15                Message msg, Object arg) {
16                    Integer id = (Integer) arg;
17                    int index = id % mqs.size();
18                    return mqs.get(index);
19                }
20            }, orderId);
21
22            System.out.println(sendResult);
23        }
24        producer.shutdown();
25    }
26 }
```

三、延时消息

1 什么是延时消息

当消息写入到Broker后，在指定的时长后才可被消费处理的消息，称为延时消息。

采用RocketMQ的延时消息可以实现 **定时任务** 的功能，而无需使用定时器。典型的应用场景是，电商交易中超时未支付关闭订单的场景，12306平台订票超时未支付取消订票的场景。

在电商平台中，订单创建时会发送一条延迟消息。这条消息将会在30分钟后投递给后台业务系统（Consumer），后台业务系统收到该消息后会判断对应的订单是否已经完成支付。如果未完成，则取消订单，将商品再次放回到库存；如果完成支付，则忽略。

在12306平台中，车票预订成功后就会发送一条延迟消息。这条消息将会在45分钟后投递给后台业务系统（Consumer），后台业务系统收到该消息后会判断对应的订单是否已经完成支付。如果未完成，则取消预订，将车票再次放回到票池；如果完成支付，则忽略。

2 延时等级

延时消息的延迟时长 **不支持随意时长** 的延迟，是通过特定的延迟等级来指定的。延时等级定义在RocketMQ服务端的 `MessageStoreConfig` 类中的如下变量中：

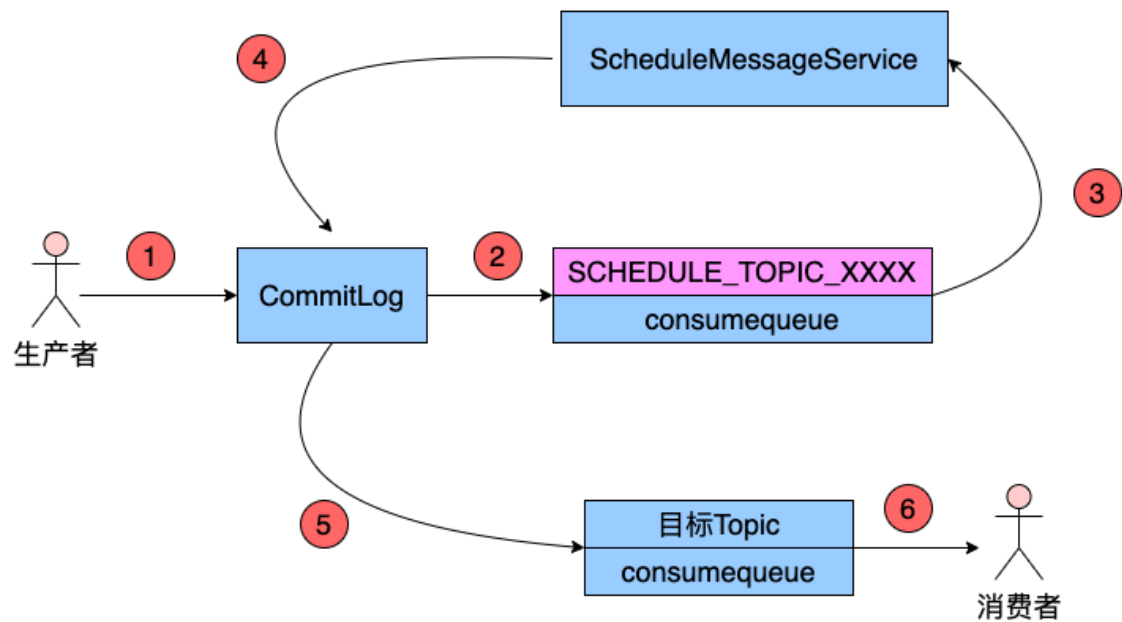
```
127 private FlushDiskType flushDiskType = FlushDiskType.ASYNC_FLUSH;
128 private int syncFlushTimeout = 1000 * 5;
129 private String messageDelayLevel = "1s 5s 10s 30s 1m 2m 3m 4m 5m 6m 7m 8m 9m 10m 20m 30m 1h 2h";
130 private long flushDelayOffsetInterval = 1000 * 10;
131 @ImportantField
```

即，若指定的延时等级为3，则表示延迟时长为10s，即延迟等级是从1开始计数的。

当然，如果需要自定义的延时等级，可以通过在broker加载的配置中新增如下配置（例如下面增加了1天这个等级1d）。配置文件在RocketMQ安装目录下的conf目录中。

```
1 messageDelayLevel = 1s 5s 10s 30s 1m 2m 3m 4m 5m 6m 7m 8m 9m 10m 20m 30m
  1h 2h 1d
```

3 延时消息实现原理



具体实施方案是：

修改消息

```

1  consumequeue
2  |— SCHEDULE_TOPIC_XXXX
3  |   |
4  |   |— 0
5  |   |   | 00000000000000000000
6  |   |— 1
7  |   |   | 00000000000000000000
8  |   |— 2
9  |   |   | 00000000000000000000
10 |   |— 3
11 |   |   | 00000000000000000000
12 |   |— 4
13 |   |   | 00000000000000000000
14 |   |.....
15 |   |.....
16 |— DelayTopicTest
17 |   |— 0
18 |   |   | 00000000000000000000
19 |   |— 1
20 |   |   | 00000000000000000000
21 |   |— 2
22 |   |   | 00000000000000000000
23 |   |— 3
24 |   |   | 00000000000000000000
  
```

Producer将消息发送到Broker后，Broker会首先将消息写入到commitlog文件，然后需要将其分发到相应的consumequeue。不过，在分发之前，系统会先判断消息中是否带有延时等级。若没有，则直接正常分发；若有则需要经历一个复杂的过程：

- 修改消息的Topic为SCHEDULE_TOPIC_XXXX

- 根据延时等级，在consumequeue目录中SCHEDULE_TOPIC_XXXX主题下创建出相应的queueId目录与consumequeue文件（如果没有这些目录与文件的话）。

延迟等级delayLevel与queueId的对应关系为 $queueId = delayLevel - 1$

需要注意，在创建queueId目录时，并不是一次性地将所有延迟等级对应的目录全部创建完毕，而是用到哪个延迟等级创建哪个目录



- 修改消息索引单元内容。索引单元中的Message Tag hashCode部分原本存放的是消息的Tag的Hash值。现修改为消息的 **投递时间**。投递时间是指该消息被重新修改为原Topic后再次被写入到commitlog中的时间。**投递时间 = 消息存储时间 + 延时等级时间**。消息存储时间指的是消息被发送到Broker时的时间戳。
- 将消息索引写入到SCHEDULE_TOPIC_XXXX主题下相应的consumequeue中

SCHEDULE_TOPIC_XXXX目录中各个延时等级Queue中的消息是如何排序的？

是按照消息投递时间排序的。一个Broker中同一等级的所有延时消息会被写入到consumequeue目录中SCHEDULE_TOPIC_XXXX目录下相同Queue中。即一个Queue中消息投递时间的延迟等级时间是相同的。那么投递时间就取决于于**消息存储时间**了。即按照消息被发送到Broker的时间进行排序的。

投递延时消息

Broker内部有一个延迟消息服务类ScheuleMessageService，其会消费SCHEDULE_TOPIC_XXXX中的消息，即按照每条消息的投递时间，将延时消息投递到目标Topic中。不过，在投递之前会从commitlog中将原来写入的消息再次读出，并将其原来的延时等级设置为0，即原消息变为了一条不延迟的普通消息。然后再次将消息投递到目标Topic中。

ScheuleMessageService在Broker启动时，会创建并启动一个定时器Timer，用于执行相应的定时任务。系统会根据延时等级的个数，定义相应数量的TimerTask，每个TimerTask负责一个延迟等级消息的消费与投递。每个TimerTask都会检测相应Queue队列的第一条消息是否到期。若第一条消息未到期，则后面的所有消息更不会到期（消息是按照投递时间排序的）；若第一条消息到期了，则将该消息投递到目标Topic，即消费该消息。

将消息重新写入commitlog

延迟消息服务类ScheuleMessageService将延迟消息再次发送给了commitlog，并再次形成新的消息索引条目，分发到相应Queue。

这其实就是一次普通消息发送。只不过这次的消息Producer是延迟消息服务类ScheuleMessageService。

4 代码举例



定义DelayProducer类

```
1 public class DelayProducer {
2     public static void main(String[] args) throws Exception {
3         DefaultMQProducer producer = new DefaultMQProducer("pg");
4         producer.setNamesrvAddr("rocketmqos:9876");
5         producer.start();
6
7         for (int i = 0; i < 10; i++) {
8             byte[] body = ("Hi," + i).getBytes();
9             Message msg = new Message("TopicB", "someTag", body);
10            // 指定消息延迟等级为3级, 即延迟10s
11            // msg.setDelayTimeLevel(3);
12            SendResult sendResult = producer.send(msg);
13            // 输出消息被发送的时间
14            System.out.print(new SimpleDateFormat("mm:ss").format(new
Date()));
15            System.out.println(" ," + sendResult);
16        }
17        producer.shutdown();
18    }
19 }
```

定义OtherConsumer类

```
1 public class OtherConsumer {
2     public static void main(String[] args) throws MQClientException {
3         DefaultMQPushConsumer consumer = new
DefaultMQPushConsumer("cg");
4         consumer.setNamesrvAddr("rocketmqos:9876");
5
6         consumer.setConsumeFromWhere(ConsumeFromWhere.CONSUME_FROM_FIRST_OFFSET
);
7         consumer.subscribe("TopicB", "*");
8
9         consumer.registerMessageListener(new
MessageListenerConcurrently() {
10             @Override
11             public ConsumeConcurrentlyStatus
consumeMessage(List<MessageExt> msgs,
12             ConsumeConcurrentlyContext context) {
13                 for (MessageExt msg : msgs) {
14                     // 输出消息被消费的时间
```

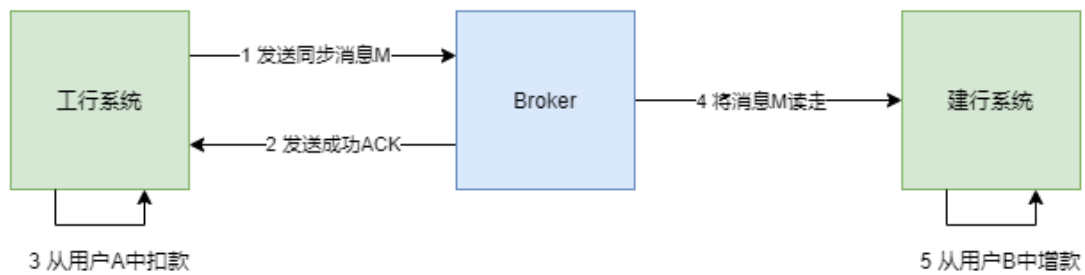
```
15         System.out.print(new
SimpleDateFormat("mm:ss").format(new Date()));
16         System.out.println(" , " + msg);
17     }
18     return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
19 }
20 });
21 consumer.start();
22 system.out.println("Consumer Started");
23 }
24 }
```

四、事务消息

1 问题引入

这里的一个需求场景是：工行用户A向建行用户B转账1万元。

我们可以使用同步消息来处理该需求场景：



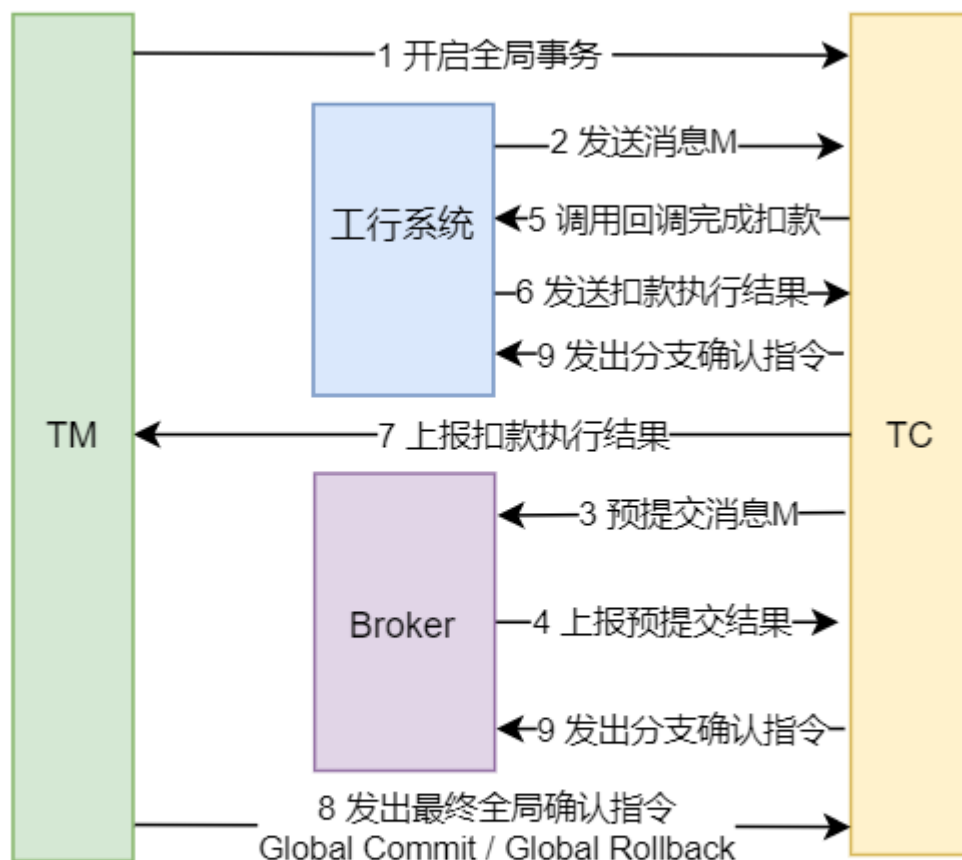
转账操作的同步消息处理方式示意图

1. 工行系统发送一个给B增款1万元的同步消息M给Broker
2. 消息被Broker成功接收后，向工行系统发送成功ACK
3. 工行系统收到成功ACK后从用户A中扣款1万元
4. 建行系统从Broker中获取到消息M
5. 建行系统消费消息M，即向用户B中增加1万元

这其中是有问题的：若第3步中的扣款操作失败，但消息已经成功发送到了Broker。对于MQ来说，只要消息写入成功，那么这个消息就可以被消费。此时建行系统中用户B增加了1万元。出现了数据不一致问题。

2 解决思路

解决思路是，让第1、2、3步具有原子性，要么全部成功，要么全部失败。即消息发送成功后，必须要保证扣款成功。如果扣款失败，则回滚发送成功的消息。而该思路即使用 **事务消息**。这里要使用 **分布式事务** 解决方案。



转账操作的事务消息处理方式示意图

使用事务消息来处理该需求场景：

1. 事务管理器TM向事务协调器TC发起指令，开启 **全局事务**
2. 工行系统发一个给B增款1万元的事务消息M给TC
3. TC会向Broker发送 **半事务消息prepareHalf**，将消息M **预提交** 到Broker。此时的建行系统是看不到Broker中的消息M的
4. Broker会将预提交执行结果Report给TC。
5. 如果预提交失败，则TC会向TM上报预提交失败的响应，全局事务结束；如果预提交成功，TC会调用工行系统的 **回调操作**，去完成工行用户A的 **预扣款** 1万元的操作
6. 工行系统会向TC发送预扣款执行结果，即 **本地事务** 的执行状态
7. TC收到预扣款执行结果后，会将结果上报给TM。

预扣款执行结果存在三种可能性：

```

1 // 描述本地事务执行状态
2 public enum LocalTransactionState {
3     COMMIT_MESSAGE, // 本地事务执行成功
4     ROLLBACK_MESSAGE, // 本地事务执行失败
5     UNKNOWN, // 不确定，表示需要进行回查以确定本地事务的执行结果
6 }
  
```

8. TM会根据上报结果向TC发出不同的确认指令

- 若预扣款成功（本地事务状态为COMMIT_MESSAGE），则TM向TC发送Global Commit指令
- 若预扣款失败（本地事务状态为ROLLBACK_MESSAGE），则TM向TC发送Global Rollback指令
- 若现未知状态（本地事务状态为UNKNOWN），则会触发工行系统的本地事务状态 [回查操作](#)。回查操作会将回查结果，即COMMIT_MESSAGE或ROLLBACK_MESSAGE Report给TC。TC将结果上报给TM，TM会再向TC发送最终确认指令Global Commit或Global Rollback

9. TC在接收到指令后会向Broker与工行系统发出确认指令

- TC接收的若是Global Commit指令，则向Broker与工行系统发送Branch Commit指令。此时Broker中的消息M才可被建行系统看到；此时的工行用户A中的扣款操作才真正被确认
- TC接收到的若是Global Rollback指令，则向Broker与工行系统发送Branch Rollback指令。此时Broker中的消息M将被撤销；工行用户A中的扣款操作将被回滚

以上方案就是为了确保 [消息投递](#) 与 [扣款操作](#) 能够在一个事务中，要成功都成功，有一个失败，则全部回滚。

以上方案并不是一个典型的XA模式。因为XA模式中的分支事务是异步的，而事务消息方案中的消息预提交与预扣款操作间是同步的。

3 基础

分布式事务

对于分布式事务，通俗地说就是，一次操作由若干分支操作组成，这些分支操作分属不同应用，分布在不同服务器上。分布式事务需要保证这些分支操作要么全部成功，要么全部失败。分布式事务与普通事务一样，就是为了保证操作结果的一致性。

事务消息

RocketMQ提供了类似X/Open XA的分布式事务功能，通过事务消息能达到分布式事务的最终一致。XA是一种分布式事务解决方案，一种分布式事务处理模式。

半事务消息

暂不能投递的消息，发送方已经成功地将消息发送到了Broker，但是Broker未收到最终确认指令，此时该消息被标记成“暂不能投递”状态，即不能被消费者看到。处于该种状态下的消息即半事务消息。

本地事务状态

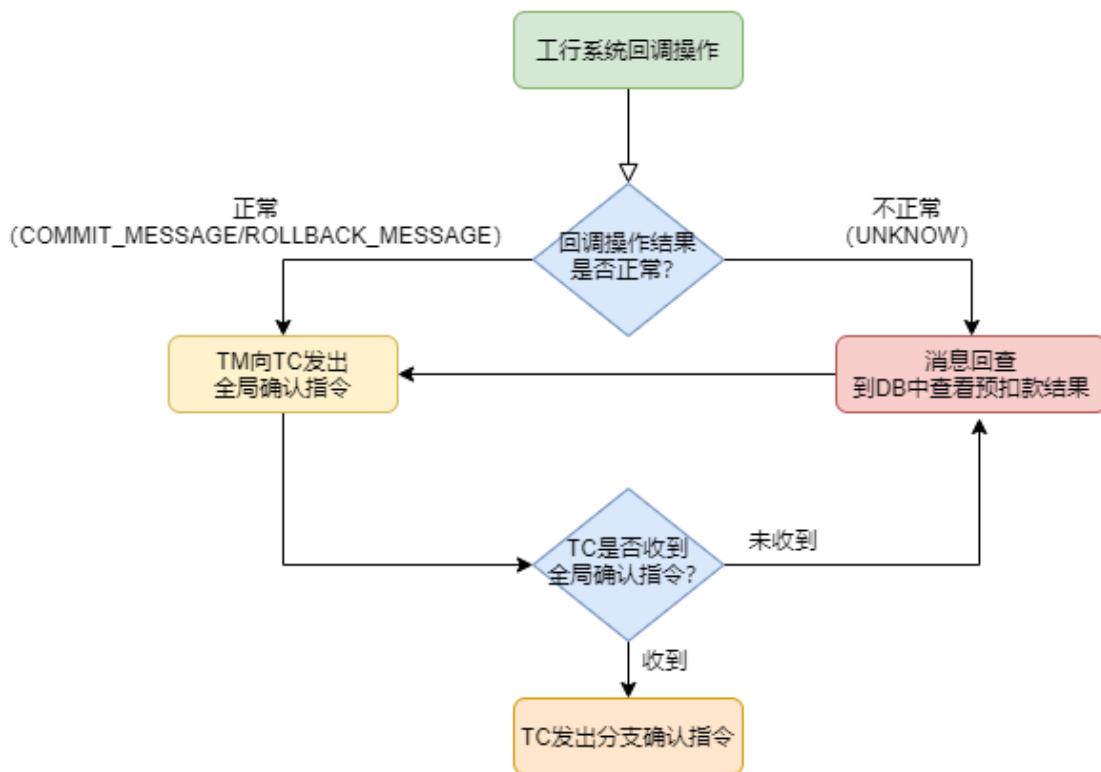
Producer [回调操作](#) 执行的结果为本地事务状态，其会发送给TC，而TC会再发送给TM。TM会根据TC发送来的本地事务状态来决定全局事务确认指令。

```

1 // 描述本地事务执行状态
2 public enum LocalTransactionState {
3     COMMIT_MESSAGE, // 本地事务执行成功
4     ROLLBACK_MESSAGE, // 本地事务执行失败
5     UNKNOW, // 不确定, 表示需要进行回查以确定本地事务的执行结果
6 }

```

消息回查



消息回查的被调用场景示意图

消息回查，即重新查询本地事务的执行状态。本例就是重新到DB中查看预扣款操作是否执行成功。

注意，消息回查不是重新执行回调操作。回调操作是进行预扣款操作，而消息回查则是查看预扣款操作执行的结果。

引发消息回查的原因最常见的有两个：

- 1) 回调操作返回UNKNWON
- 2) TC没有接收到TM的最终全局事务确认指令

RocketMQ中的消息回查设置

关于消息回查，有三个常见的属性设置。它们都在broker加载的配置文件中设置，例如：

- transactionTimeout=20, 指定TM在20秒内应将最终确认状态发送给TC, 否则引发消息回查。默认为60秒
- transactionCheckMax=5, 指定最多回查5次, 超过后将丢弃消息并记录错误日志。默认15次。
- transactionCheckInterval=10, 指定设置的多次消息回查的时间间隔为10秒。默认为60秒。

4 XA模式三剑客

XA协议

XA (Unix Transaction) 是一种分布式事务解决方案, 一种分布式事务处理模式, 是基于XA协议的。XA协议由Tuxedo (Transaction for Unix has been Extended for Distributed Operation, 分布式操作扩展之后的Unix事务系统) 首先提出的, 并交给X/Open组织, 作为资源管理器与事务管理器的接口标准。

XA模式中有三个重要组件: TC、TM、RM。

TC

Transaction Coordinator, 事务协调者。维护全局和分支事务的状态, 驱动全局事务提交或回滚。

■ *RocketMQ中Broker充当着TC。*

TM

Transaction Manager, 事务管理器。定义全局事务的范围: 开始全局事务、提交或回滚全局事务。它实际是全局事务的发起者。

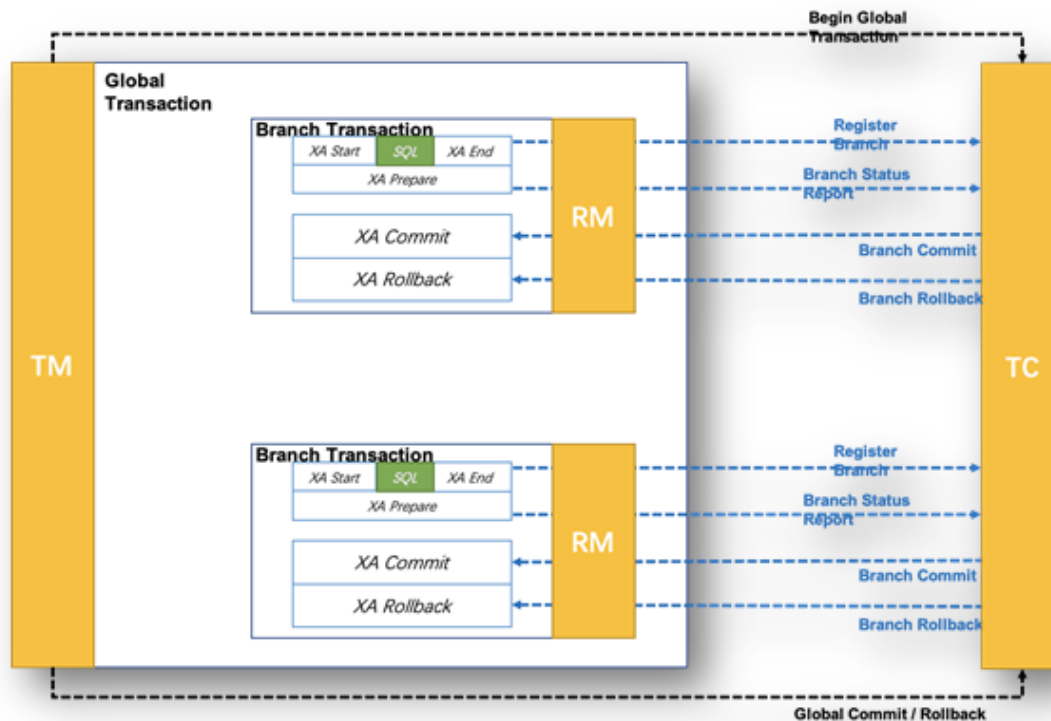
■ *RocketMQ中事务消息的Producer充当着TM。*

RM

Resource Manager, 资源管理器。管理分支事务处理的资源, 与TC交谈以注册分支事务和报告分支事务的状态, 并驱动分支事务提交或回滚。

■ *RocketMQ中事务消息的Producer及Broker均是RM。*

5 XA模式架构



XA模式是一个典型的2PC，其执行原理如下：

1. TM向TC发起指令，开启一个全局事务。
2. 根据业务要求，各个RM会逐个向TC注册分支事务，然后TC会逐个向RM发出预执行指令。
3. 各个RM在接收到指令后会在进行本地事务预执行。
4. RM将预执行结果Report给TC。当然，这个结果可能是成功，也可能是失败。
5. TC在接收到各个RM的Report后会将汇总结果上报给TM，根据汇总结果TM会向TC发出确认指令。
 - 若所有结果都是成功响应，则向TC发送Global Commit指令。
 - 只要有结果是失败响应，则向TC发送Global Rollback指令。
6. TC在接收到指令后再次向RM发送确认指令。

事务消息方案并不是一个典型的XA模式。因为XA模式中的分支事务是异步的，而事务消息方案中的消息预提交与预扣款操作间是同步的。

6 注意

- 事务消息不支持延时消息
- 对于事务消息要做好幂等性检查，因为事务消息可能不止一次被消费（因为存在回滚后再提交的情况）

7 代码举例

定义工行事务监听器

```
1 public class ICBCTransactionListener implements TransactionListener {
2
3     // 回调操作方法
4     // 消息预提交成功就会触发该方法的执行，用于完成本地事务
5     @Override
6     public LocalTransactionState executeLocalTransaction(Message msg,
7 Object arg) {
8         System.out.println("预提交消息成功：" + msg);
9         // 假设接收到TAGA的消息就表示扣款操作成功，TAGB的消息表示扣款失败，
10        // TAGC表示扣款结果不清楚，需要执行消息回查
11        if (StringUtils.equals("TAGA", msg.getTags())) {
12            return LocalTransactionState.COMMIT_MESSAGE;
13        } else if (StringUtils.equals("TAGB", msg.getTags())) {
14            return LocalTransactionState.ROLLBACK_MESSAGE;
15        } else if (StringUtils.equals("TAGC", msg.getTags())) {
16            return LocalTransactionState.UNKNOW;
17        }
18        return LocalTransactionState.UNKNOW;
19    }
20
21    // 消息回查方法
22    // 引发消息回查的原因最常见的有两个：
23    // 1)回调操作返回UNKNOW
24    // 2)TC没有接收到TM的最终全局事务确认指令
25    @Override
26    public LocalTransactionState checkLocalTransaction(MessageExt msg) {
27        System.out.println("执行消息回查" + msg.getTags());
28        return LocalTransactionState.COMMIT_MESSAGE;
29    }
30 }
```

定义事物消息生产者

```
1 public class TransactionProducer {
2     public static void main(String[] args) throws Exception {
3         TransactionMQProducer producer = new
4 TransactionMQProducer("tpg");
5         producer.setNamesrvAddr("rocketmqOS:9876");
6
7         /**
8          * 定义一个线程池
9          * @param corePoolSize 线程池中核心线程数量
10         * @param maximumPoolSize 线程池中最多线程数
11         * @param keepAliveTime 这是一个时间。当线程池中线程数量大于核心线程数量
12         是,
```

```

11      *                               多余空闲线程的存活时长
12      * @param unit 时间单位
13      * @param workQueue 临时存放任务的队列，其参数就是队列的长度
14      * @param threadFactory 线程工厂
15      */
16      ExecutorService executorService = new ThreadPoolExecutor(2, 5,
17      100, TimeUnit.SECONDS,
18      new ArrayBlockingQueue<Runnable>(2000), new
19      ThreadFactory() {
20      @Override
21      public Thread newThread(Runnable r) {
22      Thread thread = new Thread(r);
23      thread.setName("client-transaction-msg-check-thread");
24      return thread;
25      }
26      });
27
28      // 为生产者指定一个线程池
29      producer.setExecutorService(executorService);
30      // 为生产者添加事务监听器
31      producer.setTransactionListener(new ICBCTransactionListener());
32
33      producer.start();
34
35      String[] tags = {"TAGA", "TAGB", "TAGC"};
36      for (int i = 0; i < 3; i++) {
37      byte[] body = ("Hi," + i).getBytes();
38      Message msg = new Message("TTopic", tags[i], body);
39      // 发送事务消息
40      // 第二个参数用于指定在执行本地事务时要使用的业务参数
41      SendResult sendResult =
42      producer.sendMessageInTransaction(msg, null);
43      System.out.println("发送结果为: " +
44      sendResult.getSendStatus());
45      }
46      }
47      }

```

定义消费者

直接使用普通消息的SomeConsumer作为消费者即可。

```

1 public class SomeConsumer {
2
3     public static void main(String[] args) throws MQClientException {
4         // 定义一个pull消费者

```

```

5      // DefaultLitePullConsumer consumer = new
DefaultLitePullConsumer("cg");
6      // 定义一个push消费者
7      DefaultMQPushConsumer consumer = new
DefaultMQPushConsumer("cg");
8      // 指定nameServer
9      consumer.setNamesrvAddr("rocketmqOS:9876");
10     // 指定从第一条消息开始消费
11
    consumer.setConsumeFromWhere(ConsumeFromWhere.CONSUME_FROM_FIRST_OFFSET
);
12     // 指定消费topic与tag
13     consumer.subscribe("TTopic", "*");
14     // 指定采用“广播模式”进行消费，默认为“集群模式”
15     // consumer.setMessageModel(MessageModel.BROADCASTING);
16
    // 注册消息监听器
17     consumer.registerMessageListener(new
MessageListenerConcurrently() {
18
19         // 一旦broker中有了其订阅的消息就会触发该方法的执行，
20         // 其返回值为当前consumer消费的状态
21         @Override
22         public ConsumeConcurrentlyStatus
consumeMessage(List<MessageExt> msgs,
23
24         ConsumeConcurrentlyContext context) {
25             // 逐条消费消息
26             for (MessageExt msg : msgs) {
27                 System.out.println(msg);
28             }
29             // 返回消费状态：消费成功
30             return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
31         }
32     });
33
    // 开启消费者消费
34     consumer.start();
35     System.out.println("Consumer Started");
36 }
37 }
38 }

```

五、批量消息

1 批量发送消息

发送限制

生产者进行消息发送时可以一次发送多条消息，这可以大大提升Producer的发送效率。不过需要注意以下几点：

- 批量发送的消息必须具有相同的Topic
- 批量发送的消息必须具有相同的刷盘策略
- 批量发送的消息不能是延时消息与事务消息

批量发送大小

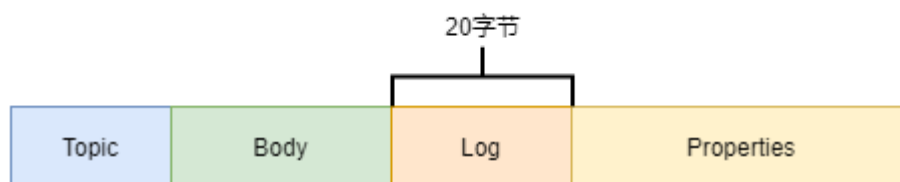
默认情况下，一批发送的消息总大小不能超过4MB字节。如果想超出该值，有两种解决方案：

- 方案一：将批量消息进行拆分，拆分为若干不大于4M的消息集合分多次批量发送
- 方案二：在Producer端与Broker端修改属性

** Producer端需要在发送之前设置Producer的maxMessageSize属性

** Broker端需要修改其加载的配置文件中的maxMessageSize属性

生产者发送的消息大小



Producer发送消息的结构

生产者通过send()方法发送的Message，并不是直接将Message序列化后发送到网络上的，而是通过这个Message生成了一个字符串发送出去的。这个字符串由四部分构成：Topic、消息Body、消息日志（占20字节），及用于描述消息的一堆属性key-value。这些属性中包含例如生产者地址、生产时间、要发送的QueueId等。最终写入到Broker中消息单元中的数据都是来自于这些属性。

2 批量消费消息

修改批量属性

```

BatchConsumer.java x
22
23 consumer.registerMessageListener(new MessageListenerConcurrently() {
24
25     @Override
26     public ConsumeConcurrentlyStatus consumeMessage(List<MessageExt> msgs,
27                                                     ConsumeConcurrentlyContext context) {
28         for (MessageExt msg : msgs) {
29             System.out.println(msg);
30         }
31         return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
32     }
33 });

```

Consumer的MessageListenerConcurrently监听接口的consumeMessage()方法的第一个参数为消息列表，但默认情况下每次只能消费一条消息。若要使其一次可以消费多条消息，则可以通过修改Consumer的consumeMessageBatchMaxSize属性来指定。不过，该值不能超过32。因为默认情况下消费者每次可以拉取的消息最多是32条。若要修改一次拉取的最大值，则可通过修改Consumer的pullBatchSize属性来指定。

存在的问题

Consumer的pullBatchSize属性与consumeMessageBatchMaxSize属性是否设置的越大越好？当然不是。

- pullBatchSize值设置的越大，Consumer每拉取一次需要的时间就会越长，且在网络上传输出现问题的可能性就越高。若在拉取过程中若出现了问题，那么本批次所有消息都需要全部重新拉取。
- consumeMessageBatchMaxSize值设置的越大，Consumer的消息并发消费能力越低，且这批被消费的消息具有相同的消费结果。因为consumeMessageBatchMaxSize指定的一批消息只会使用一个线程进行处理，且在处理过程中只要有一个消息处理异常，则这批消息需要全部重新再次消费处理。

3 代码举例

该批量发送的需求是，不修改最大发送4M的默认值，但要防止发送的批量消息超出4M的限制。

定义消息列表分割器

```

1 // 消息列表分割器：其只会处理每条消息的大小不超4M的情况。
2 // 若存在某条消息，其本身大小大于4M，这个分割器无法处理，
3 // 其直接将这条消息构成一个子列表返回。并没有再进行分割
4 public class MessageListSplitter implements Iterator<List<Message>> {
5     // 指定极限值为4M
6     private final int SIZE_LIMIT = 4 * 1024 * 1024;
7     // 存放所有要发送的消息
8     private final List<Message> messages;
9     // 要进行批量发送消息的小集合起始索引
10    private int currIndex;
11    public MessageListSplitter(List<Message> messages) {
12        this.messages = messages;

```

```

13     }
14
15     @Override
16     public boolean hasNext() {
17         // 判断当前开始遍历的消息索引要小于消息总数
18         return currIndex < messages.size();
19     }
20
21     @Override
22     public List<Message> next() {
23         int nextIndex = currIndex;
24         // 记录当前要发送的这一小批次消息列表的大小
25         int totalSize = 0;
26         for (; nextIndex < messages.size(); nextIndex++) {
27             // 获取当前遍历的消息
28             Message message = messages.get(nextIndex);
29
30             // 统计当前遍历的message的大小
31             int tmpSize = message.getTopic().length() +
message.getBody().length;
32             Map<String, String> properties = message.getProperties();
33             for (Map.Entry<String, String> entry :
properties.entrySet()) {
34                 tmpSize += entry.getKey().length() +
entry.getValue().length();
35             }
36             tmpSize = tmpSize + 20;
37
38             // 判断当前消息本身是否大于4M
39             if (tmpSize > SIZE_LIMIT) {
40                 if (nextIndex - currIndex == 0) {
41                     nextIndex++;
42                 }
43                 break;
44             }
45
46             if (tmpSize + totalSize > SIZE_LIMIT) {
47                 break;
48             } else {
49                 totalSize += tmpSize;
50             }
51
52             } // end-for
53
54             // 获取当前messages列表的子集合[currIndex, nextIndex)
55             List<Message> subList = messages.subList(currIndex, nextIndex);
56             // 下次遍历的开始索引
57             currIndex = nextIndex;

```

```

58         return subList;
59     }
60 }

```

定义批量消息生产者

```

1  public class BatchProducer {
2      public static void main(String[] args) throws Exception {
3          DefaultMQProducer producer = new DefaultMQProducer("pg");
4          producer.setNamesrvAddr("rocketmqOS:9876");
5          // 指定要发送的消息的最大大小, 默认是4M
6          // 不过, 仅修改该属性是不行的, 还需要同时修改broker加载的配置文件中的
7          // maxMessageSize属性
8          // producer.setMaxMessageSize(8 * 1024 * 1024);
9          producer.start();
10
11         // 定义要发送的消息集合
12         List<Message> messages = new ArrayList<>();
13         for (int i = 0; i < 100; i++) {
14             byte[] body = ("Hi," + i).getBytes();
15             Message msg = new Message("someTopic", "someTag", body);
16             messages.add(msg);
17         }
18
19         // 定义消息列表分割器, 将消息列表分割为多个不超出4M大小的小列表
20         MessageListSplitter splitter = new
21         MessageListSplitter(messages);
22         while (splitter.hasNext()) {
23             try {
24                 List<Message> listItem = splitter.next();
25                 producer.send(listItem);
26             } catch (Exception e) {
27                 e.printStackTrace();
28             }
29         }
30         producer.shutdown();
31     }
32 }

```

定义批量消息消费者

```

1  public class BatchConsumer {
2
3      public static void main(String[] args) throws MQClientException {
4          DefaultMQPushConsumer consumer = new
4          DefaultMQPushConsumer("cg");
5          consumer.setNamesrvAddr("rocketmqOS:9876");

```

```

6      consumer.setConsumeFromWhere(ConsumeFromWhere.CONSUME_FROM_FIRST_OFFSET
7  );
8
9      // 指定每次可以消费10条消息，默认为1
10     consumer.setConsumeMessageBatchMaxSize(10);
11     // 指定每次可以从Broker拉取40条消息，默认为32
12     consumer.setPullBatchSize(40);
13
14     consumer.registerMessageListener(new
15     MessageListenerConcurrently() {
16
17         @Override
18         public ConsumeConcurrentlyStatus
19         consumeMessage(List<MessageExt> msgs,
20         ConsumeConcurrentlyContext context) {
21             for (MessageExt msg : msgs) {
22                 System.out.println(msg);
23             }
24             // 消费成功的返回结果
25             return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
26             // 消费异常时的返回结果
27             // return ConsumeConcurrentlyStatus.RECONSUME_LATER;
28         }
29     });
30
31     consumer.start();
32     System.out.println("Consumer Started");
33 }

```

六、消息过滤

消息者在进行消息订阅时，除了可以指定要订阅消息的Topic外，还可以对指定Topic中的消息根据指定条件进行过滤，即可以订阅比Topic更加细粒度的消息类型。

对于指定Topic消息的过滤有两种过滤方式：Tag过滤与SQL过滤。

1 Tag过滤

通过consumer的subscribe()方法指定要订阅消息的Tag。如果订阅多个Tag的消息，Tag间使用或运算符(双竖线||)连接。

```
1 DefaultMQPushConsumer consumer = new  
   DefaultMQPushConsumer("CID_EXAMPLE");  
2 consumer.subscribe("TOPIC", "TAGA || TAGB || TAGC");
```

2 SQL过滤

SQL过滤是一种通过特定表达式对事先埋入到消息中的用户属性进行筛选过滤的方式。通过SQL过滤，可以实现对消息的复杂过滤。不过，只有使用PUSH模式的消费者才能使用SQL过滤。

SQL过滤表达式中支持多种常量类型与运算符。

支持的常量类型：

- 数值：比如：123, 3.1415
- 字符：必须用单引号包裹起来，比如：'abc'
- 布尔：TRUE 或 FALSE
- NULL：特殊的常量，表示空

支持的运算符有：

- 数值比较：>, >=, <, <=, BETWEEN, =
- 字符比较：=, <>, IN
- 逻辑运算：AND, OR, NOT
- NULL判断：IS NULL 或者 IS NOT NULL

默认情况下Broker没有开启消息的SQL过滤功能，需要在Broker加载的配置文件中添加如下属性，以开启该功能：

```
1 enablePropertyFilter = true
```

在启动Broker时需要指定这个修改过的配置文件。例如对于单机Broker的启动，其修改的配置文件是conf/broker.conf，启动时使用如下命令：

```
1 sh bin/mqbroker -n localhost:9876 -c conf/broker.conf &
```

3 代码举例

定义Tag过滤Producer

```
1 public class FilterByTagProducer {  
2     public static void main(String[] args) throws Exception {  
3         DefaultMQProducer producer = new DefaultMQProducer("pg");  
4         producer.setNamesrvAddr("rocketmqos:9876");  
5         producer.start();  
6         String[] tags = {"myTagA", "myTagB", "myTagC"};  
7         for (int i = 0; i < 10; i++) {
```

```

8         byte[] body = ("Hi," + i).getBytes();
9         String tag = tags[i%tags.length];
10        Message msg = new Message("myTopic", tag, body);
11        SendResult sendResult = producer.send(msg);
12        System.out.println(sendResult);
13    }
14    producer.shutdown();
15 }
16 }

```

定义Tag过滤Consumer

```

1 public class FilterByTagConsumer {
2
3     public static void main(String[] args) throws Exception {
4         DefaultMQPushConsumer consumer = new
5         DefaultMQPushConsumer("pg");
6         consumer.setNamesrvAddr("rocketmqOS:9876");
7
8         consumer.setConsumeFromWhere(ConsumeFromWhere.CONSUME_FROM_FIRST_OFFSET
9         );
10
11        consumer.subscribe("myTopic", "myTagA || myTagB");
12        consumer.registerMessageListener(new
13        MessageListenerConcurrently() {
14            @Override
15            public ConsumeConcurrentlyStatus
16            consumeMessage(List<MessageExt> msgs,
17            ConsumeConcurrentlyContext context) {
18                for (MessageExt me:msgs){
19                    System.out.println(me);
20                }
21                return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
22            }
23        });
24        consumer.start();
25        System.out.println("Consumer started");
26    }
27 }

```

定义SQL过滤Producer

```

1 public class FilterBySQLProducer {
2     public static void main(String[] args) throws Exception {
3         DefaultMQProducer producer = new DefaultMQProducer("pg");
4         producer.setNamesrvAddr("rocketmqOS:9876");

```

```

5      producer.start();
6
7      for (int i = 0; i < 10; i++) {
8          try {
9              byte[] body = ("Hi," + i).getBytes();
10             Message msg = new Message("myTopic", "myTag", body);
11             msg.putUserProperty("age", i + "");
12             SendResult sendResult = producer.send(msg);
13             System.out.println(sendResult);
14         } catch (Exception e) {
15             e.printStackTrace();
16         }
17     }
18     producer.shutdown();
19 }
20 }
21

```

定义SQL过滤Consumer

```

1  public class FilterBySQLConsumer {
2
3      public static void main(String[] args) throws Exception {
4          DefaultMQPushConsumer consumer = new
DefaultMQPushConsumer("pg");
5          consumer.setNamesrvAddr("rocketmqOS:9876");
6
7          consumer.setConsumeFromWhere(ConsumeFromWhere.CONSUME_FROM_FIRST_OFFSET
);
8
9          consumer.subscribe("myTopic", MessageSelector.bySql("age between
0 and 6"));
10
11         consumer.registerMessageListener(new
MessageListenerConcurrently() {
12             @Override
13             public ConsumeConcurrentlyStatus
consumeMessage(List<MessageExt> msgs, ConsumeConcurrentlyContext
context) {
14                 for (MessageExt me:msgs){
15                     System.out.println(me);
16                 }
17                 return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
18             }
19         });
20         consumer.start();
21         System.out.println("Consumer Started");
22     }
23 }
24

```

```
21     }
22 }
23
```

七、消息发送重试机制

1 说明

Producer对发送失败的消息进行重新发送的机制，称为消息发送重试机制，也称为消息重投机制。

对于消息重投，需要注意以下几点：

- 生产者在发送消息时，若采用 [同步或异步发送](#) 方式，发送失败 [会重试](#)，但oneway消息发送方式发送失败是没有重试机制的
- 只有普通消息具有发送重试机制，顺序消息是没有的
- 消息重投机制可以保证消息尽可能发送成功、不丢失，但可能会造成消息重复。消息重复在RocketMQ中是无法避免的问题
- 消息重复在一般情况下不会发生，当出现消息量大、网络抖动，消息重复就会成为大概率事件
- producer主动重发、consumer负载变化（发生Rebalance，不会导致消息重复，但可能出现重复消费）也会导致重复消息
- 消息重复无法避免，但要避免消息的重复消费。
- 避免消息重复消费的解决方案是，为消息添加唯一标识（例如消息key），使消费者对消息进行消费判断来避免重复消费
- 消息发送重试有三种策略可以选择：同步发送失败策略、异步发送失败策略、消息刷盘失败策略

2 同步发送失败策略

对于普通消息，消息发送默认采用round-robin策略来选择所发送到的队列。如果发送失败，默认重试2次。但在重试时是不会选择上次发送失败的Broker，而是选择其它Broker。当然，若只有一个Broker其也只能发送到该Broker，但其会尽量发送到该Broker上的其它Queue。

```
1 // 创建一个producer，参数为Producer Group名称
2 DefaultMQProducer producer = new DefaultMQProducer("pg");
3 // 指定nameServer地址
4 producer.setNamesrvAddr("rocketmqos:9876");
5 // 设置同步发送失败时重试发送的次数，默认为2次
6 producer.setRetryTimesWhenSendFailed(3);
7 // 设置发送超时时限为5s，默认3s
8 producer.setSendMessageTimeout(5000);
```

同时，Broker还具有 [失败隔离](#) 功能，使Producer尽量选择未发生过发送失败的Broker作为目标Broker。其可以保证其它消息尽量不发送到问题Broker，为了提升消息发送效率，降低消息发送耗时。

思考：让我们自己实现 [失败隔离](#) 功能，如何做？

- 1) 方案一: Producer中维护某UC的Map集合, 其key是发生失败的时间戳, value为Broker实例。Producer中还维护着一个Set集合, 其中存放着所有未发生发送异常的Broker实例。选择目标Broker是从该Set集合中选择的。再定义一个定时任务, 定期从Map集合中将长期未发生发送异常的Broker清理出去, 并添加到Set集合。
- 2) 方案二: 为Producer中的Broker实例添加一个标识, 例如是一个AtomicBoolean属性。只要该Broker上发生过发送异常, 就将其置为true。选择目标Broker就是选择该属性值为false的Broker。再定义一个定时任务, 定期将Broker的该属性置为false。
- 3) 方案三: 为Producer中的Broker实例添加一个标识, 例如是一个AtomicLong属性。只要该Broker上发生过发送异常, 就使其值增一。选择目标Broker就是选择该属性值最小的Broker。若该值相同, 采用轮询方式选择。

如果超过重试次数, 则抛出异常, 由Producer去保证消息不丢。当然当生产者出现RemotingException、MQClientException和MQBrokerException时, Producer会自动重投消息。

3 异步发送失败策略

异步发送失败重试时, 异步重试不会选择其他broker, 仅在同一个broker上做重试, 所以该策略无法保证消息不丢。

```
1 DefaultMQProducer producer = new DefaultMQProducer("pg");
2 producer.setNamesrvAddr("rocketmqos:9876");
3 // 指定异步发送失败后不进行重试发送
4 producer.setRetryTimesWhenSendAsyncFailed(0);
```

4 消息刷盘失败策略

消息刷盘超时 (Master或Slave) 或slave不可用 (slave在做数据同步时向master返回状态不是SEND_OK) 时, 默认是不会将消息尝试发送到其他Broker的。不过, 对于重要消息可以通过在Broker的配置文件设置retryAnotherBrokerWhenNotStoreOK属性为true来开启。

八、消息消费重试机制

1 顺序消息的消费重试

对于顺序消息, 当Consumer消费消息失败后, 为了保证消息的顺序性, 其会自动不断地进行消息重试, 直到消费成功。消费重试默认间隔时间为1000毫秒。重试期间应用会出现消息消费被阻塞的情况。

```
1 DefaultMQPushConsumer consumer = new DefaultMQPushConsumer("cg");
2 // 顺序消息消费失败的消费重试时间间隔, 单位毫秒, 默认为1000, 其取值范围为[10, 30000]
3 consumer.setSuspendCurrentQueueTimeMillis(100);
```

由于对顺序消息的重试是无休止的，不间断的，直至消费成功，所以，对于顺序消息的消费，务必要保证应用能够及时监控并处理消费失败的情况，避免消费被永久性阻塞。

注意，顺序消息没有发送失败重试机制，但具有消费失败重试机制

2 无序消息的消费重试

对于无序消息（普通消息、延时消息、事务消息），当Consumer消费消息失败时，可以通过设置返回状态达到消息重试的效果。不过需要注意，无序消息的重试 **只对集群消费方式生效**，广播消费方式不提供失败重试特性。即对于广播消费，消费失败后，失败消息不再重试，继续消费后续消息。

3 消费重试次数与间隔

对于 **无序消息集群消费** 下的重试消费，每条消息默认最多重试16次，但每次重试的间隔时间是不同的，会逐渐变长。每次重试的间隔时间如下表。

重试次数	与上次重试的间隔时间	重试次数	与上次重试的间隔时间
1	10秒	9	7分钟
2	30秒	10	8分钟
3	1分钟	11	9分钟
4	2分钟	12	10分钟
5	3分钟	13	20分钟
6	4分钟	14	30分钟
7	5分钟	15	1小时
8	6分钟	16	2小时

若一条消息在一直消费失败的前提下，将会在正常消费后的第 **4小时46分** 后进行第16次重试。若仍然失败，则将消息投递到 **死信队列**

修改消费重试次数

```
1 DefaultMQPushConsumer consumer = new DefaultMQPushConsumer("cg");
2 // 修改消费重试次数
3 consumer.setMaxReconsumeTimes(10);
```

对于修改过的重试次数，将按照以下策略执行：

- 若修改值小于16，则按照指定间隔进行重试
- 若修改值大于16，则超过16次的重试时间间隔均为2小时

对于Consumer Group，若仅修改了一个Consumer的消费重试次数，则会应用到该Group中所有其它Consumer实例。若出现多个Consumer均做了修改的情况，则采用覆盖方式生效。即最后被修改的值会覆盖前面设置的值。

4 重试队列

对于需要重试消费的消息，并不是Consumer在等待了指定时长后再次去拉取原来的消息进行消费，而是将这些需要重试消费的消息放入到了一个特殊Topic的队列中，而后进行再次消费的。这个特殊的队列就是重试队列。

当出现需要进行重试消费的消息时，Broker会为每个消费组都设置一个Topic名称为`%RETRY%consumerGroup@consumerGroup`的重试队列。

1) 这个重试队列是针对消息才组的，而不是针对每个Topic设置的（一个Topic的消息可以让多个消费者组进行消费，所以会为这些消费者组各创建一个重试队列）

2) 只有当出现需要进行重试消费的消息时，才会为该消费者组创建重试队列

```
mqos [root@mqOS config]# cat consumerOffset.json
{
  "offsetTable": {
    "%RETRY%cg@cg": {0:0
    },
    "somTopic@cg": {0:0,1:1,2:0,3:1
    },
    "someTopic@myConsumerGroup": {0:8,1:7,2:7,3:8
    },
    "persons@persons-group": {0:6,1:7,2:6,3:6
    },
    "%RETRY%persons-group@persons-group": {0:0
    },
    "abcxxx@persons-group": {0:7,1:6,2:3,3:3
    },
    "abc000@persons-group": {0:8,1:11,2:11,3:8
    },
    "someTopic@cg": {0:29,1:31,2:33,3:33
    },
    "TopicTest@please_rename_unique_group_name_4": {0:294,1:
    },
    "RMQ_SYS_TRANS_HALF_TOPIC@CID_RMQ_SYS_TRANS": {0:18
    },
    "%RETRY%please_rename_unique_group_name_4@please_rename
    },
    "aaaxxxaaa@persons-group": {0:3,1:5,2:4,3:2
    },
    "%RETRY%myConsumerGroup@myConsumerGroup": {0:0
    },
    "RMQ_SYS_TRANS_OP_HALF_TOPIC@CID_RMQ_SYS_TRANS": {0:12
    }
  }
}[root@mqOS config]#
```

注意，消费重试的时间间隔与[延时消费](#)的[延时等级](#)十分相似，除了没有延时等级的前两个时间外，其它的时间都是相同的

Broker对于重试消息的处理是通过[延时消息](#)实现的。先将消息保存到SCHEDULE_TOPIC_XXXX延迟队列中，延迟时间到后，会将消息投递到`%RETRY%consumerGroup@consumerGroup`重试队列中。

5 消费重试配置方式

```

consumer.registerMessageListener(new MessageListenerConcurrently() {

    @Override
    public ConsumeConcurrentlyStatus consumeMessage(List<MessageExt> msgs,
                                                    ConsumeConcurrentlyContext context) {

        try {
            // ....
        } catch (Throwable e) {
            // 以下三种情况均可引发消费重试
            return ConsumeConcurrentlyStatus.RECONSUME_LATER;
            // return null;
            // throw new RuntimeException("消费异常");
        }

        // 返回消费状态：消费成功
        return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
    }
});

```

集群消费方式下，消息消费失败后若希望消费重试，则需要在消息监听器接口的实现中明确进行如下三种方式之一的配置：

- 方式1：返回ConsumeConcurrentlyStatus.RECONSUME_LATER（推荐）
- 方式2：返回Null
- 方式3：抛出异常

6 消费不重试配置方式

```

consumer.subscribe("someTopic", "",
consumer.registerMessageListener(new MessageListenerConcurrently() {

    @Override
    public ConsumeConcurrentlyStatus consumeMessage(List<MessageExt> msgs,
                                                    ConsumeConcurrentlyContext context) {

        try {
            // ....
        } catch (Throwable e) {
            return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
        }

        // 返回消费状态：消费成功
        return ConsumeConcurrentlyStatus.CONSUME_SUCCESS;
    }
});

```

集群消费方式下，消息消费失败后若不希望消费重试，则在捕获到异常后同样也返回与消费成功后的相同的结果，即ConsumeConcurrentlyStatus.CONSUME_SUCCESS，则不进行消费重试。

九、死信队列

1 什么是死信队列

当一条消息初次消费失败，消息队列会自动进行消费重试；达到最大重试次数后，若消费依然失败，则表明消费者在正常情况下无法正确地消费该消息，此时，消息队列不会立刻将消息丢弃，而是将其发送到该消费者对应的特殊队列中。这个队列就是死信队列（Dead-Letter Queue，DLQ），而其中的消息则称为死信消息（Dead-Letter Message，DLM）。

死信队列是用于处理无法被正常消费的消息的。

2 死信队列的特征

死信队列具有如下特征：

- 死信队列中的消息不会再被消费者正常消费，即DLQ对于消费者是不可见的
- 死信存储有效期与正常消息相同，均为 3 天（commitlog文件的过期时间），3 天后会被自动删除
- 死信队列就是一个特殊的Topic，名称为`%DLQ%consumerGroup@consumerGroup`，即每个消费者组都有一个死信队列
- 如果一个消费者组未产生死信消息，则不会为其创建相应的死信队列

3 死信消息的处理

实际上，当一条消息进入死信队列，就意味着系统中某些地方出现了问题，从而导致消费者无法正常消费该消息，比如代码中原本就存在Bug。因此，对于死信消息，通常需要开发人员进行特殊处理。最关键的步骤是要排查可疑因素，解决代码中可能存在的Bug，然后再将原来的死信消息再次进行投递消费。